Thor Whalen

(...but mostly his sleepless unpaid digital ghostwriters)

Cryptanalysis guides for young codebreakers



Cryptanalysis guides for young codebreakers

How to break classical ciphers with a pencil, squared paper and your own head

Thor Whalen

(...but mostly his sleepless unpaid digital ghostwriters)

First edition, November 2026

Companion app: https://apps.thorwhalen.com/cryptology/

Cryptanalysis guides for young codebreakers
How to break classical ciphers with a pencil, squared paper and your own head

© 2026 Thor Whalen. First edition, November 2026.

Text and figures licensed under Creative Commons Attribution 4.0 (CC BY 4.0): https://creativecommons.org/licenses/by/4.0/. You may copy, share and adapt them, for any purpose, with credit.

Source, figure programs and every guide as its own PDF: https://github.com/thorwhalen/cryptology

The companion app, with tools and exercises: https://apps.thorwhalen.com/cryptology/

To my daughter Cora, and her honorary brothers Luca and Leo:

May your brilliance keep expanding until the smart machines have to learn from you, whatever tomorrow decides human intelligence is worth.

Contents

Before you start 1

Brain, pencil or computer? 3

1 Reading the clues 6

2 Frequency analysis 20

3 Keys and brute force 34

4 The index of coincidence 47

5 Repeats and key length 59

6 Cribs and patterns 72

7 Transposition detective work 88

8 Scoring and hill climbing 99

9 Two letters at a time 112

Words to know, A to Z 126

Where to go next 129

About this book 130

Before you start

How this book works, and how to use it

This book teaches you how codebreakers think: how to look at a page of scrambled letters, work out what kind of cipher made it, and then break it, with nothing more than a pencil, some squared paper and your own head.

Who it is for

It is written for curious 11 to 13 year olds, the kind who enjoy math competitions and puzzles, and for anyone older who wants the same ideas without the fuss. You do not need to know any cryptography. You also do not need any math beyond arithmetic: every grown-up idea (clock arithmetic, the sigma sign, a little probability) is introduced when it is needed, in plain words first, then in the real notation, then with a worked example.

How the book is built

There are nine guides. Each one teaches one method of attack, and each one stands on its own, so you can open the book at any guide. They all have the same shape:

1. **The big idea**, in one paragraph and a picture.
2. **Which ciphers it breaks**, and which it does not.
3. **Why it works**, with a small experiment you could do by hand.
4. **The grown-up version**: the notation and words that real cryptanalysts use.
5. **A worked example**, start to finish, with every number shown.
6. **When to try it**, and when not to.
7. **Traps and tips.**
8. **By hand or by computer**: the jobs the method involves, roughly how long each takes by hand, and the tool that does it for you.
9. **Try it yourself**: a few exercises, with the answers at the end of the guide.
10. **Words to know**, and **Going further**.

All the words from every guide are collected again, A to Z, at the back of the book.

Brain, pencil or computer?

This book teaches the methods the classical way, but it does not ask you to do everything by hand forever. Do each mechanical job by hand once, so you know what a machine does when it does it for

you; after that, let the machine do it, and spend your time on the thinking. The next chapter, *Brain, pencil or computer?*, explains why, and what that choice costs.

Which order to read them in

Read *Brain, pencil or computer?* first: it is short. Then start with Guide 1, *Reading the clues*. It shows how to tell, before trying a single key, which family a cipher belongs to, and the other guides build on that habit. After that, follow what interests you:

If you want to break	Read
Caesar, affine and substitution ciphers	Guides 2 and 3, then 6
Vigenère and other ciphers with several alphabets	Guides 4 and 5, then 6
Ciphers that move letters instead of changing them	Guides 7, then 6
Big ciphers by computer, and ciphers that work on pairs of letters	Guides 8 and 9

The app that goes with it

The book has a companion website, the cryptology app, at **apps.thorwhalen.com/cryptology**. It has tools for measuring and cracking a ciphertext, exercises with fresh texts every time, a family tree of ciphers, and a glossary. Throughout the book, green boxes like this one point to the right place in it:

> **In the app:** The method, step by step asks one question at a time about a ciphertext and leads you to its family, which is Guide 1 as a walk.

In the PDF, the names in the boxes are links. On paper, type the address above and find the tool by its name. Other tools (spreadsheets that do the arithmetic, and solvers that just give the answer) are in each guide's footnotes.

For the adult helping

The best way to use this book with a young reader is to let them do the work. Read a guide's big idea together, then hand over the pencil for the worked example and the exercises. Ask questions rather than giving answers: "What did you count?" and "What would English look like here?" are more useful than any hint. Breaks help: forty minutes of real concentration is a lot.

Brain, pencil or computer?

Which parts of codebreaking to do by hand, which to hand over, and what each choice costs

By the end you know which parts of codebreaking are worth doing by hand once, which are worth handing to a machine every time, and why that choice is less simple than either side likes to say.

Long division

Suppose you need to know what 7,834 divided by 46 is. You could do it with a pencil: it takes a few minutes and some care, and you might slip a digit. Or you could use a calculator, which takes three seconds and does not slip.

Nobody needs the answer to a division for its own sake. You want it because it tells you something: how many buses the school trip needs, or whether a cipher's length divides by 5. Doing it by hand slows down that purpose, and every other purpose you had that afternoon. So I use the calculator, and I think you should too.

That is the idea behind this book. Use the tools you have to the fullest, because every chore a tool takes off you frees your mind for the part that needs a mind.

Every old way was once a new technology

People have argued about this for a long time. In Plato's dialogue *Phaedrus*, written about 2,400 years ago, Socrates worries that writing will make people forgetful: they will trust marks on a page instead of their own memory [1]. He was not wrong. Most of us remember far less by heart than people who could not read. But writing gave us libraries, science, and this book.

About 1,900 years later, the abbot Johannes Trithemius wrote *In Praise of Scribes*, arguing that monks should keep copying books by hand rather than trust the new printing press. He had it printed [2].

Writing, printing, even language itself: each was once the new technology, and each one cost us something and gave us more. Whatever you think of as "the old way" was someone else's new way.

Codebreakers have always used machines

Codebreaking is where this argument was settled first. During the Second World War, the codebreakers at Bletchley Park in England did not break Enigma with pencils alone. Alan Turing, whose ideas still define what a computer is [3], designed the Bombe, a machine that checked possible Enigma settings far faster than any room of people could [4]. To break a different German cipher, the engineer

Tommy Flowers built Colossus, one of the first electronic computers in the world [5]. Some of the first computers were built for exactly this job: breaking codes.

In any real situation today, a codebreaker gives the work to a computer. The programs exist, and with AI they need less and less help from a person. If all you want is the answer, how you get it does not matter: hand the ciphertext to a good solver. Guide 8, *Scoring and hill climbing*, shows how such programs work.

So why learn any of it?

Because some of us want to know how it works, and you cannot judge a tool you do not understand. Doing something the slow way, once, brings the idea close. You feel why E stands out in a count of letters when you have made the tally marks yourself.

But the slow way has a cost too. Every hour spent counting letters is an hour not spent seeing the whole picture: which ciphers exist, how they relate, which attack fits which. The slow way brings one idea close and pushes the bird's-eye view further into the future.

So here is the rule this book follows: **do each mechanical task by hand once, so you know what the machine is doing for you. After that, let the machine do it.** Count the letters of one ciphertext with a pencil. Then never again.

What a tool costs

A tool is not free. We pay for it with our own skills. Someone who always uses a calculator gets slower at mental arithmetic. That is called **deskilling**, and it is real.

But deskilling in one thing usually comes with **upskilling** in another. Freed from the chore, you spend your attention, your time and your creativity on something else, and you learn the skills that the new thing needs. A codebreaker who no longer counts letters by hand has time to learn statistics, or programming, or how a whole family of ciphers fits together.

So be aware of what a tool takes from you. But be at least as aware of what it costs to refuse it: all the time spent doing by hand what a machine could do, and everything you could have learned in that time instead.

How this book helps you choose

Each guide teaches its method the classical way, with explanations, worked examples and exercises. It also has a section called **By hand or by computer**: a table of the jobs the method involves, roughly how long each takes by hand, and the tool that does it for you. Tools come in three kinds:

- **Tools to learn with**: the cryptology app's tools and exercises, and a set of spreadsheets that do the arithmetic while you watch every step. They show their working, so you can see what the method does.
- **Tools that just do it**: solvers that take a ciphertext and return the answer. Use them when the answer is all you need.
- **Your own programs**: the best tool of all is one you wrote yourself, because then you understand it completely.

The green boxes in the guides point into the app, and the footnotes point to the other tools. Even the learning tools are a stepping stone. They are for people who want, or need, to know how things work.

For the adults

With AI, intelligence is becoming something anyone can switch on, like electricity. We are entering that age very fast; in my opinion, too fast, because it is not clear what we should be teaching our children. How to make the best use of machine intelligence? How to manage swarms of AI agents? More manual skills, leaving the thinking to machines, when robots may take over manual work too? It is hard to tell.

This book was composed around an app I made to get some kids I was teaching from meeting the ideas to playing with them, quickly. Should they count letters by hand? Once, yes, so that they appreciate what the computer does for them. And so on for every part of cryptanalysis.

I am betting that counting how many times each letter appears in a ciphertext is a waste of a child's time if they do it more than once. That is the philosophy of this book. A cryptanalysis curriculum, I believe, should be built around the best tools available, not around pencil and paper, with the pencil kept for the moments when it teaches something a tool cannot.

Words to know

- **Deskilling**: losing a skill because a tool now does that job for you.
- **Upskilling**: learning a new skill, often in the time a tool has freed.
- **Solver**: a program that takes a ciphertext and returns the answer.
- **Bombe**: the machine Alan Turing designed at Bletchley Park to find Enigma settings.
- **Colossus**: one of the first electronic computers, built by Tommy Flowers at Bletchley Park to break the Lorenz cipher.

References

1. Plato. Phaedrus, 274c-275b. Translated by Benjamin Jowett. Project Gutenberg.
2. Trithemius J. In Praise of Scribes (De laude scriptorum), 1492; see Johannes Trithemius, Wikipedia.
3. Turing AM. On computable numbers, with an application to the Entscheidungsproblem. Proceedings of the London Mathematical Society. 1937;s2-42(1):230-265.
4. Bombe, Wikipedia.
5. Colossus computer, Wikipedia.

Guide 1

Reading the clues

What a ciphertext tells you before you try a single key

By the end you can look at a ciphertext, take six quick measurements, and say which family of cipher made it, before you try a single key.

The big idea

A detective does not arrest the first person they meet. They look at the footprints, the mud on the boots, the time on the stopped clock, and only then do they name a suspect. Codebreakers work the same way. Every cipher changes a message, but no cipher changes everything. Some keep the letters and only move them. Some keep the shape of English but rename every letter. Some need the message to have an even number of letters. Some never use the letter J. These leftovers are clues, and you can read them without knowing the key. Taking five minutes to read them saves you hours of trying the wrong attack.



Figure 1.1: A mystery ciphertext at the top, with an arrow down to six colored cards: which symbols appear, how long the message is, the shape of the letter counts, the index of coincidence, doubled letters, and whether spaces are kept.

Which ciphers it helps with

Reading the clues does not break any cipher by itself. It tells you which attack to use next, so it helps with every cipher.

Cipher family	What the clues tell you	Which clue gives it away
Morse, binary, Baconian	It is an encoding, not a cipher: decode it, then start again	Only two or three different symbols
Polybius, Nihilist, ADFGVX	Letters were turned into pairs of symbols	Digits, or only the letters A D F G V X
Transposition (rail fence, columns)	Letters kept, only their order changed	Letter counts look exactly like English
Caesar	One alphabet, slid along	Counts look like English moved sideways
Affine, keyword substitution	One alphabet, scrambled	Counts as bumpy as English, but on the wrong letters

Cipher family	What the clues tell you	Which clue gives it away
Vigenère, Beaufort	Several alphabets taking turns	Counts flatter than English; all 26 letters appear
Playfair and other 5 x 5 squares	Letters encrypted two at a time	Even length, J never appears, no pair is a double
Hill	Letters mixed in blocks	Even length (or a multiple of 3), very flat counts

What the clues cannot do: tell you the key, or tell apart two ciphers that leave the same leftovers (an affine cipher and a keyword substitution look the same from the outside). For that you need the attacks in the later guides.

Why it works

Think about what each kind of cipher actually does to a message.

A **transposition** shuffles the letters, like shaking the tiles of a word game in a bag. The tiles are still the same tiles. If the message had 24 copies of E before, it still has 24 copies of E after.

A **substitution** keeps every letter in its place but gives it a new name. Every E becomes, say, L. So the ciphertext has 24 copies of L. The pattern of “one letter very common, a few fairly common, some rare” survives, only on different letters.

A **polyalphabetic** cipher like Vigenère uses several alphabets in turn. The first E might become L, the next E might become S, the next one K. The 24 copies of E get spread over several letters, and the tall bars get flattened.

Here is a pencil-and-paper experiment. Take any paragraph from a book and count how many times each letter appears. You will find E near the top, then T, A, O, I, N, and you will find almost no J, Q, X or Z. That pattern is a fingerprint of English. Ciphers smudge the fingerprint in different ways, and the way it is smudged tells you which cipher did it.

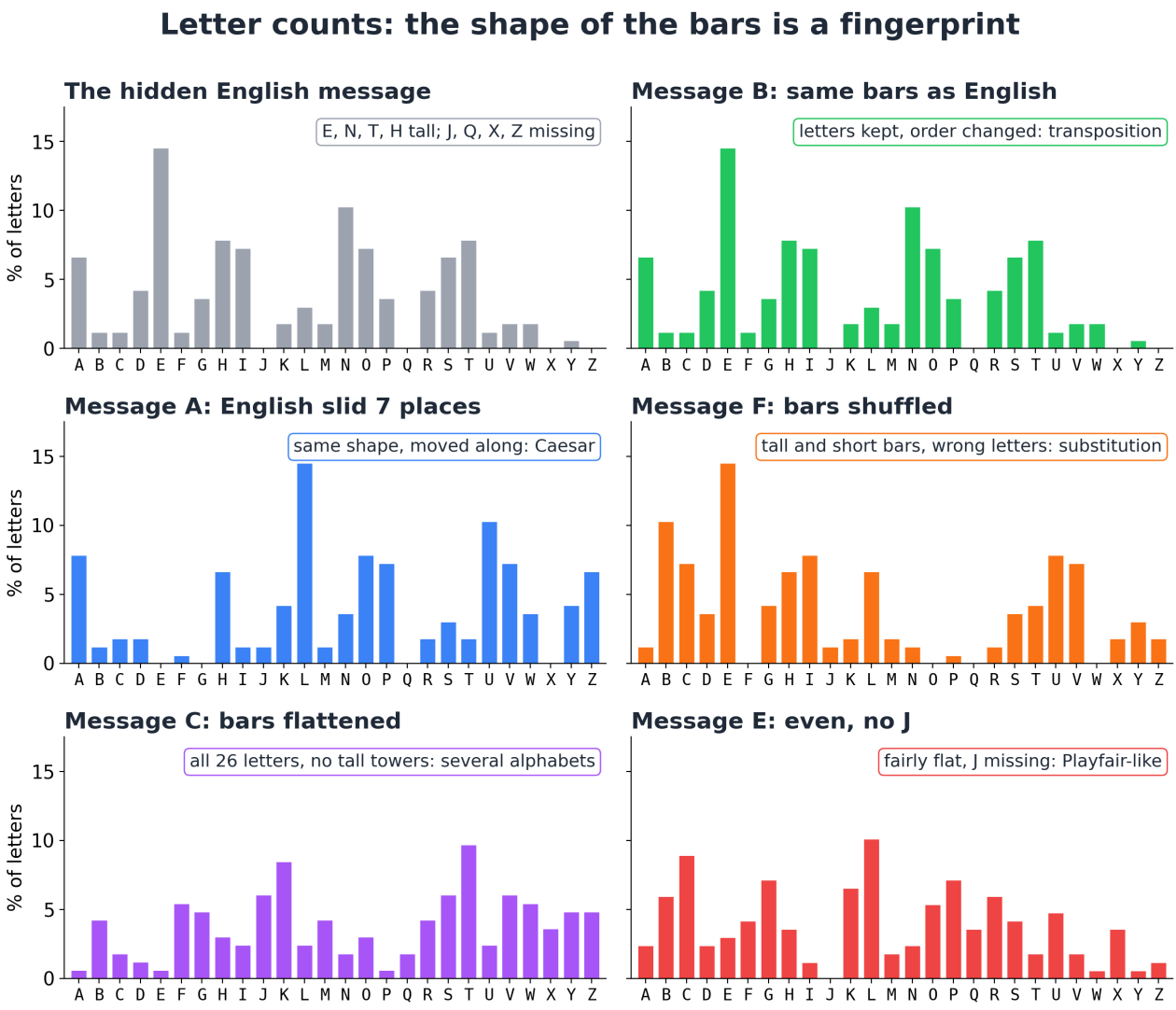


Figure 1.2: Six bar charts of letter counts. The hidden English message and message B have identical bars. Message A has the same bars moved seven letters to the right. Message F has bumpy bars on the wrong letters. Message C is flatter, with all 26 letters. Message E is fairly flat with no J.

The six clues, one by one

Clue 1: which symbols appear?

Look before you count. Write down every different symbol the ciphertext uses.

- **Two or three symbols** (dots and dashes, zeros and ones, A and B in groups of five) mean an **encoding** such as Morse code, binary or the Baconian code. An encoding has no secret key. Decode it, and then start the clue-reading again on what comes out, because a setter often hides a real cipher under an encoding.
- **Digits 1 to 5 in pairs** mean a **Polybius square**: a 5 x 5 grid where each letter is named by its row and column. The pair 15 means row 1, column 5.
- **Larger two-digit numbers**, often above 55, suggest the **Nihilist cipher**, which adds a key to

Polybius numbers.

- **Only A, D, F, G and X** (or A, D, F, G, V and X) is the famous signature of the **ADFGX** and **ADFGVX** ciphers from World War I.
- **25 letters, with J never appearing** suggests a cipher built on a 5 x 5 square, which only has room for 25 letters (usually I and J share a cell). Playfair, bifid and four-square all do this.
- **More than 26 different symbols** suggests a **homophonic** cipher, where common letters get several symbols each.
- **Pictures or odd marks** are usually a plain substitution in disguise: give each symbol a letter of your own and solve it as a substitution.
- **Ordinary letters A to Z**: keep measuring.

Step one: look at which symbols appear

What you see	The clue	Suspect
`- .... . .-.. .. --. ....`	only dots, dashes and gaps	**Morse code (decode it first)**
`10011 00111 00100 01011`	only two symbols, in groups of 5	**binary or Baconian (an encoding)**
`44 23 15 31 24 22 23 44`	digits 1 to 5, in pairs	**Polybius square**
`68 26 48 57 46 50 35 59`	two-digit numbers, often above 55	**Nihilist cipher**
`GAFVA DAFDA DDGGA GADDD`	only A D F G V X	**ADFGVX (or ADFGX without V)**
`PBLSN CBPAH PTDLL RCSLE`	25 letters, J never appears, even length	**a 5 x 5 square: Playfair, bifid, four-square**
`YVKQW MMHNT IYJYK JDKWQ`	all 26 letters	**keep measuring: length, shape, IoC**

Figure 1.3: A table drawn as colored rows. Left column: samples such as dots and dashes, groups of zeros and ones, digit pairs, only A D F G V X, and letters with no J. Middle column: the clue. Right column: the suspected cipher.

Clue 2: how long is it?

Count the letters, then break the count into its **factors** (the whole numbers that divide it exactly).

- An **odd** length rules out Playfair, four-square and the 2 x 2 Hill cipher. All three work on pairs of letters, so their ciphertexts always have an even length.

- A length that divides neatly into a rectangle is a hint for a **columnar transposition**, which writes the message into a grid. If the length is 165, the grid might be 5 by 33, 11 by 15 or 15 by 11.
- A length that is a multiple of 3 allows a 3 x 3 Hill cipher. A multiple of 25 suggests a cipher called Cadenus.

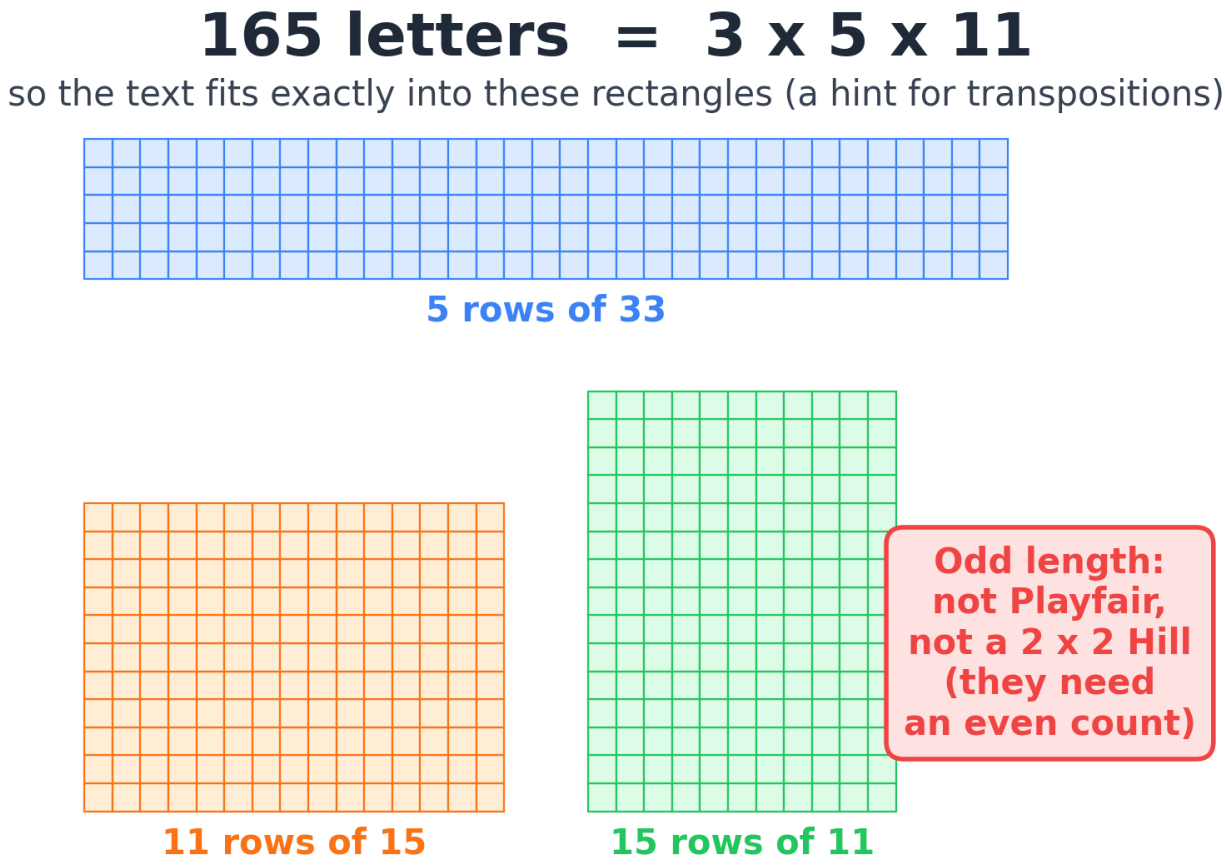


Figure 1.4: The number 165 broken into 3 times 5 times 11, with three grids of squares that hold exactly 165 letters: 5 rows of 33, 11 rows of 15 and 15 rows of 11. A red note says an odd length rules out Playfair and the 2 by 2 Hill cipher.

Clue 3: the shape of the letter counts

Draw a bar chart: one bar for each letter, as tall as the number of times it appears. Then ask which of these four shapes you see.

1. **Exactly like English.** E tallest, then T, A, O, I, N; J, Q, X, Z almost absent. The letters are real; only their order changed. Suspect a **transposition**.
2. **English slid sideways.** The same skyline of tall and short bars, but moved along the alphabet. Suspect a **Caesar** cipher. A good landmark: English has a run of rare letters at the end, V W X Y Z. In a Caesar ciphertext, that run of near-empty bars appears somewhere else.
3. **Bumpy, but on the wrong letters.** A few very tall bars and several empty ones, like English, but the tall bars sit on letters such as B or U, and no slid pattern fits. Suspect an **affine** or **keyword**

substitution.

4. **Flattened.** No tower stands far above the rest, and almost every letter appears. Suspect **several alphabets** (Vigenère and its cousins) or a cipher that works on pairs.

Clue 4: the index of coincidence

Shape is easy to see but hard to compare. The **index of coincidence** (IoC) turns the shape into one number. It answers a simple question: if you pick two letters from the text at random, what is the chance they are the same letter?

In English, letters repeat a lot (so many E's and T's), so the chance is about 0.067, roughly 1 in 15. If every letter were equally likely, the chance would be 1 in 26, about 0.038. A substitution or a transposition does not change how often letters repeat, so they keep the English value. Several alphabets spread the repeats out, so the IoC drops toward 0.038. Guide 04, The index of coincidence, explains how to use this number to find a Vigenère key's length.

Clue 5: doubled letters

English is full of doubled letters: LL, EE, SS, OO, TT. A substitution keeps them (LL becomes ZZ). A transposition breaks most of them up but makes new ones by chance. The interesting clue is a double that **never** appears where you would expect it.

Playfair cannot encrypt a pair of identical letters, so the setter splits doubles with an X before encrypting. The result: if you cut a Playfair ciphertext into pairs (letters 1 and 2, letters 3 and 4, and so on), **no pair is ever two copies of the same letter**. Doubles can still appear across the boundary between two pairs. In a long text with no in-pair doubles at all, Playfair is the prime suspect.

Clue 6: are the spaces kept?

If the ciphertext keeps its word breaks, you have a gift. One-letter words are almost always A or I. The commonest three-letter word is THE. A word like XYZZX has the same letter pattern as a real word you can search for. Most competition ciphers remove the spaces or group the letters in fives, precisely to hide these clues. When they are left in, start there (Guide 02, Frequency analysis, and Guide 06, Cribs and patterns, show how).

> **In the app:** Counts measures a ciphertext for you: its length, every letter's count beside English's, the commonest letters and the IoC. The method, step by step asks the same questions one at a time and says where each answer leads.

The grown-up version

Codebreakers have names for these measurements.

Letter frequency. The frequency of a letter is how often it appears, usually as a fraction or percentage of all letters. If E appears 24 times in 165 letters, its frequency is $24/165 \approx 0.145$, or 14.5%. The sign $\approx$ means "is approximately equal to".

Factorization. Writing a number as a product of primes, such as $165 = 3 \times 5 \times 11$ or $168 = 2 \times 2 \times 2 \times 3 \times 7$. Every rectangle that holds the text exactly has sides made from these factors.

Index of coincidence. If a text has N letters, and a particular letter appears f times, then the number of ways to pick two copies of that letter is $f(f-1)/2$, and the number of ways to pick any two letters is $N(N-1)/2$. Add up the first count over all 26 letters and divide by the second (the halves cancel):

$$\text{IoC} = \frac{\sum f(f-1)}{N(N-1)}$$

The symbol $\sum$ (the Greek capital letter sigma) means "add this up for every letter". So for each letter you work out $f(f-1)$, add the 26 results, and divide by $N(N-1)$.

Here is a tiny example you can check in your head: the text AABB has $N = 4$, $f = 2$ for A and $f = 2$ for B. So the IoC is $\frac{2\times1+2\times1}{4\times3} = \frac{4}{12} \approx 0.333$. One time in three, two letters picked from AABB match.

The two numbers to remember:

$$\kappa_{\text{English}} \approx 0.067 \qquad \kappa_{\text{random}} = \tfrac{1}{26} \approx 0.038$$

The Greek letter κ (kappa) is the traditional name for these benchmark values. They are guides, not sharp lines: a short text wobbles. Below about 100 letters, trust the overall shape more than the third decimal place.

Monoalphabetic and polyalphabetic. A monoalphabetic cipher uses one cipher alphabet for the whole message (Caesar, affine, substitution). A polyalphabetic cipher switches between several (Vigenère, Beaufort). The IoC is the quickest way to tell them apart.

Worked example: a lineup of six suspects

Here is an original message, written for this guide:

```
the lighthouse keeper counted the ships every evening and wrote their names in
a small green book one night a ship with no name and no lights slipped past the
rocks and vanished into the fog before dawn
```

It has 165 letters. I encrypted it six different ways. Pretend you do not know which is which. Here are the first 50 symbols of each.

```
A: AOLSP NOAOV BZLRL LWLYJ VBUAL KAOLZ OPWZL CLYFL CLUPU NHUKD
B: ETEEN HPRNN TIESG BNHIH MNHIP HKVHT FFALH KRTES YIDER SMROE
C: YVKQW MMHNT IYJYK JDKWQ UZBZJ RZMSY MWVXS BJFEJ JKSWT LOTIK
D: 44 23 15 31 24 22 23 44 23 34 45 43 15 25 15 15 35 15 42 13 34 45 33 44 15
E: PBLSN CBPAH PTDLL RCSLE GHZUS FFQRC PRPVX LXCBX CXGLK IDOKG
F: IUEYV SUIUC JHEXE EDEGN CJBIE TIUEH UVDHE KEGPE KEBVB SLBTM
```

Now take the measurements. These are the real counts for the whole of each message.

	Symbols used	Length	Missing letters	Commonest letters	IoC	Doubles inside pairs
A	22 letters	165	E G Q X	L 24, U 17, A 13, O 13	0.067	(odd length)

	Symbols used	Length	Missing letters	Commonest letters	IoC	Doubles inside pairs
B	22 letters	165	J Q X Z	E 24, N 17, T 13, H 13	0.067	(odd length)
C	all 26	165	none	T 16, K 14	0.046	(odd length)
D	digits 1 to 5	165 pairs		15 appears 24 times		
E	25 letters	168	J	L 17, C 15	0.050	0 out of 84
F	22 letters	165	F O Q W	E 24, B 17, I 13, U 13	0.067	(odd length)

Now read the clues, one suspect at a time.

Message D is the easiest. It uses only the digits 1 to 5, always in pairs. That is a Polybius square. Notice that the pair 15 appears 24 times, exactly as often as E appears in the plaintext: Polybius is really a substitution that writes each letter as two digits. Replace each pair with a letter of your choice and solve it as a substitution.

Message E has an even length (168, when the plaintext had 165: the setter added padding), uses 25 letters with no J, and has an IoC of 0.050, between English and random. Cut it into pairs and look for doubles: there are five doubled letters in the text (`LL`, `FF`, `EE`, `CC`, `CC`), but every one of them straddles the boundary between two pairs. Not one of the 84 pairs is a double. Prime suspect: **Playfair**.

Message C uses all 26 letters, its bars are flat (no letter appears more than 16 times in 165), and its IoC is 0.046, well below English. That points to **several alphabets**. As a test, take every third letter, then every third letter starting from the second, then from the third. The average IoC of these three slices jumps to 0.068, right at the English value. For slices of 2, 4, 5 and 7 it stays between 0.042 and 0.051. So this is a **Vigenère-type cipher with a key of length 3** (guides 04 and 05 show how to finish it).

That leaves A, B and F. All three have an IoC of exactly 0.067: one alphabet each. So which is which?

Message B's commonest letters are E, N, T, H, and J, Q, X, Z never appear. That is just what English looks like. Its bar chart is identical to the plaintext's. The letters are all real; only the order is scrambled. Suspect: **transposition**. The length, $165 = 3 \times 5 \times 11$, says to try grids that are 3, 5, 11 or 15 columns wide. (It was, in fact, a 5-column transposition.)

Message A's commonest letter is L, and E and G never appear. Compare its bar chart with English, slid along. English's tallest bar is E. If E moved to L, that is a slide of 7 places: count them, F, G, H, I, J, K, L. Check the landmarks: under a slide of 7, English's rare letters X and Z would move to E and G. Those are exactly the two letters missing from message A. English's common T would move to A, and A has 13 copies, one of its tallest bars. The whole skyline fits. Suspect: **Caesar, shift 7**. Try it: moving every letter back 7 turns `AOLSP NOAOV` into `theli ghtho`. Solved.

Message F's commonest letter happens to be E, but its second-tallest bar is B, and O, a very common English letter, never appears at all. No slide of the English skyline fits. Bumpy bars on the wrong letters: suspect a **keyword substitution** (or an affine cipher). Use frequency analysis and common words to finish (Guide 02).

Six suspects, six different ciphers, and you named every family without trying a single key.

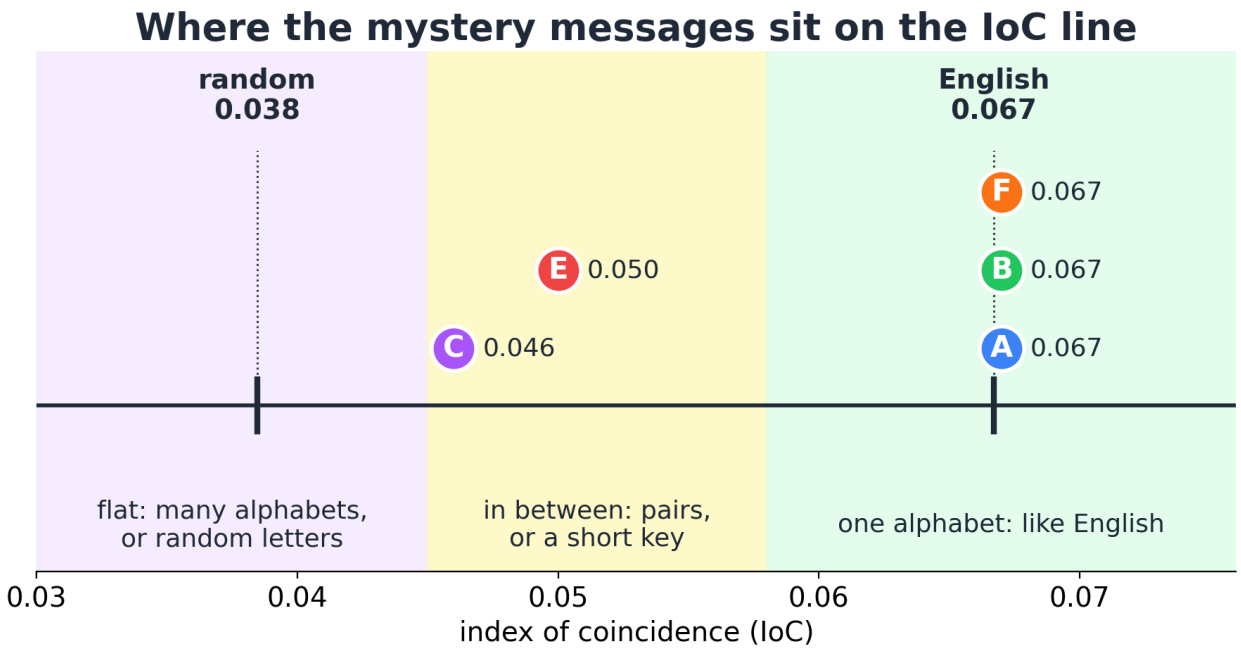


Figure 1.5: A number line for the index of coincidence, from 0.03 to 0.076. Random text is marked at 0.038 and English at 0.067. Messages A, B and F sit stacked at 0.067, message E at 0.050 and message C at 0.046.

A decision flowchart

Here is the whole routine on one page. Start at the top and answer each question in order.

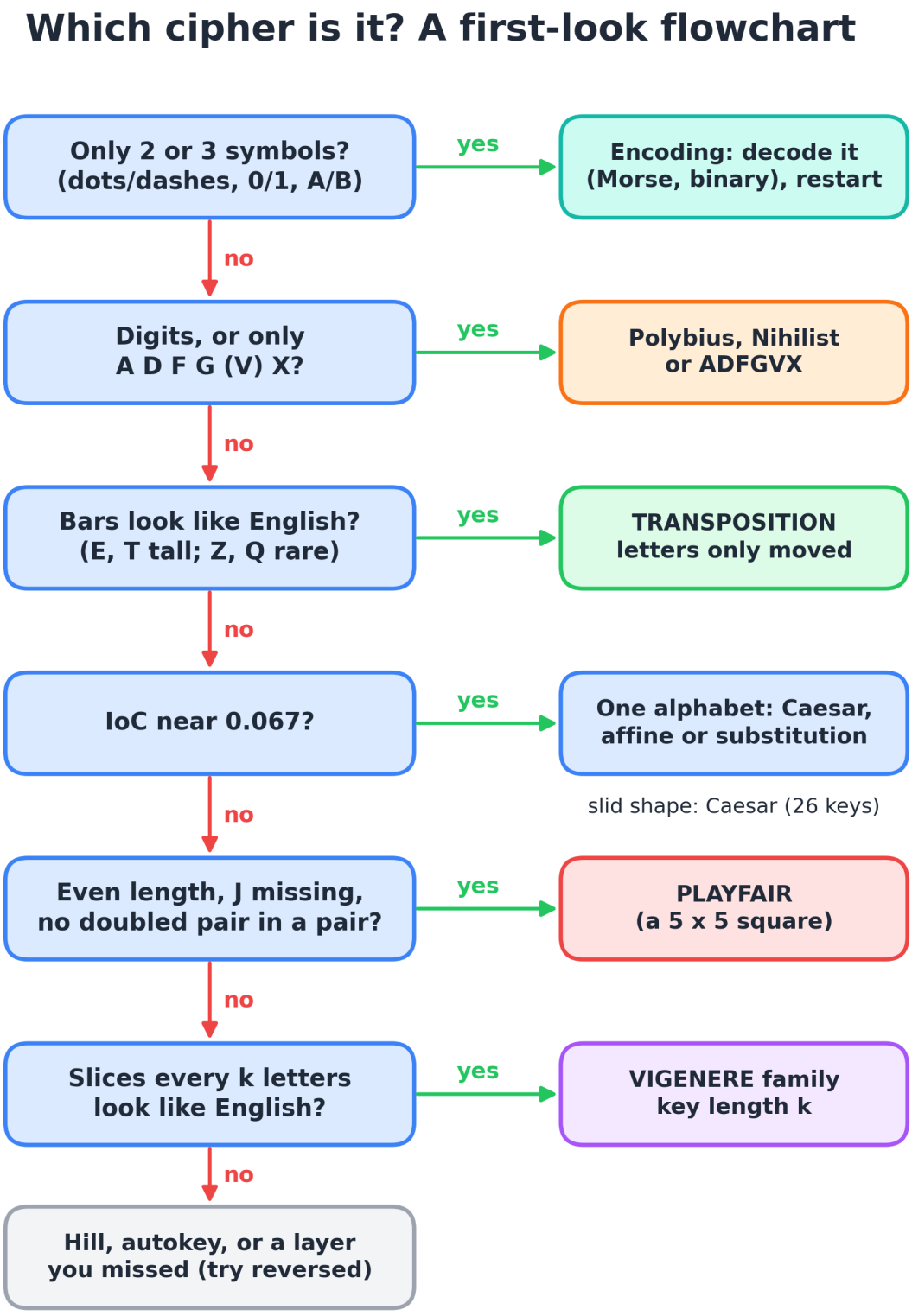


Figure 1.6: A flowchart. Questions on the left, in order: only two or three symbols? digits or only A D F G V X? bars like English? IoC near 0.067? even length with no J and no doubled pair? do slices look like English? Each yes leads right to a suspect: an encoding, Polybius or ADFGVX, transposition, a one-alphabet cipher, Playfair, the Vigenère family. The last no leads to Hill, autokey, or a hidden extra layer.

> **In the app:** The family tree shows every family of cipher, how they are related, and the clues each one leaves.

When to use it, and when not to

Always use it first. Every time you meet a new ciphertext, before you guess anything. It takes five minutes and points you at the right attack.

Use it again when you are stuck. If you have been trying Vigenère keys for half an hour with no luck, go back and remeasure. Did you check that the IoC was actually low? Did you assume the length was even without counting?

Do not treat it as a verdict. The clues narrow down the suspects; they do not convict. A short message can fool any measurement, and a clever setter can add a twist (see below). If the clues say "Caesar" and none of the 26 shifts reads, the clues were not wrong about everything, but something else is going on.

Traps and tips

- **Short texts wobble.** With 50 letters, an English text might easily show T above E, or have an IoC of 0.055 instead of 0.067. Lean on the shape and the missing letters more than on exact numbers.
- **Unusual texts.** A message about "jazz quizzes" has lots of Z and Q. A list of names or a text written to avoid the letter E will throw off the counts. Read the story or context that comes with the cipher.
- **Padding.** Setters often add X's or random letters to fill the last row of a grid or to make the length even. A length one or two more than you expect is not a disaster.
- **Reversed text.** Writing the message backwards does not change any count. If every clue says "transposition", try reading it backwards before anything clever.
- **Layers.** A message might be encoded in Morse and then enciphered, or substituted and then transposed. If you decode one layer and get gibberish, measure the gibberish: it is a new ciphertext with its own clues.

By hand or by computer

Reading the clues is mostly measuring, and measuring is what computers are for. The judgement, which family the measurements point to, is the part worth keeping. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
Note the symbols, the length and its factors	2 to 5 minutes	Counts[2]	dCode identifier[3]

[1] Brain, pencil or computer?, the chapter before Guide 1.

[2] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English's, the IoC, the length and its factors.

[3] dCode's cipher identifier: dcode.fr/cipher-identifier.

Job	By hand	Learn with	Just do it
Count every letter	10 to 15 minutes	Counts[4], letter-count sheet[5]	dCode[6]
Work out the IoC from the counts	10 to 15 minutes, with a calculator	Counts[7], letter-count sheet[8]	dCode[9]
Name the likely cipher	a minute of thinking, once you have the numbers	n/a	dCode identifier[10], Boxentriq[11]

Try it yourself

1. A ciphertext uses only the letters A, D, F, G, V and X, always an even number of them. What is your first suspect?
2. A ciphertext of 120 letters has an IoC of 0.066, and its four commonest letters are E, T, O and A. Which family is it, and which grid widths would you try first?
3. A ciphertext's commonest letter is H, and the letters A, C and M never appear. Assuming it is a Caesar cipher, what is the shift? (Hint: which English letter is commonest, and how far is it from H?)

> **In the app:** Spot the cipher gives you texts to diagnose from their counts and IoC. The Family tree quiz and the Strategy quiz check the order of the clues.

Answers

1. ADFGVX. The six letters are its signature. (It writes each letter as two of these symbols and then only rearranges them, so the length stays even.)
2. A transposition: the IoC is English and so are the commonest letters. $120 = 2 \times 2 \times 2 \times 3 \times 5$, so it fits many grids: try widths 4, 5, 6, 8, 10 and 12 first.
3. If H stands for E, the shift is 3 (E to F, G, H). Check: a shift of 3 sends English's rare letters X, Z and J to A, C and M, which explains why exactly those three are missing. Now try the shift and see if words appear.

[4] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English's, the IoC, the length and its factors.

[5] Letter counts and IoC, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-letter-counts, to copy into Google Sheets or download for Excel.

[6] dCode's frequency analysis: dcode.fr/frequency-analysis.

[7] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English's, the IoC, the length and its factors.

[8] Letter counts and IoC, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-letter-counts, to copy into Google Sheets or download for Excel.

[9] dCode's index of coincidence: dcode.fr/index-coincidence.

[10] dCode's cipher identifier: dcode.fr/cipher-identifier.

[11] Boxentriq's cipher identifier: boxentriq.com/code-breaking/cipher-identifier.

Words to know

- **Ciphertext**: the scrambled message. **Plaintext**: the original message.
- **Encoding**: a fixed way of rewriting letters (Morse, binary) with no secret key.
- **Frequency**: how often a letter appears, as a count or a percentage.
- **Factor**: a whole number that divides another exactly; 5 is a factor of 165.
- **Index of coincidence (IoC)**: the chance that two letters picked at random from a text are the same; about 0.067 for English and 0.038 for random letters.
- **Monoalphabetic**: using one cipher alphabet throughout. **Polyalphabetic**: switching between several.
- **Transposition**: a cipher that moves letters without changing them. **Substitution**: a cipher that changes letters without moving them.
- **Padding**: extra letters added to make the length fit a grid.

Going further

- Video: How to Break an Unknown Cipher, Cryptography for Everybody (CrypTool), 15 minutes.
- Index of coincidence on Wikipedia.
- Frequency analysis on Wikipedia.
- Practical Cryptography: English letter frequencies.
- The National Cipher Challenge resources, including the BOSS training lessons.
- Simon Singh, *The Code Book* (1999), and his Black Chamber site with interactive cipher tools.

Guide 2

Frequency analysis

Counting letters to break Caesar, affine and substitution ciphers

By the end you can break a Caesar cipher by sliding a bar chart, score how English a text looks with one number, and crack a full substitution cipher using letter counts and common words.

The big idea

In English, letters do not appear equally often. E turns up about once in every eight letters. Z turns up about once in every 1,300. A simple cipher swaps each letter for another, but it swaps it the same way every time. So if E was the commonest letter in the message, then whatever E turned into is now the commonest letter in the ciphertext. Count the letters in the ciphertext, compare the counts with English, and the disguise starts to slip. This is **frequency analysis**, and it is the oldest serious codebreaking method there is.

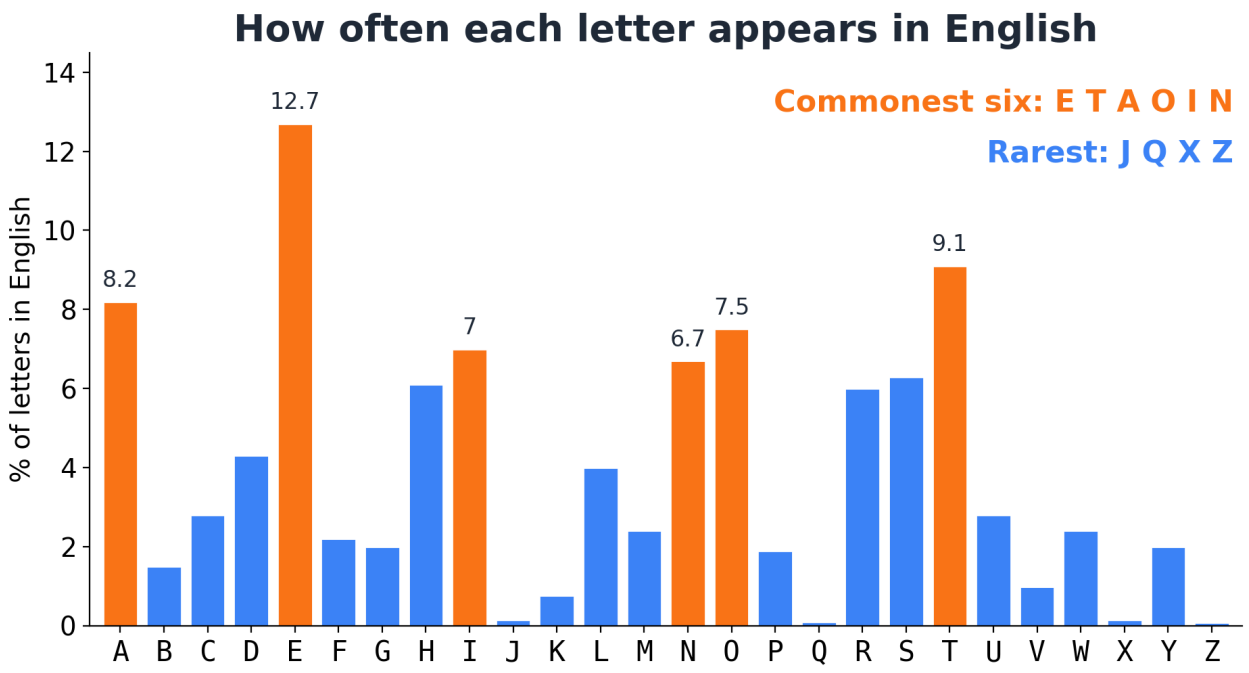


Figure 2.1: A bar chart of how often each letter appears in English. E is tallest at 12.7 percent, followed by T, A, O, I and N, highlighted in orange. J, Q, X and Z are almost zero.

A thousand years ago in Baghdad

The first person known to have written this method down was **al-Kindi**, a scholar who worked in Baghdad in the 800s. His book, *A Manuscript on Deciphering Cryptographic Messages*, was lost for centuries and rediscovered in an archive in Istanbul in 1987. His instructions were roughly: take a long text in the same language, count how often each letter appears, then count the letters of the secret message, and match the commonest with the commonest, the second with the second, and so on. That idea broke every simple substitution cipher for the next several hundred years.

Which ciphers it breaks

Cipher family	Does frequency analysis help?	How
Caesar	Yes, completely	Slide the ciphertext's bar chart until it matches English. Only 26 positions to try
Affine	Yes	Guess which letters are E and T, then solve for the key (see Guide 03, Keys and brute force)
Keyword substitution	Yes	Frequencies give the first few letters; common words and the keyword finish it

Cipher family	Does frequency analysis help?	How
General substitution	Yes, with more work	Frequencies, then common words, then letter patterns
Vigenère, Beaufort	Only after splitting	Each slice of the ciphertext is a Caesar cipher; use frequency analysis on each slice (see Guide 05)
Transposition	No	The letter counts are already exactly English; only the order changed. (That fact itself tells you it is a transposition)
Playfair, Hill	No, not on single letters	These encrypt pairs of letters, so you count pairs instead (see Guide 09)
Homophonic substitution	Weakly	Common letters get several symbols each, which flattens the counts on purpose

In the app: The family tree has a page for each of these: Caesar, affine, simple substitution and keyword substitution.

Why it works

Try this experiment. Take a page of any book and count every letter on it, making a tally mark under A, B, C and so on. It takes about twenty minutes. Then do the same with a different page from a different book. The two tallies will look remarkably alike. E will be at or near the top. T, A, O, I and N will follow. J, Q, X and Z will have only a few marks. Codebreakers remember the order of the top dozen as a nonsense phrase: **ETAOIN SHRDLU**.

Now imagine the page had been put through a substitution cipher, where every e was written as F, every t as V, and so on. Your tally for the ciphertext would have the same tall bars and short bars. They would just be standing over different letters. The tall bar over F is the clue that F means e.

This works because a simple substitution is **consistent**: the same plaintext letter always becomes the same ciphertext letter. That consistency is the weakness.

The grown-up version

Frequencies

The **frequency** of a letter is the number of times it appears, divided by the total number of letters. In a text of 152 letters where t appears 18 times, its frequency is $18/152 \approx 0.118$, or 11.8%. (The sign $\approx$ means “approximately equal to”.)

The **expected count** of a letter is how many times it would appear if the text were perfectly average English. If English uses e 12.7% of the time, then in 152 letters you would expect $152 \times 0.127 \approx 19.3$ copies of e.

Fitting a Caesar shift

A Caesar cipher slides every letter the same number of places along the alphabet. So its bar chart is the English bar chart slid along by the same number of places. To find the shift, slide the ciphertext's bar chart back one place at a time until its tall bars sit on top of English's tall bars. With paper, you can draw the ciphertext's counts on a strip and literally slide it along a printed English chart.

Chi-squared: how English is this?

Looking at two bar charts and saying "those match well" is a judgment. A computer, or a tired code-breaker at midnight, needs a number. The standard one is called **chi-squared** (say "kai squared"; χ is the Greek letter chi).

For each letter, compare the count you **observed** (O) with the count you **expected** (E) for English. Then:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Read it from the inside out:

- $O - E$ is how far off this letter's count is.
- Squaring it, $(O - E)^2$, makes every difference positive (too many and too few both count as bad) and makes big misses count much more than small ones.
- Dividing by E makes the miss relative to what was expected. Being 3 short of an expected 19 is a small surprise; seeing 3 copies when you expected 0.1 is a huge one.
- The symbol $\sum$ (capital sigma) means "add this up over all 26 letters".

A **low** score means the counts look like English. A **high** score means they do not. A perfect, impossible match would score 0. The exact value of a good score depends on the length of the text; what matters is that the right answer scores far lower than all the wrong ones.

To use it on a Caesar cipher, try all 26 shifts. For each one, shift the ciphertext back, count the letters, and work out χ^2. The shift with the lowest score is almost always the key.

> **In the app:** Caesar slides English over your text's letter counts. Its Disk view turns a cipher wheel, and its Scores view scores every shift, the way a computer would.

Worked example 1: a Caesar cipher, by sliding and by scoring

Here is a ciphertext I made from an original sentence:

```
PGPCJ XZCYT YRESP MLVPC ZYZFC DECPP EDPWW DESPQ TCDEW ZLQZQ
ESPOL JEZLW LCRPM WLNVN CZHES PNCZH HLTED ZYESP HTYOZ HDTWW
LYOAL JDHTE SLDST YJMFE EZYES LETEQ ZFYOD ZXPHS PCPTY ESPEZ
HY
```

Step 1: count. There are 152 letters. The commonest are E (18 times), P (16), Z (14), then Y, L (11 each), and C, S (10 each).

Step 2: the obvious guess, and why it fails. The commonest ciphertext letter is E. If E stands for e, the shift is 0, and the ciphertext would already be English. It is not. So in this message, e is not the

commonest letter. That happens in short texts: here, as you will see, the message has more t's than e's.

Step 3: try the next guess. Suppose P stands for e. From e to P is 11 places (f, g, h, i, j, k, l, m, n, o, P). Then E would stand for the letter 11 places before E, which is t. And Z would stand for o. Commonest letters t, e, o: that sounds like English.

Step 4: check with the bar charts. The picture below shows the ciphertext moved back by 0, 4 and 11 places, laid over English. Only the shift of 11 puts the ciphertext's tall bars on English's tall bars.

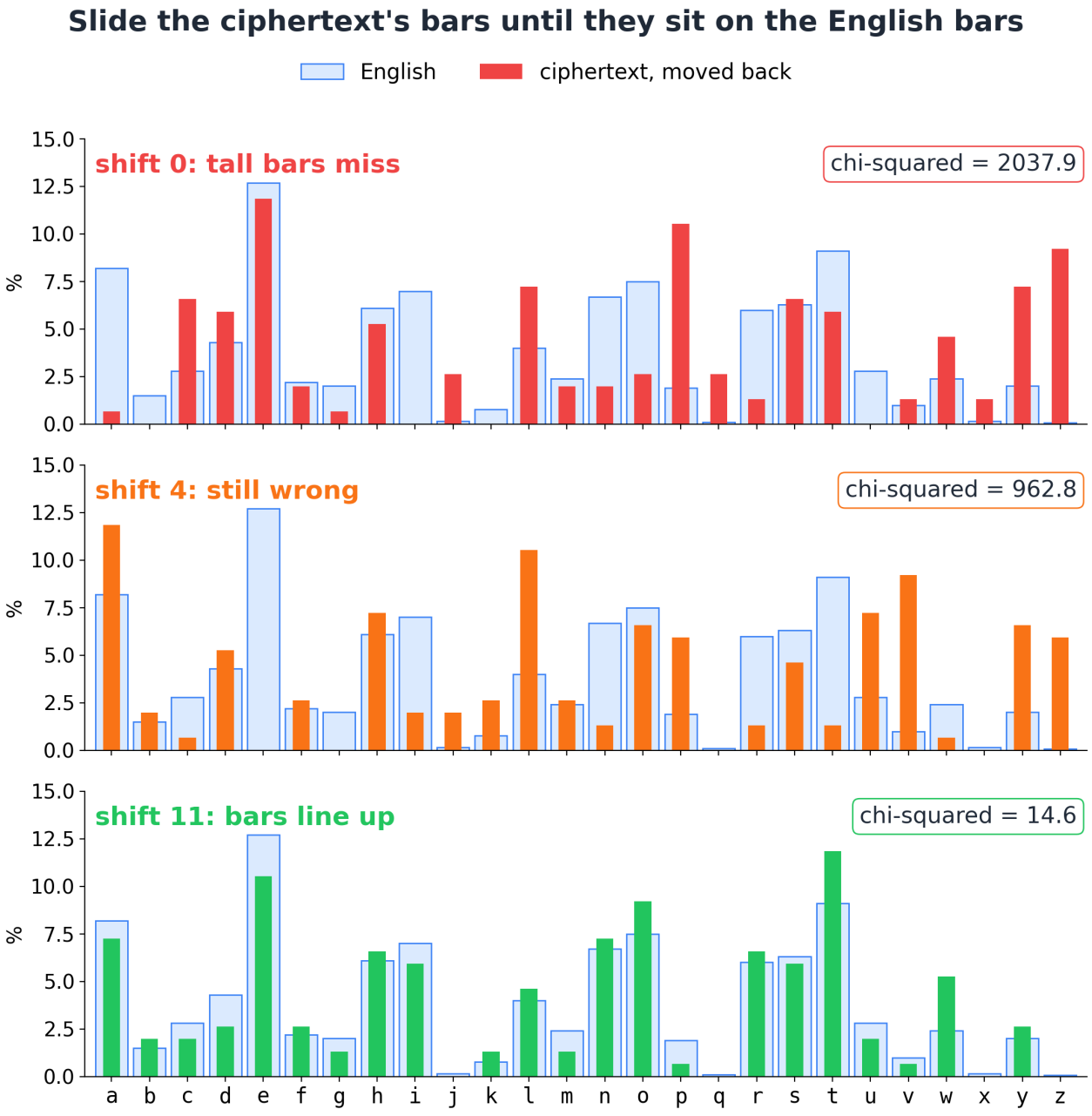


Figure 2.2: Three bar charts. In each, light blue bars show English letter frequencies and solid bars show the ciphertext moved back by some shift. At shift 0 and shift 4 the solid bars miss the blue ones, with chi-squared scores of 2037.9 and 962.8. At shift 11 they line up closely, with a chi-squared score of 14.6.

Step 5: score every shift. Here is the chi-squared calculation for the shift of 11, letter by letter. The text has 152 letters, so the expected count of each letter is 152 times its English frequency.

Letter	Observed O	Expected E	$(O-E)^2/E$
a	11	12.46	0.17

Letter	Observed O	Expected E	$(O-E)^2/E$
b	3	2.28	0.23
c	3	4.26	0.37
d	4	6.54	0.98
e	16	19.30	0.57
f	4	3.34	0.13
g	2	3.04	0.36
h	10	9.27	0.06
i	9	10.64	0.25
j	0	0.23	0.23
k	2	1.17	0.59
l	7	6.08	0.14
m	2	3.65	0.74
n	11	10.18	0.07
o	14	11.40	0.59
p	1	2.89	1.23
q	0	0.14	0.14
r	10	9.12	0.08
s	9	9.58	0.03
t	18	13.83	1.26
u	3	4.26	0.37
v	1	1.49	0.16
w	8	3.65	5.19
x	0	0.23	0.23
y	4	3.04	0.30
z	0	0.11	0.11

The last column adds up to 14.58, which rounds to $\chi^2 \approx 14.6$. The biggest single contribution is w (5.19): this message happens to mention a crow, a windowsill and somewhere, so it has more w's than average English.

Compare the shift of 0. Then the ciphertext letter Z, which appears 14 times, would have to be the plaintext letter z, which English expects only $152 \times 0.00074 \approx 0.11$ times in a text this long. That one letter contributes $(14 - 0.11)^2/0.11 \approx 1715$ to the score. Wrong shifts are punished hard.

Doing this for all 26 shifts gives the chart below. Shift 11 scores 14.6. The next-best shift, 22, scores 586.9. It is not close.

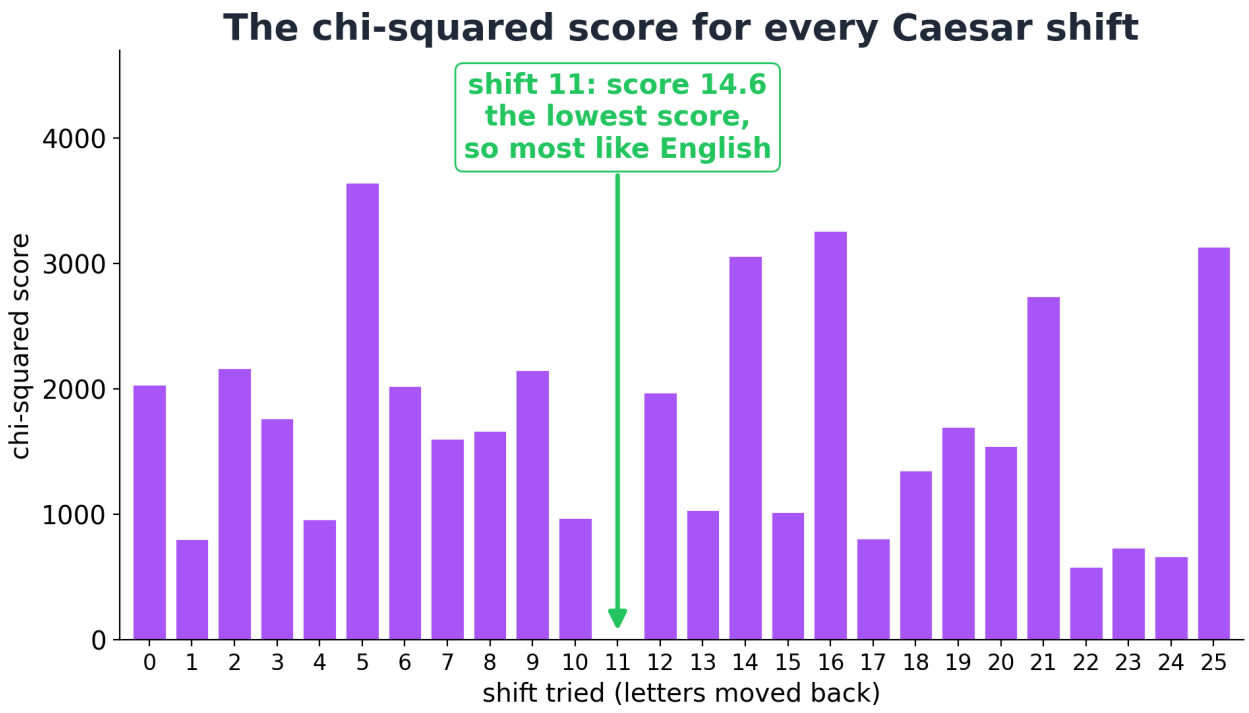


Figure 2.3: A bar chart of the chi-squared score for each of the 26 Caesar shifts. Most scores are between about 600 and 3,650. Shift 11 has an almost invisible bar with a score of 14.6, marked as the lowest and most English.

Step 6: decrypt. Moving every letter back 11 places gives:

```
every morning the baker on our street sells the first loaf of the day to a large
black crow the crow waits on the windowsill and pays with a shiny button that
it found somewhere in the town
```

Worked example 2: a substitution cipher, by counts and common words

A keyword substitution scrambles the alphabet, so there is no single shift to find: each letter has its own disguise. There are far too many possible keys to try them all (Guide 03 explains just how many). Frequency analysis lets you find them one letter at a time. Here is a ciphertext that kept its spaces and punctuation, which makes it a good first one to break:

```
EV ZSU S OPIW SMW RXEFV MEAKV ZKFM VKF PIW OIPOH EM VKF VPZFT
UVPQQFW SV FBSOVIC MEMF. MPNPWC EM VKF YEIISAF OPXIW TFJFJNFT S
VEJF ZKFM VKF OIPOH KSW NFFM ZTPMA, UP VKF QFPQIF ASVKFTFW EM VKF
URXSTF VP UVSTF SV EV. VKF NSHFT USEW VKSV VKF VPZFT ZSU KSXMVFW.
VKF VFSOKFT USEW VKSV VKF AFSTU ZFTF UEJQIC LTPDFM. NXV S UJSII
AETI ZEVK S ISMVFTM OIEJNFW VKF UVSETU SMW LPXMW S MPVF VXOHFW
NFVZFFM VKF ZKFFIU. EV USEW VKSV VKF OIPOH ZPXIW UVSTV SASEM ZKFM
UPJFPMF EM VKF YEIISAF VPIW VKF VTXVK.
```

Step 1: count the letters. There are 389 letters. The ten commonest are F (57), V (50), S (34), K

(29), M (24), E (22), P (22), I (21), W (18) and T (18). The letter G never appears.

Step 2: match by rank, but only as a first guess. English's commonest letters are e, t, a, o, i, n. So F is probably e, V is probably t, S might be a. Beyond the top three, the ranks get unreliable: in a few hundred letters, letters with similar frequencies easily swap places. The picture shows how the true answers line up. The top three match exactly; the rest are close but shuffled.

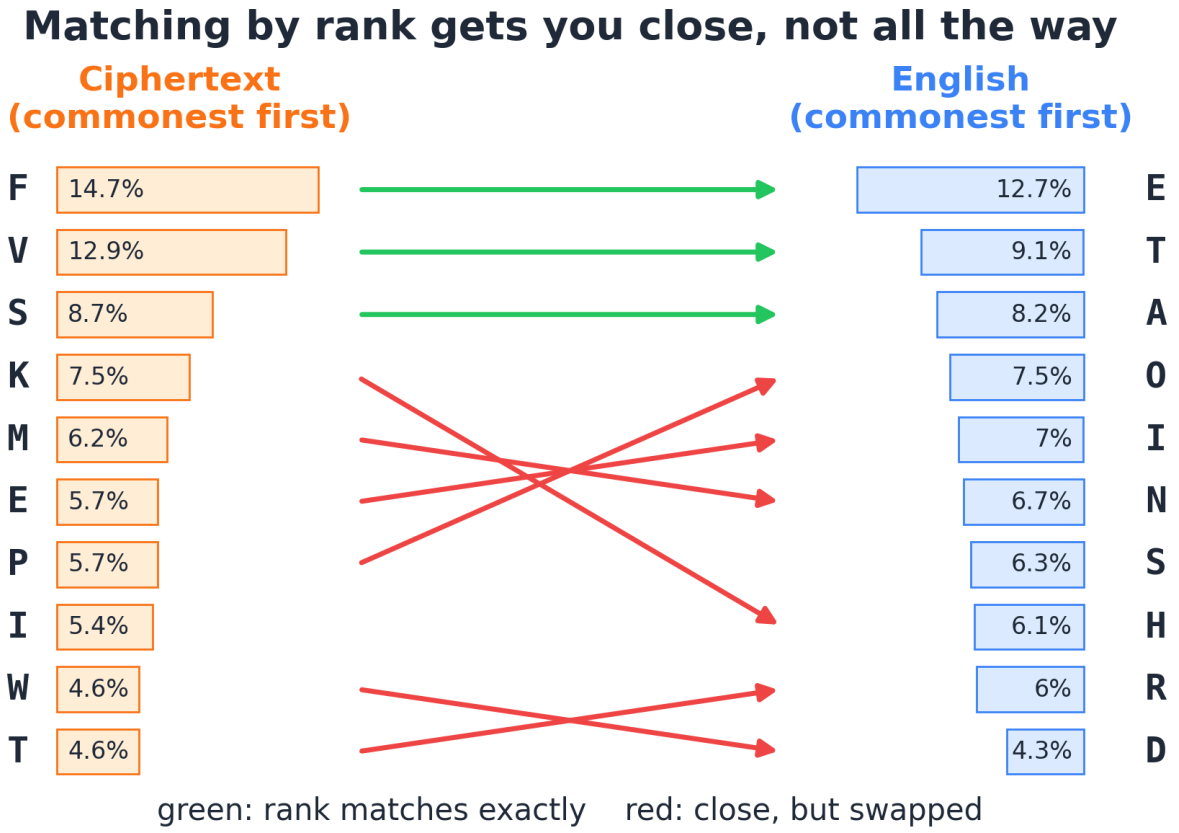


Figure 2.4: Two ranked lists side by side. On the left, the ten commonest ciphertext letters F, V, S, K, M, E, P, I, W, T with their percentages. On the right, the ten commonest English letters E, T, A, O, I, N, S, H, R, D. Green arrows join F to E, V to T and S to A. Red arrows show the rest cross over: K goes to H, M to N, E to I, P to O.

Step 3: use the commonest word. The three-letter word VKF appears 15 times. The commonest three-letter word in English is "the". And V and F are the two commonest letters, which fits t and e perfectly. So **V = t, K = h, F = e**.

Step 4: one-letter words. The word S stands alone 5 times. One-letter English words are "a" and "I". S is also the third commonest letter, like a. So **S = a**. Check it: the word VKSV, which appears 3 times, now reads t-h-a-t. "That." It fits.

Step 5: short words. Two-letter words EV (3 times) and EM (4 times) both start with E. With V = t, EV is "?t", so it could be "it" or "at". Since a is taken, E = i. Then EM is "i?": "in" or "is". The four-letter word ZKFM appears 3 times and reads "?he?". Words that fit are "when", "then", "them", "they". If M = n (from "in"), then ZKFM is "?hen", and since t is already taken, it must be "when". So **E = i, M = n, Z = w**.

Step 6: more words fall. Now ZSU reads “wa?”, which is surely “was”: **U = s**. SMW reads “an?”, most likely “and”: **W = d**. Check: USEW now reads “said”, a real word, so both guesses hold up. The word PIW now reads “??d”, with two letters you have not found yet, in the phrase VKF PIW OIPOH: “the ??d ?????”. A common three-letter word ending in d is “old”: **P = o, I = l**. Then OIPOH reads “?lo??” with its first and fourth letters the same: “clock”. So **O = c, H = k**.

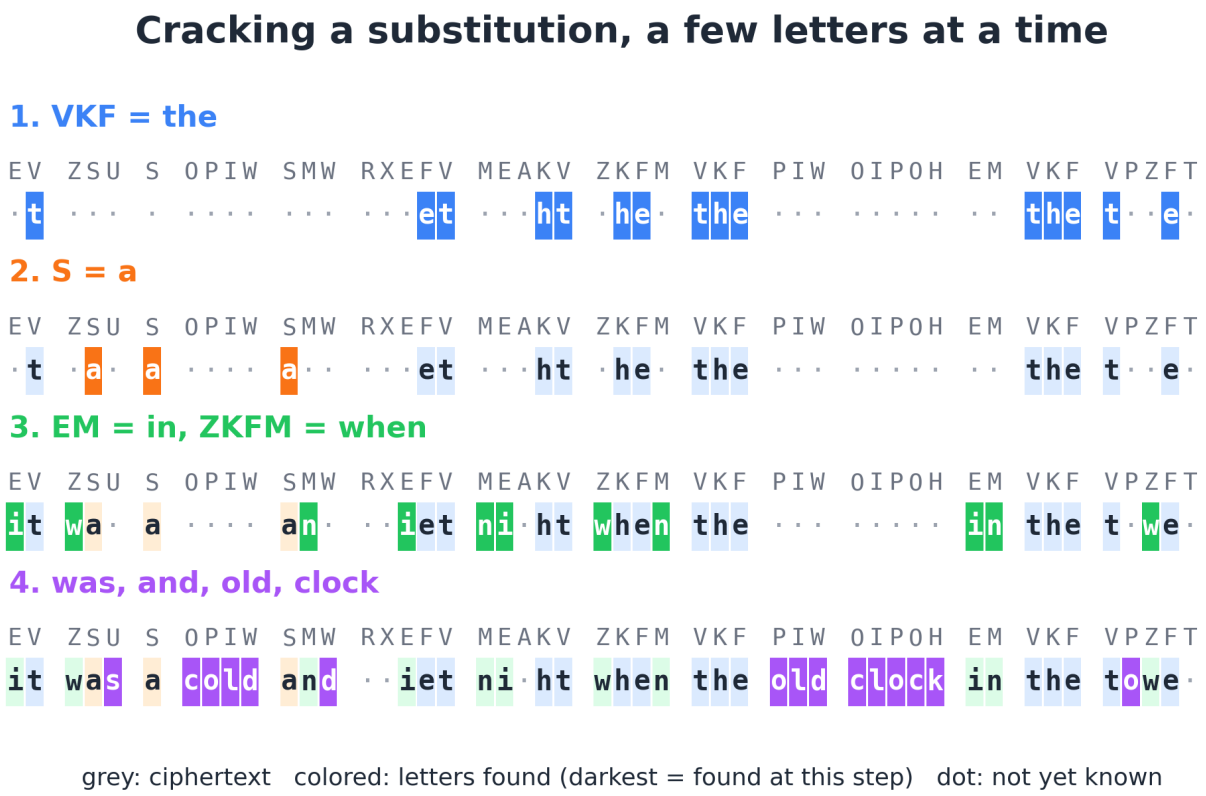


Figure 2.5: Four rows of the same ciphertext line, each with the plaintext found so far written underneath. Row 1 fills in t, h and e. Row 2 adds a. Row 3 adds i, n and w. Row 4 adds s, d, o, l, c and k, so the line reads “it was a cold and ..iet ni.ht when the old clock in the towe.”. Letters found at each step are highlighted in a new color.

Step 7: finish by reading. The first sentence now reads “it was a cold and ??iet ni?ht when the old clock in the towe? …” and your brain fills the gaps. “quiet” gives **R = q, X = u**; “night” gives **A = g**. The rest fall the same way: VPZFT is “tower” (T = r), MPNPWC is “nobody” (N = b, C = y), TFJFJNFT is “remember” (J = m), UVPQQFW is “stopped” (Q = p), YEIISAF is “village” (Y = v), LTPDFM is “frozen” (L = f, D = z), and FBSOVIC is “exactly” (B = x). That is 25 letters. The 26th, G, never appears because the message has no j.

Step 8: recover the key. Write down the cipher letter for each plaintext letter, in alphabetical order. A keyword substitution starts its cipher alphabet with a keyword, then lists the unused letters in order. Here the keyword jumps out:

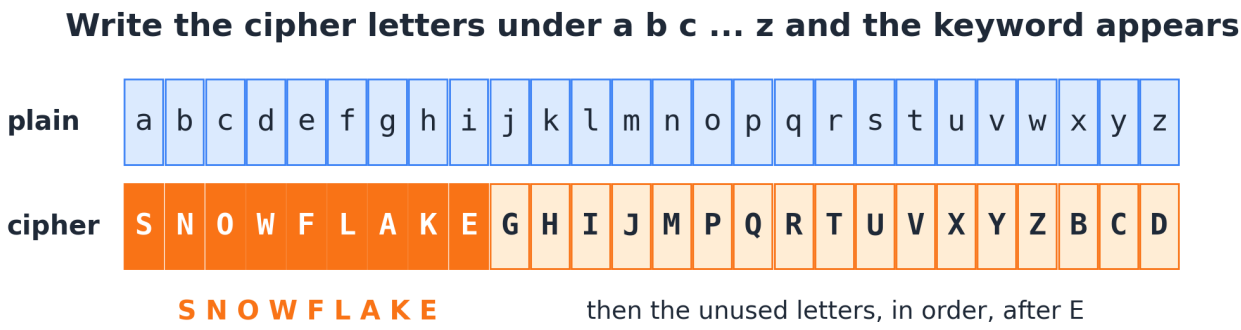


Figure 2.6: The plain alphabet a to z in blue boxes, with the cipher alphabet underneath in orange boxes: S N O W F L A K E, highlighted, followed by G H I J M P Q R T U V X Y Z B C D.

The keyword was SNOWFLAKE. Once you spot a keyword, you can fill in any letters you were unsure about, and check every one.

The full message: `it was a cold and quiet night when the old clock in the tower stopped at exactly nine. nobody in the village could remember a time when the clock had been wrong, so the people gathered in the square to stare at it. ...`

> **In the app:** Substitution lets you tap a cipher letter, then the plain letter you think it stands for, and fills in the text as you go.

When to try it

- The IoC is near English (about 0.067), so there is one cipher alphabet (see Guide 01, Reading the clues).
- The letter counts are bumpy, with a few very common letters and a few missing ones, but not on the English letters.
- The ciphertext keeps its spaces: then common words make it fast.
- You suspect a Caesar or affine cipher, and want to score the candidates rather than read 26 or 312 attempts by eye.
- You have split a Vigenère ciphertext into slices: each slice is a Caesar cipher, and chi-squared finds each slice's shift.

When not to

- The letter counts already look exactly like English. Then the letters are untouched and only their order has changed: it is a transposition, and no letter swap will help.
- The counts are flat (IoC near 0.040). Several alphabets are mixing the counts, so first find the key length (guides 04 and 05).
- The text is very short (under about 50 letters). Counts that small are unreliable. Look for cribs instead (Guide 06).

Traps and tips

- **The commonest letter is not always e.** In worked example 1, t beat e. Never stake everything on one letter; match the whole shape, or use chi-squared.
- **Ranks below the top few are a mess.** Use frequencies to get started, then switch to words as soon as you can.
- **Unusual texts.** A message about a zoo full of zebras, or a text full of names, has strange counts. A message written without the letter e (it has been done, as a whole novel) would fool you completely.
- **Missing spaces.** Most competition ciphers remove spaces or group letters in fives. Then look for common letter groups instead of words: TH, HE, IN, ER, AN are the commonest pairs, and THE, AND, ING are the commonest triples. A ciphertext triple that appears many times is probably THE.
- **Double letters.** A substitution keeps doubles: LL stays a double, just of a different letter. English doubles are usually LL, EE, SS, OO, TT.
- **Write in pencil.** Every guess might be wrong. Keep a table of which cipher letter maps to which plaintext letter, and when a guess produces nonsense, undo it.
- **Look for the keyword.** After a dozen letters, write the cipher alphabet out in order. A keyword substitution leaves a recognizable word at the start, and a long run of the alphabet in order after it. Spotting it fills in the rest.

By hand or by computer

Counting and scoring are chores; deciding what the counts mean is the skill. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
Count every letter	10 to 15 minutes	Counts[2], letter-count sheet[3]	dCode[4]
Slide English along the counts to fit a shift	1 to 2 minutes a shift, up to 25 shifts	Caesar[5], Caesar sheet[6]	dCode[7]

[1] Brain, pencil or computer?, the chapter before Guide 1.

[2] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English's, the IoC, the length and its factors.

[3] Letter counts and IoC, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-letter-counts, to copy into Google Sheets or download for Excel.

[4] dCode's frequency analysis: dcode.fr/frequency-analysis.

[5] Caesar, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/caesar. Its Scores view scores all 26 shifts at once.

[6] Caesar shifts and chi-squared, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-caesar, to copy into Google Sheets or download for Excel.

[7] dCode's Caesar solver: dcode.fr/caesar-cipher.

Job	By hand	Learn with	Just do it
Chi-squared for every shift	about 5 minutes a shift with a calculator: over 2 hours for all 26	Caesar[8], Caesar sheet[9]	dCode[10]
Solve a full substitution	30 to 90 minutes	Substitution[11], substitution sheet[12]	quipqiup[13], Guballa[14]

Try it yourself

1. A long Caesar ciphertext has commonest letters X, M, T and H, in that order. What is the shift? Check your answer using all four letters.
2. A text has 200 letters. The letter e appears 30 times and the letter z appears 2 times. Using English frequencies of 12.7% for e and 0.074% for z, work out each letter's contribution $(O - E)^2/E$ to the chi-squared score. Which letter makes the score worse, and why?
3. A substitution ciphertext with spaces contains the word ZIT nine times, the word ZIQZ twice, and the word Q on its own three times. What do these three words probably say?

In the app: Practice with Caesar: match the bars, then Substitution with word breaks, then the harder Substitution without word breaks.

Answers

1. If X is e, the shift is 19 (e is letter 4 and X is letter 23, counting A as 0; $23 - 4 = 19$). Check: t is letter 19, and $19 + 19 = 38$, which goes past Z, so subtract 26 to get 12, which is M. a moves to T, and o moves to H. All four fit, so the shift is 19.
2. For e: $E = 200 \times 0.127 = 25.4$, so the contribution is $(30 - 25.4)^2/25.4 = 21.16/25.4 \approx 0.83$. For z: $E = 200 \times 0.00074 = 0.148$, so the contribution is $(2 - 0.148)^2/0.148 \approx 23.2$. The two z's make the score much worse, because z is so rare that even two of them are a big surprise, while 30 e's is only a little more than expected.
3. ZIT = the, ZIQZ = that, Q = a. The letters agree: Z = t and I = h in both words, and Q = a in both "that" and "a".

Words to know

- **Frequency analysis**: breaking a cipher by comparing its letter counts with those of the language.

[8] Caesar, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/caesar. Its Scores view scores all 26 shifts at once.

[9] Caesar shifts and chi-squared, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-caesar, to copy into Google Sheets or download for Excel.

[10] dCode's Caesar solver: dcode.fr/caesar-cipher.

[11] Substitution, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/substitution. You choose the letters; it fills in the text.

[12] Substitution key and partial decrypt, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-substitution, to copy into Google Sheets or download for Excel.

[13] quipqiup, a substitution solver: quipqiup.com.

[14] Guballa's substitution solver: guballa.de/substitution-solver.

- **Frequency**: how often a letter appears, as a fraction of all the letters.
- **Expected count**: how many times a letter would appear in this many letters of average English.
- **Chi-squared** (χ^2): a score of how far a set of counts is from English; lower means more English-like.
- **Monoalphabetic substitution**: a cipher that replaces each letter with another, the same way every time.
- **Keyword substitution**: a substitution whose cipher alphabet starts with a keyword, followed by the unused letters in order.
- **ETAOIN SHRDLU**: the approximate order of the twelve commonest letters in English.

Going further

- Frequency analysis on Wikipedia.
- Al-Kindi on Wikipedia.
- Practical Cryptography: chi-squared statistic and English letter frequencies.
- The National Cipher Challenge resources, whose BOSS lessons build up to cracking keyword ciphers.
- Simon Singh, *The Code Book* (1999), chapter 1, tells the story of al-Kindi and of Mary Queen of Scots, whose substitution cipher was broken by frequency analysis.

Guide 3

Keys and brute force

Counting keys, clock arithmetic, and cracking the affine cipher with two equations

By the end you can count how many keys a cipher has, decide whether trying them all is possible, do arithmetic on a 26-hour clock, and recover an affine cipher's key from just two letters.

The big idea

Every cipher has a **key**: the secret setting that both the sender and the receiver know. For a Caesar cipher the key is one number, the shift. If you can list every possible key and try each one, you will certainly find the right one. That is called a **brute-force attack**, and it is the most reliable attack there is, when it is possible. So the first question to ask about any cipher is: **how many keys does it have?** If the answer is 26, you can try them all before lunch. If the answer has 27 digits, you cannot, and you need to be cleverer.

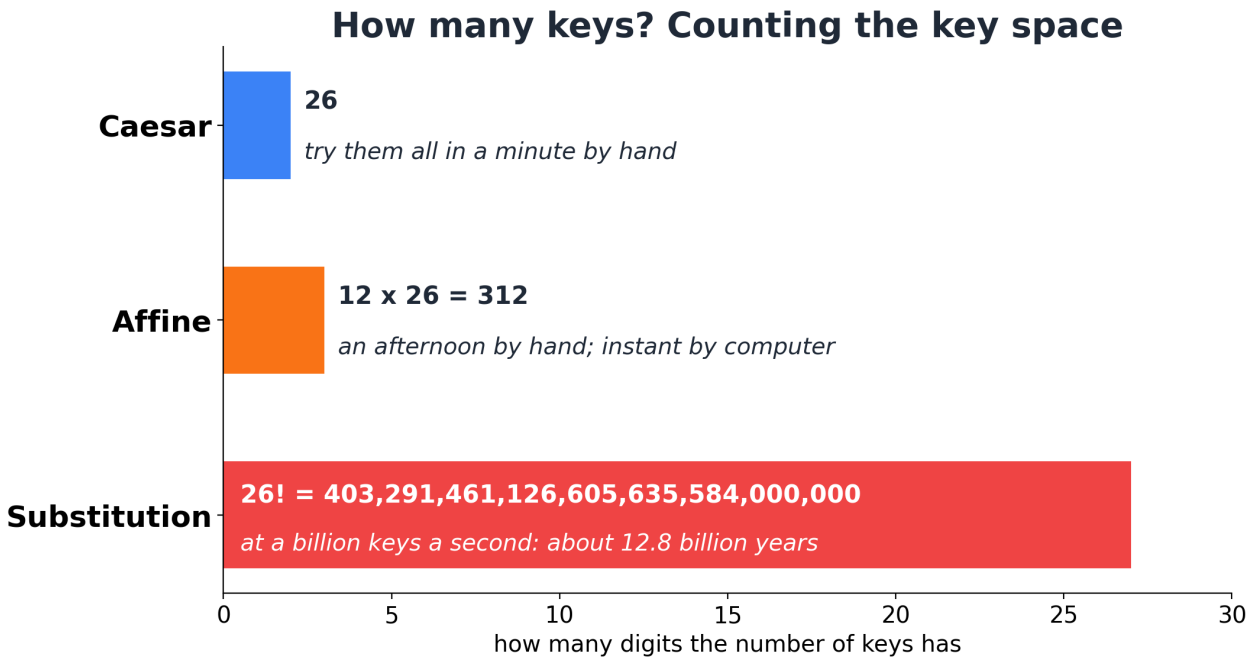


Figure 3.1: A bar chart of how many digits the number of keys has. Caesar has 26 keys, a 2-digit number. Affine has 312, a 3-digit number. Substitution has 26 factorial, which is 403,291,461,126,605,635,584,000,000, a 27-digit number that would take about 12.8 billion years to try at a billion keys per second.

Which ciphers it breaks

Cipher family	Number of keys	Can you try them all?
Caesar	26	Yes, by hand in minutes
Atbash (A to Z, B to Y, …)	1	There is nothing to try: it is always the same
Affine	312	Yes by computer, instantly; by hand, use two equations instead (this guide)
Keyword or general substitution	$26!$, about 4×10^{26}	No. Use frequency analysis (Guide 02) or hill climbing (Guide 08)
Vigenère with a 5-letter key	$26^5 = 11{,}881{,}376$	By computer, yes; but splitting into slices is much faster (Guide 05)
Columnar transposition, 6 columns	$6! = 720$ orders	Yes by computer; by hand, rearrange columns until words appear (Guide 07)

The method also tells you something important about the ciphers it cannot break: a cipher with too many keys to try forces you to find a weakness, like uneven letter frequencies.

Why it works

Counting keys

Counting keys is a counting puzzle, the same kind you meet in math competitions.

Caesar. The key is a shift from 0 to 25. That is 26 keys (and a shift of 0 does nothing, so really 25 useful ones).

Substitution. The key is a scrambled alphabet. How many scrambled alphabets are there? The letter a can become any of 26 letters. Then b can become any of the 25 left. Then c any of the 24 left, and so on down to z, which gets the 1 letter remaining. So the number of keys is

$$26 \times 25 \times 24 \times \cdots \times 2 \times 1$$

Mathematicians write this as $26!$ and call it "26 factorial". It equals 403,291,461,126,605,635,584,000,000. That is about 4×10^{26}, a 4 followed by 26 zeros. Imagine a computer that tries a billion keys every second. Trying all of them would take about 12.8 billion years, nearly the age of the universe (about 13.8 billion years). Brute force is hopeless. And yet you broke one in Guide 02 with a pencil, because frequency analysis does not need to try keys at all.

Affine. This guide's main cipher. Its key is two numbers. Before you can count them, you need a new kind of arithmetic.

Clock arithmetic

Number the letters from 0 to 25: A = 0, B = 1, C = 2, and so on up to Z = 25. A Caesar shift of 5 adds 5 to each letter's number. But what is X + 5? X is 23, and $23 + 5 = 28$, and there is no letter 28. The alphabet wraps around: after Z comes A again. So 28 should mean 2, which is C.

This is exactly how a clock works. On a 12-hour clock, 5 hours after 9 o'clock is 2 o'clock, not 14 o'clock. Our alphabet is a 26-hour clock: whenever you go past 25, you subtract 26.

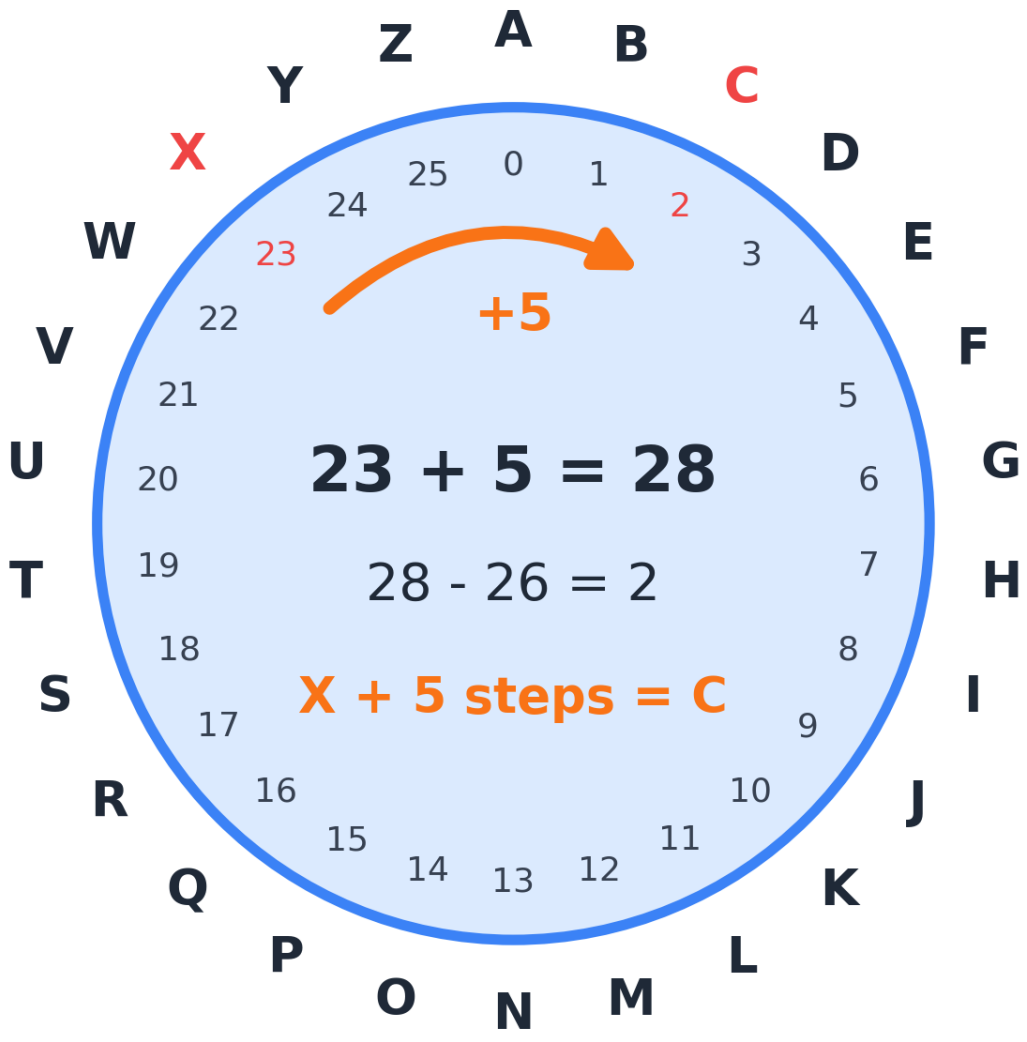


Figure 3.2: A clock face with 26 positions numbered 0 to 25, each labeled with a letter from A to Z. An orange arrow goes 5 steps forward from 23 (X) round past the top to 2 (C), with the sums 23 + 5 = 28 and 28 - 26 = 2 written in the middle.

The same works for subtraction and multiplication:

- **Subtraction:** $3 - 8 = -5$. A negative number means you went backwards past 0, so add 26: $-5 + 26 = 21$, which is V. (Check: 8 steps back from D lands on V.)
- **Multiplication:** $7 \times 8 = 56$. Take away 26 as many times as you can: $56 - 26 - 26 = 4$, which is E.

> **In the app:** Caesar has a Disk view: a cipher wheel you turn one step at a time, through all 26 shifts.

The grown-up version

Modular arithmetic

This clock arithmetic has a proper name: **modular arithmetic**, and 26 is called the **modulus**. Mathematicians write

$$28 \equiv 2 \pmod{26}$$

and read it "28 is congruent to 2, mod 26". The symbol $\equiv$ (three lines) means "lands on the same spot on the clock". Two numbers are congruent mod 26 when they differ by a whole number of 26s: 28 and 2 differ by 26, and -5 and 21 differ by 26, so $-5 \equiv 21 \pmod{26}$.

When we write x mod 26 we mean the remainder when x is divided by 26, a number from 0 to 25. So 56 mod $26 = 4$.

Here is a very useful fact: you can reduce mod 26 at any point in a calculation and the final answer comes out the same. That keeps the numbers small.

The affine cipher

The Caesar cipher adds a number. The affine cipher multiplies **and** adds. With the letters numbered 0 to 25, its rule for encrypting a plaintext letter x is

$$E(x) = ax + b \text{ mod } 26$$

The key is the pair of numbers (a, b). For example, with $a = 11$ and $b = 6$, the letter t (19) becomes $11 \times 19 + 6 = 215$, and $215 = 8 \times 26 + 7$, so 215 mod $26 = 7$, which is H. Likewise h (7) becomes $77 + 6 = 83 \equiv 5$, F, and e (4) becomes $44 + 6 = 50 \equiv 24$, Y. So "the" encrypts to HFY.

Notice that a Caesar cipher is just an affine cipher with $a = 1$.

Why a must share no factor with 26

Can a be any number? Try $a = 2$. Then a (0) becomes $2 \times 0 = 0$ and n (13) becomes $2 \times 13 = 26 \equiv 0$. Two different letters both encrypt to A! The person receiving the message would have no way to tell which was meant. A key that does this is useless.

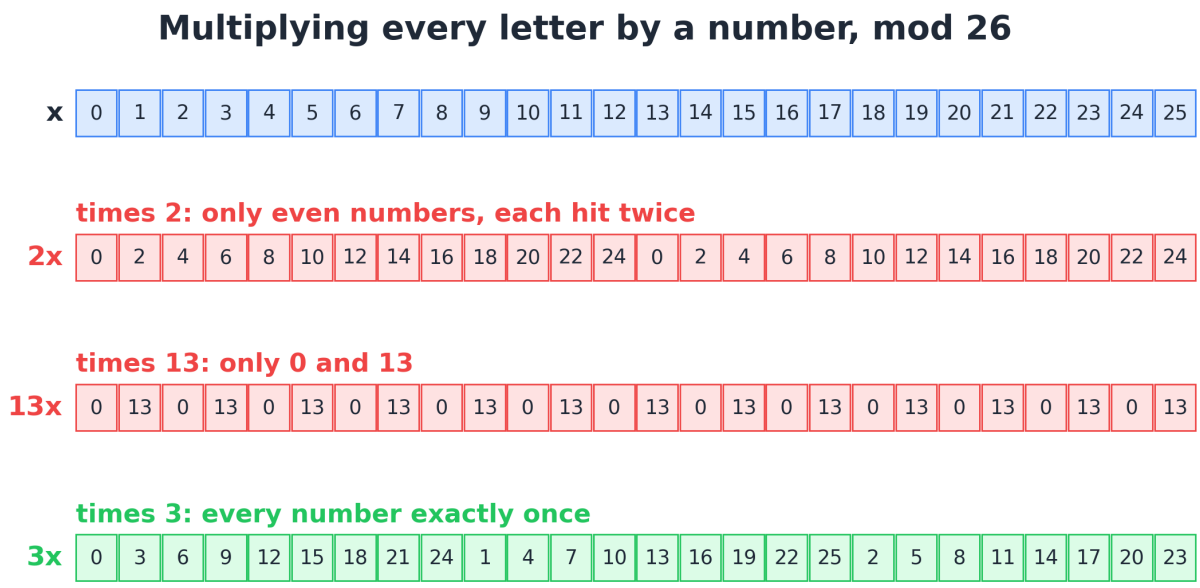


Figure 3.3: Three rows of 26 numbered boxes. The top row is 0 to 25. Multiplying by 2 gives only even numbers, each appearing twice, colored red. Multiplying by 13 gives only 0 and 13, also red. Multiplying by 3 gives every number from 0 to 25 exactly once, colored green.

The problem is that 2 shares a factor with 26 (because $26 = 2 \times 13$). Whenever a and 26 share a factor, some multiple of a is a whole number of 26s, and letters collide. So a must **share no factor with 26** except 1. Mathematicians say a and 26 are **coprime**, and write $\gcd(a, 26) = 1$, where gcd stands for "greatest common divisor".

Since $26 = 2 \times 13$, that rules out all even numbers and 13. The allowed values are

$$a \in \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$$

That is 12 choices for a (the symbol $\in$ means "is one of"). The shift b can be anything from 0 to 25: 26 choices. So the affine cipher has $12 \times 26 = 312$ keys. One of them, $a = 1, b = 0$, leaves the message unchanged.

Multiplicative inverses: undoing a multiplication

To decrypt a Caesar cipher you subtract the shift. To decrypt an affine cipher you must subtract b and then undo the multiplication by a. But you cannot simply divide: $7 \div 3$ is not a whole number, and there are no fractions on a clock.

Instead, you multiply by a number that undoes it. Look: $3 \times 9 = 27 \equiv 1 \pmod{26}$. So multiplying by 9 cancels out multiplying by 3, the way dividing by 3 would. We call 9 the **multiplicative inverse** of 3 mod 26, and write $3^{-1} \equiv 9 \pmod{26}$.

A quick way to find an inverse: list the numbers that are 1 more than a multiple of 26: 27, 53, 79, 105, 131, 157, 183, 209, 235, … and look for one that a divides exactly. For $a = 15$: $105 = 15 \times 7$, so $15^{-1} \equiv 7$. For $a = 11$: $209 = 11 \times 19$, so $11^{-1} \equiv 19$.

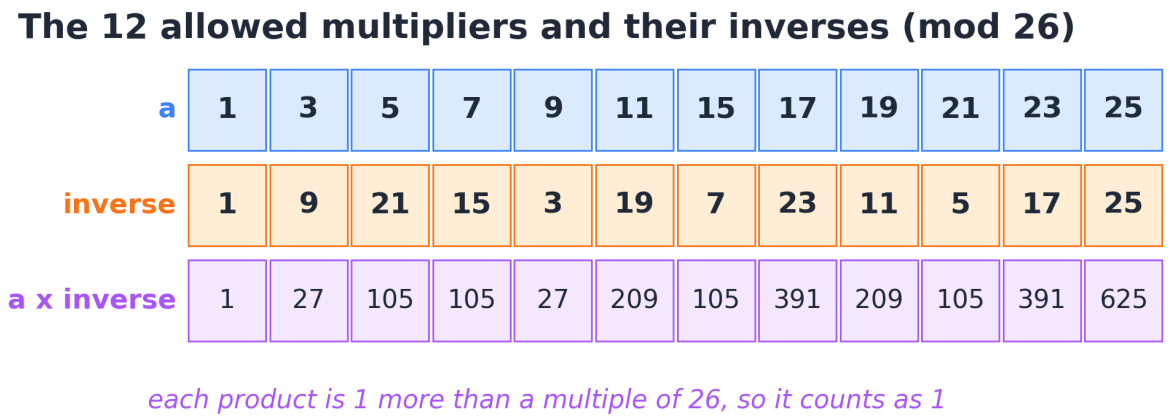


Figure 3.4: A table of the 12 allowed multipliers and their inverses mod 26: 1 and 1, 3 and 9, 5 and 21, 7 and 15, 9 and 3, 11 and 19, 15 and 7, 17 and 23, 19 and 11, 21 and 5, 23 and 17, 25 and 25. A third row shows each product, such as 27, 105, 209, 391 and 625, each one more than a multiple of 26.

Only the 12 allowed values of a have inverses. The even numbers and 13 do not, which is the same fact as before seen from the other side.

With the inverse, the decryption rule is

$$D(y) = a^{-1}(y - b) \bmod 26$$

where y is a ciphertext letter's number.

> **In the app:** Affine has a dial for a and one for b. Its wheel shows what multiplying does to the alphabet, and why 2 and 13 make letters collide.

Worked example 1: brute force on a Caesar cipher

Here is a short Caesar ciphertext: `DVVKDVSPKYVFCUDZCC`. It is only 18 letters, too short for reliable letter counts. But a Caesar cipher has only 26 keys, so just try them all. Write the ciphertext moved back by 0, 1, 2, … places, one per line, and read down the list.

Trying all 26 keys on DVVKDVSPKYVFCUDZCC

back 0	dvvkdvspkyvfcudzcc	
back 1	cuujcurojxuebtcybb	
back 2	bttibtqniwtdasbxaa	
back 3	asshaspmhvsczrawzz	
back 4	zrrgzrolgurbyqzvyy	
back 5	yqqfyqnkftqaxpyuxx	
back 6	xppexpmjespzwoxtww	
back 7	woodwolidroyvnwsvv	
back 8	vnncvnkhcqnxumvruu	
back 9	ummbumjgbpmwtluqtt	
back 10	tllatlifaolvsktpss	
back 11	skkzskheznkurjsorr	
back 12	rjjyrjgdymjtqirnqq	
back 13	qiixqifcxlisphqmpp	
back 14	phhwphebwkhrogploo	
back 15	oggvogdavjgqnfoknn	
back 16	nffunfczuifpmenjmm	
back 17	**meetmebytheoldmill**	**reads!**
back 18	lddsldaxsgdnkclhkk	
back 19	kccrkczwrfcmjbkgjj	
back 20	jbbqjbyvqebliajfii	
back 21	iaapiaxupdakhziehh	
back 22	hzzohzwtoczjgyhdgg	
back 23	gyyngyvsnbyifxgcff	
back 24	fxxmfxurmaxhewfbee	
back 25	ewwlewtqlzwgdveadd	

Figure 3.5: A table of 26 rows, one for each Caesar shift. Each row shows the ciphertext DVVKDVSPKYVFCUDZCC moved back by that many places. Every row is gibberish except row 17, highlighted in green, which reads “meetmebytheoldmill”.

Row 17 reads `meetmebytheoldmill`: “meet me by the old mill”. No other row comes close. That is a brute-force attack: no cleverness, just certainty.

Worked example 2: an affine cipher from two letters

Here is a ciphertext I made with an affine cipher from an original message:

```
HFYWY CLYHI YYHQT UQWWY HJELH YTHET QUFHG HHFYY GWHUG HYRLQ
TUHFY XYHHY LHFYH ELCFG TNHFY WHLYY HIGPG TNHYX XTERE NKOFY
LYOYG LYUEQ TU
```

You could try all 312 keys by computer and score each result with chi-squared (Guide 02). By hand, there is a much faster way: **guess two letters and solve for the key**.

Step 1: count. The message has 112 letters. The commonest is Y (22 times), then H (20 times). Nothing else comes close: the next letters appear 8 times each.

Step 2: guess. The two commonest letters in English are e and t. So guess that Y stands for e and H stands for t. In numbers: Y = 24, e = 4, H = 7, t = 19.

Step 3: write two equations. The rule is $E(x) = ax+b$. The guess “e becomes Y” says $E(4) = 24$, and “t becomes H” says $E(19) = 7$:

$$4a + b \equiv 24 \pmod{26}$$
$$19a + b \equiv 7 \pmod{26}$$

Two unknowns, two equations: the same kind of puzzle as “two numbers add to 10 and differ by 4”, only on a clock.

Step 4: subtract to get rid of b. Take the first equation from the second:

$$15a \equiv 7 - 24 = -17 \equiv -17 + 26 = 9 \pmod{26}$$

Step 5: undo the 15. The inverse of 15 is 7, because $15 \times 7 = 105 = 4 \times 26 + 1$. Multiply both sides by 7:

$$a \equiv 7 \times 9 = 63 = 2 \times 26 + 11 \equiv 11 \pmod{26}$$

Step 6: find b. Put $a = 11$ back into the first equation:

$$b \equiv 24 - 4 \times 11 = 24 - 44 = -20 \equiv -20 + 26 = 6 \pmod{26}$$

So the key is $a = 11, b = 6$. And 11 is on the list of allowed values, which is a good sign. (If you had got an even number or 13, your guess would have been wrong.)

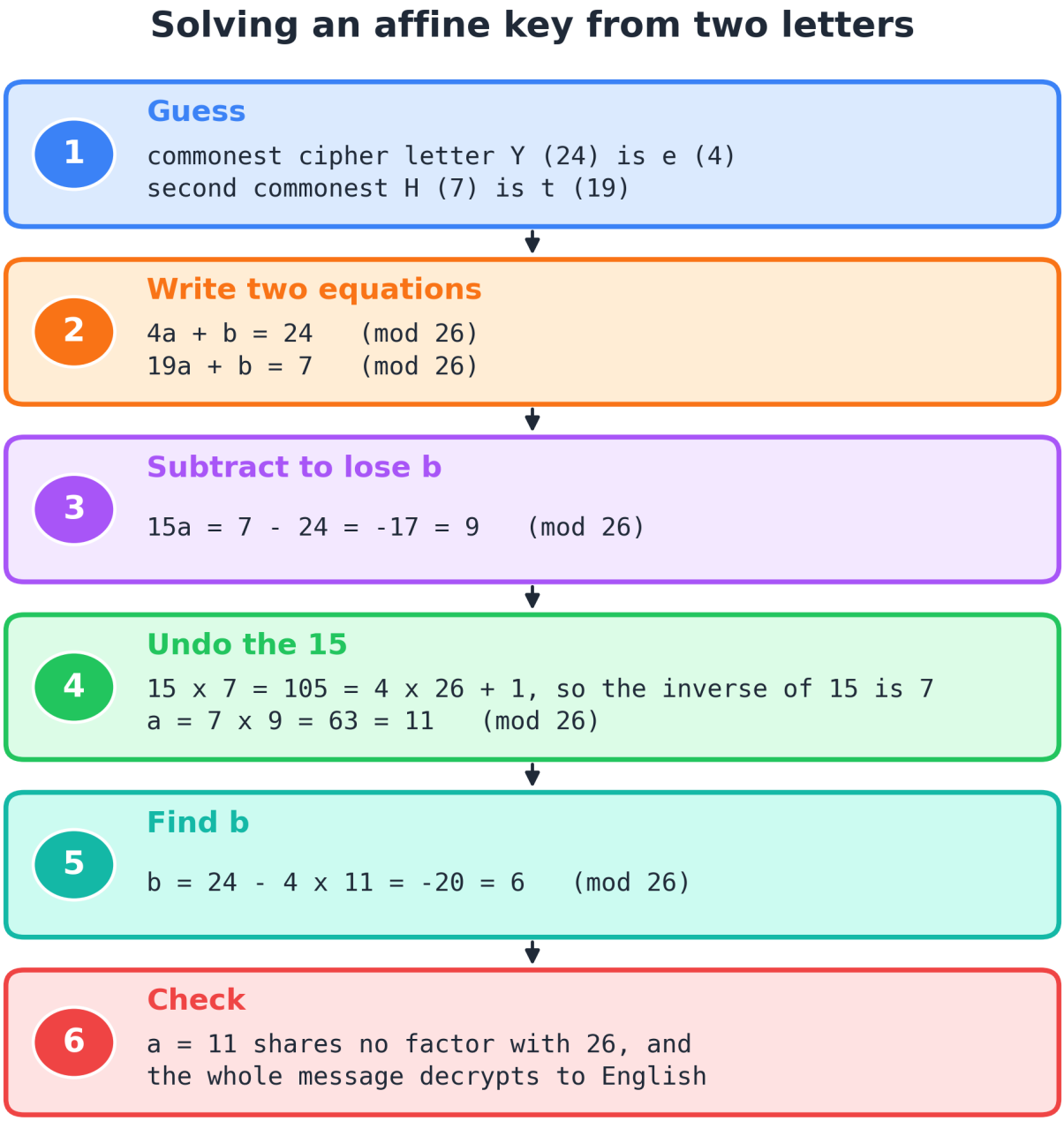


Figure 3.6: Six colored steps for solving an affine key. 1: guess the commonest cipher letter Y (24) is e (4) and H (7) is t (19). 2: write 4a + b = 24 and 19a + b = 7, mod 26. 3: subtract to get 15a = 9. 4: the inverse of 15 is 7, so a = 63 = 11. 5: b = 24 - 44 = -20 = 6. 6: check that a = 11 is allowed and the message decrypts. In the picture, = stands for “is congruent to”.

Step 7: decrypt. The inverse of 11 is 19 (since $11 \times 19 = 209 = 8 \times 26 + 1$). So the decryption rule is $D(y) = 19(y - 6)$ mod 26. Apply it to the first few ciphertext letters:

Cipher letter	y	$y-6$	$19(y-6)$	mod 26	Plain letter
H	7	1	19	19	t
F	5	-1	-19	7	h
Y	24	18	342	4	e
W	22	16	304	18	s
C	2	-4	-76	2	c
L	11	5	95	17	r

(For example, $342 = 13 \times 26 + 4$, and $-76 + 3 \times 26 = 2$.)

"thesecr…". It works. The whole message reads:

```
the secret meeting is set for ten tonight at the east gate. bring the letter,
the torch and the street map, and tell nobody where we are going.
```

When to try it

- **Brute force:** whenever the number of keys is small: Caesar (26), affine (312, by computer), short transposition keys, or any cipher where you can narrow the key down to a few hundred possibilities.
- **Two equations:** whenever the IoC says one alphabet (about 0.067) and the letter counts are bumpy but not a slid copy of English (see Guide 01). An affine cipher looks exactly like a keyword substitution from the outside, so try the two-equation attack first: it takes five minutes, and if it gives a sensible key, you are done.
- **Counting keys:** before you start any attack, to decide whether brute force is even an option.

When not to

- **Substitution ciphers.** $26!$ keys is far too many. Use frequency analysis.
- **When the two-equation attack gives a bad key.** If a comes out even or 13, or if the decryption is gibberish, your guess about which letters are e and t was wrong. Do not force it.
- **Very short texts and brute force by eye.** With only a few letters, two different keys might both give something that looks like a word. Prefer longer stretches of text, or score with chi-squared.

Traps and tips

- **The guess might be wrong, so have a plan.** In a short text, t or a might beat e. If the first guess fails, keep Y = e and try the third-commonest letter as t, or swap the two guesses. There are only a handful of sensible pairs to try.
- **Negative numbers.** When a subtraction goes below 0, add 26. Forgetting this is the commonest mistake.
- **Agree on A = 0.** Some books number the letters from A = 1. Either works, but everyone on the team must use the same convention, or your keys will not match.
- **Not every number can be divided.** If you end up with an equation like $2a \equiv 6 \pmod{26}$, you cannot just halve it: both $a = 3$ and $a = 16$ satisfy it. This happens when you subtract equations whose letters differ by an even number. Choose a different pair of letters whose difference is odd and not 13, or try both answers.

- **Keep an inverse table handy.** 3 and 9, 5 and 21, 7 and 15, 11 and 19, 17 and 23, 25 and 25 (and 1 and 1). Memorizing these saves time.
- **Check with a third letter.** Once you have a key, test it on a third common letter before decrypting everything. If that letter decrypts sensibly, you are on track.

By hand or by computer

Brute force is the clearest case of all: trying every key is exactly the job a machine does best. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
Try all 26 Caesar shifts	25 to 50 minutes	Caesar[2], Caesar sheet[3]	dCode[4]
Try all 312 affine keys	many hours	Affine[5], affine sheet[6]	dCode[7]
Solve the two equations for an affine key	5 to 10 minutes	Affine[8], affine sheet[9]	dCode[10]
Find an inverse mod 26	2 to 5 minutes	affine sheet[11]	n/a

Try it yourself

1. Work out each of these mod 26: (a) $17 + 15$, (b) 7×8, (c) $5 - 9$.
2. Find the inverse of 5 mod 26. (Hint: look through 27, 53, 79, 105, …)
3. An affine ciphertext's commonest letter is C and its second commonest is Z. Guess that C is e and Z is t. Find the key (a, b), then decrypt the word `GCYU`.

> **In the app:** Try Caesar: match the bars and the Affine exercise, where you solve for the multiplier and the shift. The affine page of the family tree shows its relatives, Caesar and decimation.

[1] Brain, pencil or computer?, the chapter before Guide 1.

[2] Caesar, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/caesar. Its Scores view scores all 26 shifts at once.

[3] Caesar shifts and chi-squared, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-caesar, to copy into Google Sheets or download for Excel.

[4] dCode's Caesar solver: dcode.fr/caesar-cipher.

[5] Affine, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/affine. Dials for a and b, and the multiplier wheel.

[6] Affine keys and inverses, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-affine, to copy into Google Sheets or download for Excel.

[7] dCode's affine solver: dcode.fr/affine-cipher.

[8] Affine, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/affine. Dials for a and b, and the multiplier wheel.

[9] Affine keys and inverses, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-affine, to copy into Google Sheets or download for Excel.

[10] dCode's affine solver: dcode.fr/affine-cipher.

[11] Affine keys and inverses, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-affine, to copy into Google Sheets or download for Excel.

Answers

1. (a) $17+15 = 32$, and $32-26 = 6$. (b) $7 \times 8 = 56$, and $56-26-26 = 4$. (c) $5-9 = -4$, and $-4 + 26 = 22$.
2. $105 = 5 \times 21$, and $105 = 4 \times 26 + 1$. So $5 \times 21 \equiv 1$, and the inverse of 5 is 21.
3. C = 2, Z = 25, e = 4, t = 19. The equations are $4a+b \equiv 2$ and $19a+b \equiv 25$. Subtracting: $15a \equiv 23$. Multiply by 7 (the inverse of 15): $a \equiv 161 = 6\times26+5 \equiv 5$. Then $b \equiv 2-4\times5 = -18 \equiv 8$. The key is $(5, 8)$. To decrypt, use the inverse of 5, which is 21: $D(y) = 21(y-8)$ mod 26. G (6) gives $21 \times (-2) = -42 \equiv 10$, k. C (2) gives $21 \times (-6) = -126 \equiv 4$, e. Y (24) gives $21 \times 16 = 336 \equiv 24$, y. U (20) gives $21 \times 12 = 252 \equiv 18$, s. The word is “keys”.

Words to know

- **Key**: the secret setting of a cipher that the sender and receiver share.
- **Key space**: the set of all possible keys; its size is the number of keys.
- **Brute force**: trying every possible key.
- **Factorial** ($n!$): $n \times (n-1) \times \cdots \times 1$; $26!$ counts the scrambled alphabets.
- **Modular arithmetic**: arithmetic on a clock; mod 26, after 25 comes 0.
- **Congruent** ($\equiv$): landing on the same spot on the clock; $28 \equiv 2 \pmod{26}$.
- **Coprime**: sharing no factor except 1; $\gcd(a, 26) = 1$.
- **Affine cipher**: $E(x) = ax + b$ mod 26, with a coprime to 26.
- **Multiplicative inverse** (a^{-1}): the number that multiplies with a to give 1 mod 26.

Going further

- Affine cipher on Wikipedia.
- Modular arithmetic on Wikipedia.
- Brute-force attack on Wikipedia.
- Practical Cryptography: the affine cipher.
- The National Cipher Challenge resources, including the BOSS training lessons.
- Simon Singh, *The Code Book* (1999), on why a huge number of keys is not the same as a secure cipher.

Guide 4

The index of coincidence

How often do two letters match? One number that tells one alphabet from several

By the end you can tell, from counting letters alone, whether a message uses one alphabet or several, and if several, how many.

The big idea

Take a page of English. Close your eyes, put your finger on a letter, then put your finger on another letter somewhere else. How often are the two letters the same? In English it happens about 66 times in every 1000 tries. In a jumble of random letters it happens only about 38 times in 1000. That one number, the chance that two letters picked at random match, is called the **index of coincidence**. It survives some kinds of encryption and is destroyed by others. So it tells you which kind of cipher you are facing before you have read a single word.

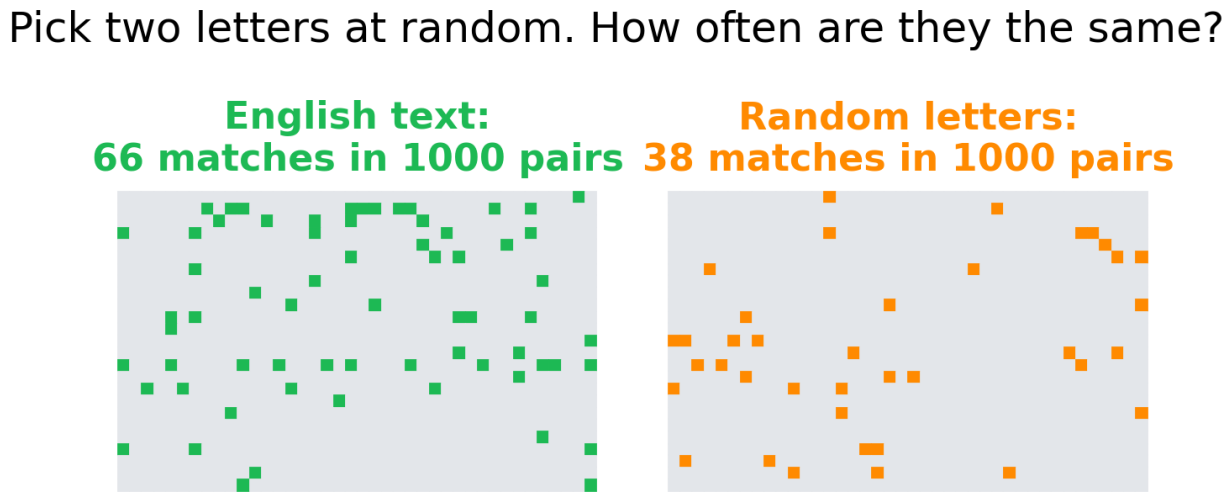


Figure 4.1: Two grids of 1000 squares. On the left, English text: 66 green squares mark the pairs of letters that matched. On the right, random letters: only 38 orange squares matched.

Which ciphers it breaks

The index of coincidence (IoC for short) does not usually hand you the key. It tells you what kind of cipher you have, and for the Vigenère family it tells you the length of the key. That is often the hardest step.

Cipher family	Does the IoC help?	How
Caesar, affine, keyword, any simple substitution	Yes, as an identifier	IoC stays near 0.066, like English. Tells you "one alphabet": go to frequency analysis (Guide 02).
Transposition (rail fence, columnar)	Yes, as an identifier	IoC stays near 0.066, and the letter counts themselves look exactly like English.
Vigenère, Beaufort, variant Beaufort	Yes, strongly	IoC drops toward 0.038. Splitting into columns finds the key length.
Autokey, running key	Partly	IoC is low, but no key length makes the columns look like English. That itself is a clue.
Playfair, Hill (two letters at a time)	A little	IoC lands in between (around 0.045 to 0.055). Other tests settle it (Guide 09).
Enigma and other machines	No	The alphabet changes every letter and never repeats in a short message.

> **In the app:** The family tree shows the polyalphabetic family, where the Vigenère cipher lives.

Why it works

A tiny experiment

Write the word BANANA on six slips of paper and put them in a hat. Pull out one slip, keep it out, then pull a second slip. What is the chance that both slips show the same letter?

Count the ways. There are 6 slips for your first pick and 5 left for your second pick, so there are $6 \times 5 = 30$ ways to pull two slips in order. Now count the ways that give a match:

- Two A's: there are 3 A's for the first pick and 2 A's left for the second, so $3 \times 2 = 6$ ways.
- Two N's: $2 \times 1 = 2$ ways.
- Two B's: there is only one B, so $1 \times 0 = 0$ ways.

That gives $6 + 2 + 0 = 8$ matching ways out of 30, so the chance of a match is $8/30 \approx 0.267$. BANANA is a very "repetitive" word, so its matching chance is high.

Why English is lumpy

English uses its letters very unevenly. About 12.7% of the letters in ordinary English are E, about 9.1% are T, and fewer than 1 in 1000 are Z. When you pick two letters at random, the chance that both are E is $0.127 \times 0.127 \approx 0.016$. The chance that both are T is $0.091 \times 0.091 \approx 0.008$. Do this for all 26 letters and add the answers: you get about **0.066**.

Now imagine a language in which every letter is equally common, so each one is 1/26 of the text. The chance that both picks are A is $\frac{1}{26} \times \frac{1}{26}$, and the same for B, and so on. Twenty-six of those add up to exactly $\frac{1}{26} \approx$ **0.038**.

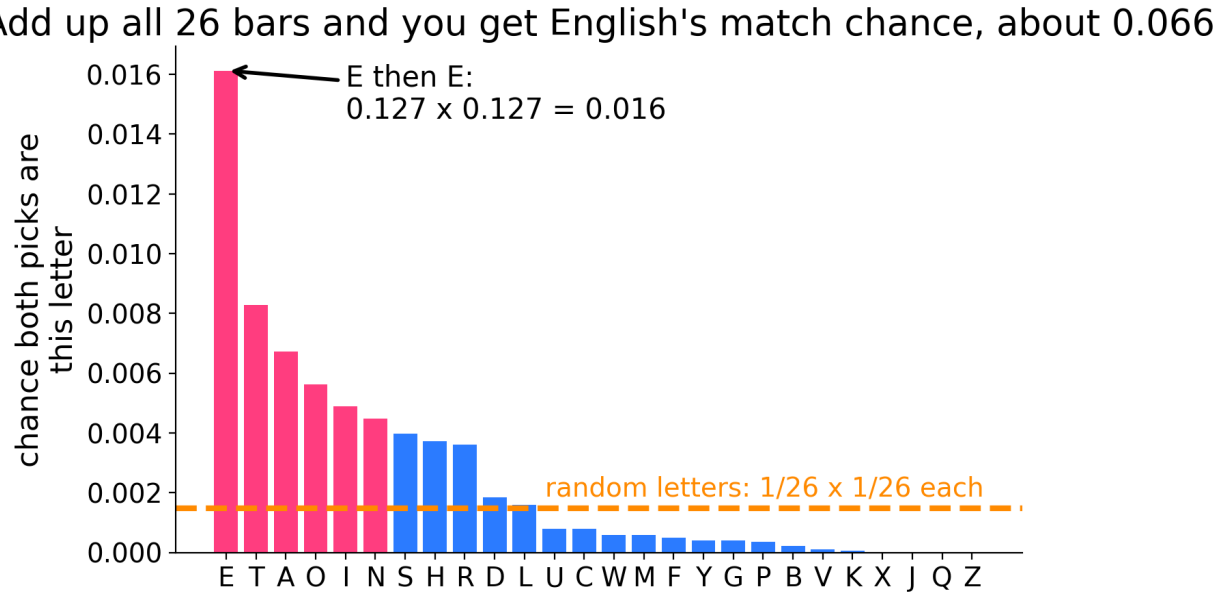


Figure 4.2: Bar chart, letters sorted from E down to Z. Each bar is the chance that both of two random picks are that letter. The E bar is the tallest at 0.016; a dashed orange line shows the much lower 1/26 times 1/26 value for random letters.

So the IoC measures **how lumpy** the letter counts are. Tall peaks and deep valleys give a high IoC. A flat spread gives a low one.

What encryption does to the lumps

Here is the key insight, and it is worth reading twice.

- **Substitution renames letters.** If every e becomes X, then X is now the most common letter. The peaks move to new letters, but they are just as tall. The IoC does not care which letter is tall, only how tall the peaks are. So the IoC stays the same.
- **Transposition moves letters around.** The letters are all still there, just in a different order. The counts do not change at all, so the IoC does not change at all.
- **Vigenère uses several alphabets.** In a Vigenère cipher, each letter is shifted by an amount that depends on its position. With the key `STORM`, the 1st, 6th, 11th… letters are shifted by S, the 2nd, 7th, 12th… by T, and so on. So the plaintext e's turn into five different ciphertext letters. Each

tall peak gets chopped into five smaller peaks spread across the alphabet. The counts flatten out, and the IoC drops toward 0.038.

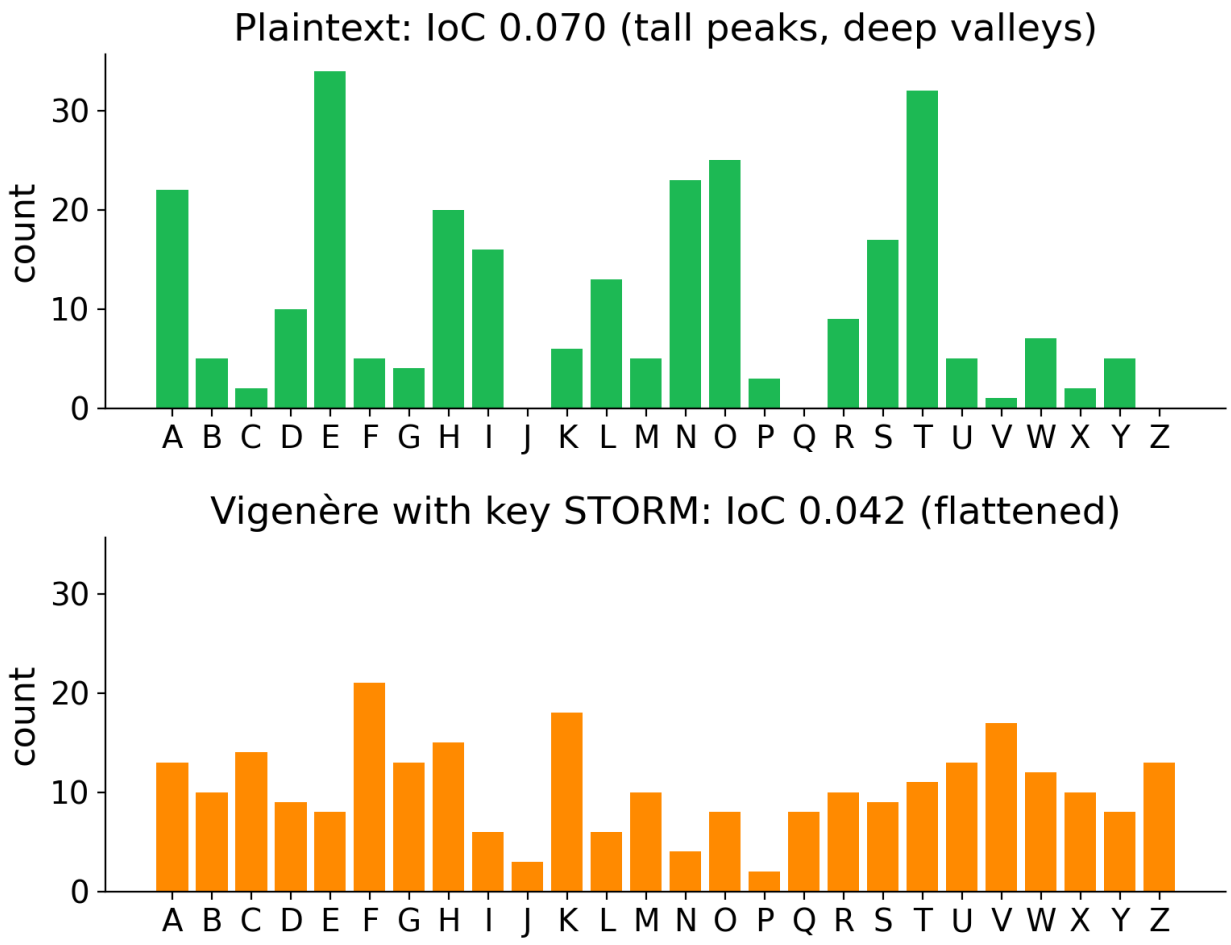


Figure 4.3: Two bar charts of letter counts for the same message. Top, the plaintext in green: tall peaks at E, T, O, N, A. Bottom, the same message after a Vigenère cipher with key STORM, in orange: the bars are much more even.

The grown-up version

Naming the pieces

Mathematicians use a few short symbols here. Each one is just a name for something you already did with BANANA.

- N is the total number of letters in the text. For BANANA, $N = 6$.
- f_A is the number of times A appears (the "frequency" of A). For BANANA, $f_A = 3$, $f_B = 1$, $f_N = 2$, and every other letter has $f = 0$.
- f_i means "the count of letter number i", where i runs through the alphabet: $i = 1$ for A, $i = 2$ for B, up to $i = 26$ for Z.

The sigma sign

The Greek capital letter sigma, $\sum$, means "add up". Writing

$$\sum_{i=1}^{26} f_i$$

means "work out f_i for every letter from $i = 1$ (A) to $i = 26$ (Z), and add all 26 answers together". That sum is just N, the total number of letters. Sigma is shorthand that saves you writing a sum with 26 terms.

The formula

In BANANA you worked out "count times (count minus one)" for each letter, added them up, and divided by "total times (total minus one)". With the symbols above, that is:

$$\text{IoC} = \frac{\sum_{i=1}^{26} f_i \, (f_i - 1)}{N \, (N - 1)}$$

Read it out loud like this: "For every letter, multiply its count by one less than its count. Add those up. Divide by the total number of letters times one less than the total." The top counts the ordered pairs of slips that match. The bottom counts all ordered pairs of slips. Check it on BANANA: the top is $3 \times 2 + 1 \times 0 + 2 \times 1 = 8$, the bottom is $6 \times 5 = 30$, and $8/30 \approx 0.267$.

Kappa: the numbers to remember

Cryptographers write the expected IoC of a language with the Greek letter kappa, κ.

- $\kappa_{\text{English}} \approx 0.066$. (Different counts of English give values from 0.065 to 0.067. Any of these is fine.)
- $\kappa_{\text{random}} = \frac{1}{26} \approx 0.038$.

If p_i is the fraction of English text that is letter i (so $p_E = 0.127$), then $\kappa_{\text{English}} = \sum_{i=1}^{26} p_i^2$, which is exactly the "add up all 26 bars" calculation from the chart above.

Some books multiply the IoC by 26, so English becomes about 1.73 and random text becomes 1.00. It is the same measurement on a different scale. Check which scale a tool uses before comparing numbers.

A worked example

Here is a 271-letter message, encrypted with a Vigenère cipher. We will pretend we do not know the key.

```
LASCU YAHYA MLSBQ WISIT SWBFF KICBQ FMCRZ QHBVR GKHYD WXKVQ
CLKYQ FMVVN GTHWU FTZCK UTAVU LUFFG YAHWX GNFFU DTGKM UDCWX
WMHVD KTBUM KFOCX OHCUQ FUCOI AMVEA FTAVA FBHZZ KBRVF ZXPFJ
OTGRN JTGJW WROEP SGCKQ LAOKE SBRFZ DRHYU KMVVX AZVKU KGCKI
```

```
ZTHZF KXSDE DHCBG FWSIF ZXGVH WGHYE LXDRF EBREU YAHRZ VMSCX
FHCEQ OAOKK GNTZZ VMVVD W
```

(The spaces every five letters are only there to make it easier to read. They are not word breaks.)

Step 1: the IoC of the whole message

Count every letter. The most common are F (21 times), K (18), V (17) and H (15). The rarest are P (2) and J (3). Now work out $f(f-1)$ for each of the 26 letters and add them up. For example F gives $21 \times 20 = 420$ and K gives $18 \times 17 = 306$. All 26 together come to 3064. The bottom of the fraction is $271 \times 270 = 73170$. So:

$$\text{IoC} = \frac{3064}{73170} \approx 0.042$$

That is far below English (0.066) and close to random (0.038). It is not a substitution and not a transposition. Something has flattened the letter counts: our first suspect is the Vigenère family.

For comparison, the plaintext (the message before encryption) has an IoC of $5086/73170 \approx 0.070$. That is a little above 0.066, which is normal for a short text with lots of t's and e's. If we encrypt the same plaintext with a keyword substitution, or shuffle it with a columnar transposition, the IoC stays at exactly 0.070.

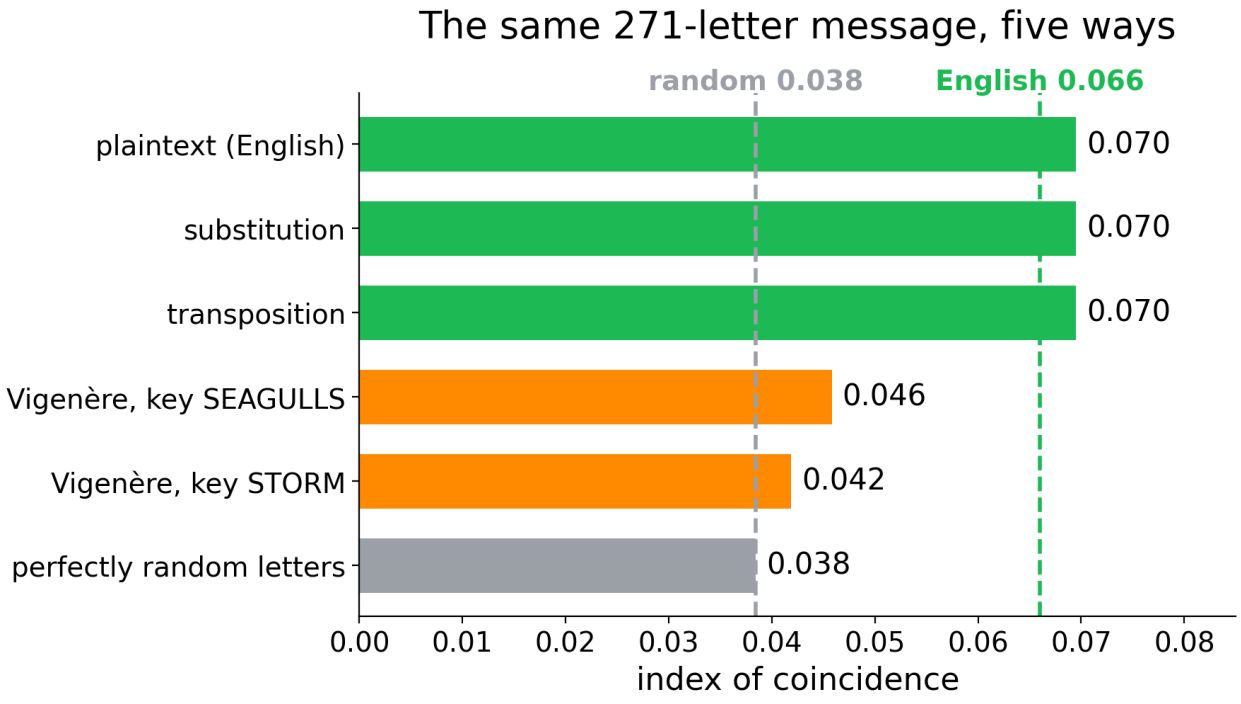


Figure 4.4: Horizontal bars showing the index of coincidence of the same 271-letter message: plaintext, substitution and transposition all at 0.070, near the English line at 0.066; two Vigenère versions at 0.046 and 0.042, near the random line at 0.038.

Step 2: split into columns

If the key has 5 letters, then every 5th letter was shifted by the same amount. Write the ciphertext in rows of 5. Each column is then a plain Caesar cipher: one alphabet, shifted. A Caesar cipher keeps the lumps of English, so each column should have an IoC near 0.066.

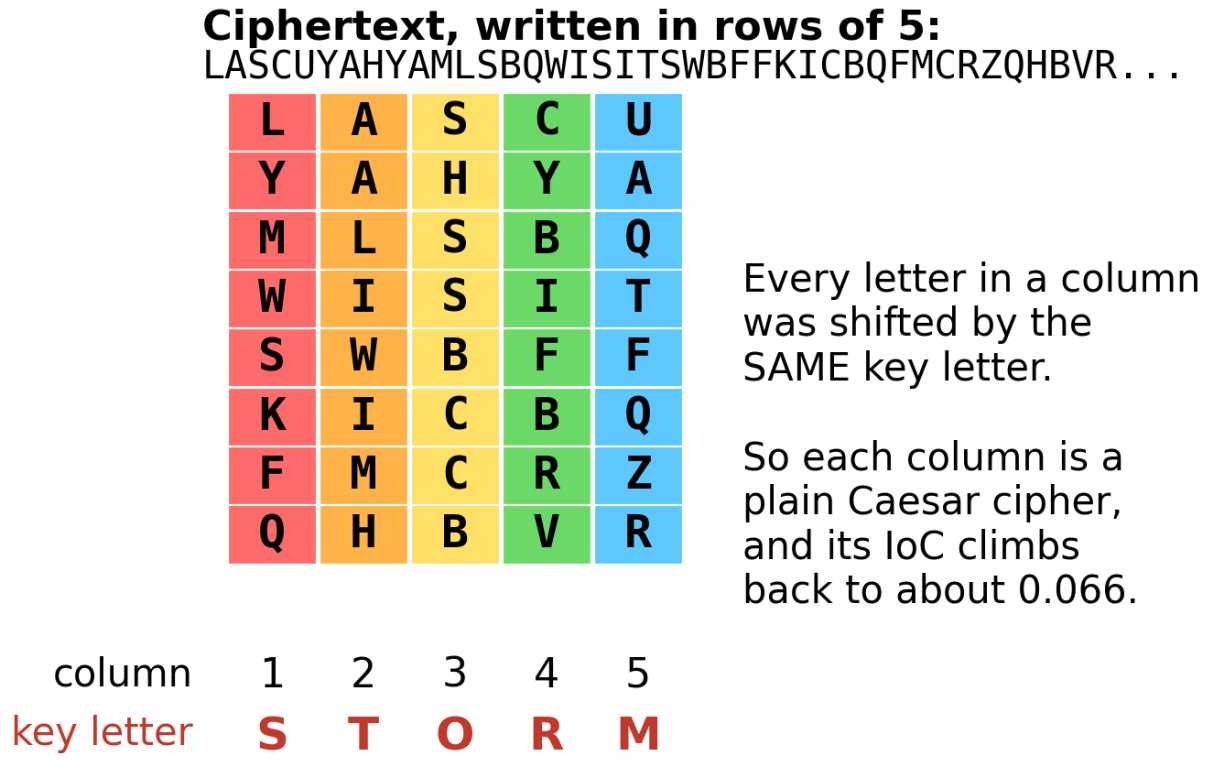


Figure 4.5: The ciphertext written in rows of five letters, each column painted a different color. Under each column is the key letter that shifted it: S, T, O, R, M.

We do not know the key length yet, so we try them all. For each guess L (for "length"), split the ciphertext into L columns, work out the IoC of each column, and take the average. For the guess $L = 5$, the five columns have 55, 54, 54, 54 and 54 letters, and their IoCs are:

Column	1	2	3	4	5
IoC	0.063	0.072	0.089	0.076	0.060

The average is $(0.063 + 0.072 + 0.089 + 0.076 + 0.060)/5 \approx 0.072$. That looks like English.

Step 3: compare all the guesses

Guess L	1	2	3	4	**5**	6	7	8	9	**10**	11	12	13	14	**15**
Average IoC	.042	.043	.045	.045	**.072**	.048	.040	.044	.050	**.070**	.041	.048	.040	.043	**.080**

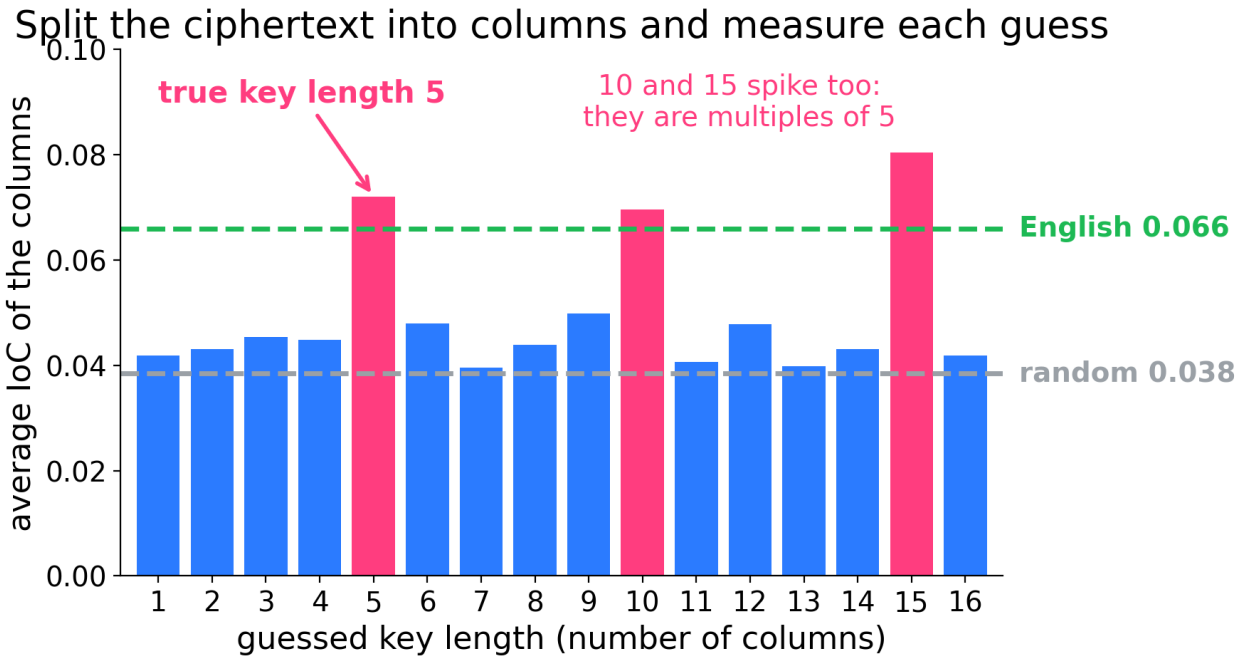


Figure 4.6: Bar chart of the average column IoC for guessed key lengths 1 to 16. Most bars sit near the random line. Bars at 5, 10 and 15 jump up past the English line.

Three guesses jump up: 5, 10 and 15. Why three? If the key is `STORM`, then splitting into 10 columns also puts only same-shifted letters together: column 1 holds the 1st, 11th, 21st… letters, which were all shifted by S. Any multiple of the true length works. So **choose the smallest length that jumps**: 5.

Notice that 15 scored even higher than 5. With 15 columns, each column holds only 18 letters, and short columns give jumpy numbers (more on that below). Never pick the tallest bar blindly.

Step 4: what next?

Now you know you are facing five Caesar ciphers, side by side. Solve each column with frequency analysis (see Guide 02, Frequency analysis, and Guide 05, Repeats and key length, which finishes this kind of example). For the record, the key was `STORM`, and the message begins `the lighthouse keeper had not spoken to anyone...`.

> **In the app:** Counts shows the IoC of any text, with English (0.067) and random letters (0.038) marked. Vigenère cuts a text into slices for each key length and shows each slice's IoC.

Friedman's estimate

William Friedman, the American cryptologist who invented the index of coincidence around 1920, also found a way to guess the key length straight from the IoC of the whole message, with no splitting at all.

Here is the idea. Pick two letters at random from a Vigenère message with a key of length L. Either they are in the same column, or they are not.

- They land in the same column about $\frac{1}{L}$ of the time. Then they were shifted by the same amount, so they match as often as English letters do: κ_{English}.
- The rest of the time, $1 - \frac{1}{L}$, they were shifted by different amounts, and they match about as often as random letters: κ_{random}.

So we expect

$$\text{IoC} \approx \frac{1}{L}\,\kappa_{\text{English}} + \left(1 - \frac{1}{L}\right)\kappa_{\text{random}}$$

Solve that for L and you get **Friedman's estimate**:

$$L \approx \frac{\kappa_{\text{English}} - \kappa_{\text{random}}}{\text{IoC} - \kappa_{\text{random}}} = \frac{0.066 - 0.038}{\text{IoC} - 0.038}$$

Try it on our message, where the IoC was 0.0419:

$$L \approx \frac{0.066 - 0.0385}{0.0419 - 0.0385} = \frac{0.0275}{0.0034} \approx 8$$

The true answer is 5, so Friedman's guess is rough. Look at the bottom of that fraction: it is a tiny difference, 0.0034. A wobble of 0.002 in the IoC (perfectly normal for 271 letters) changes the answer a lot. Treat Friedman's estimate as a ballpark ("small key, not a huge one") and let the column test give the exact length. Longer texts make it more reliable. (There is a more exact version of the formula that also uses N, but for a few hundred letters it gives nearly the same answer, here 7.9.)

When to try it

- **Always, early.** The IoC takes a few minutes by hand and seconds with a spreadsheet. Work it out for every new ciphertext, right after counting the letters.
- **IoC near 0.066 and the counts look like English** (e, t, a, o at the top): a transposition. The letters are right, only their order is wrong.
- **IoC near 0.066 but the wrong letters are common**: a substitution of some kind (Caesar, affine, keyword, general).
- **IoC between 0.038 and about 0.055**: several alphabets. Run the column test. A clear jump at some length means Vigenère, Beaufort or a relative.
- **Low IoC and no length jumps**: think autokey, running key, or a cipher that works on pairs of letters.

When not to

- **Very short messages** (under about 50 letters). The numbers wobble too much to trust.
- **Messages made of digits or symbols** that stand for pairs of letters (such as a Polybius square). Turn them back into single symbols first, or the IoC measures the wrong thing.
- When you already know the cipher type from the story or from a hint. Use the IoC to check, not to waste time.

Traps and tips

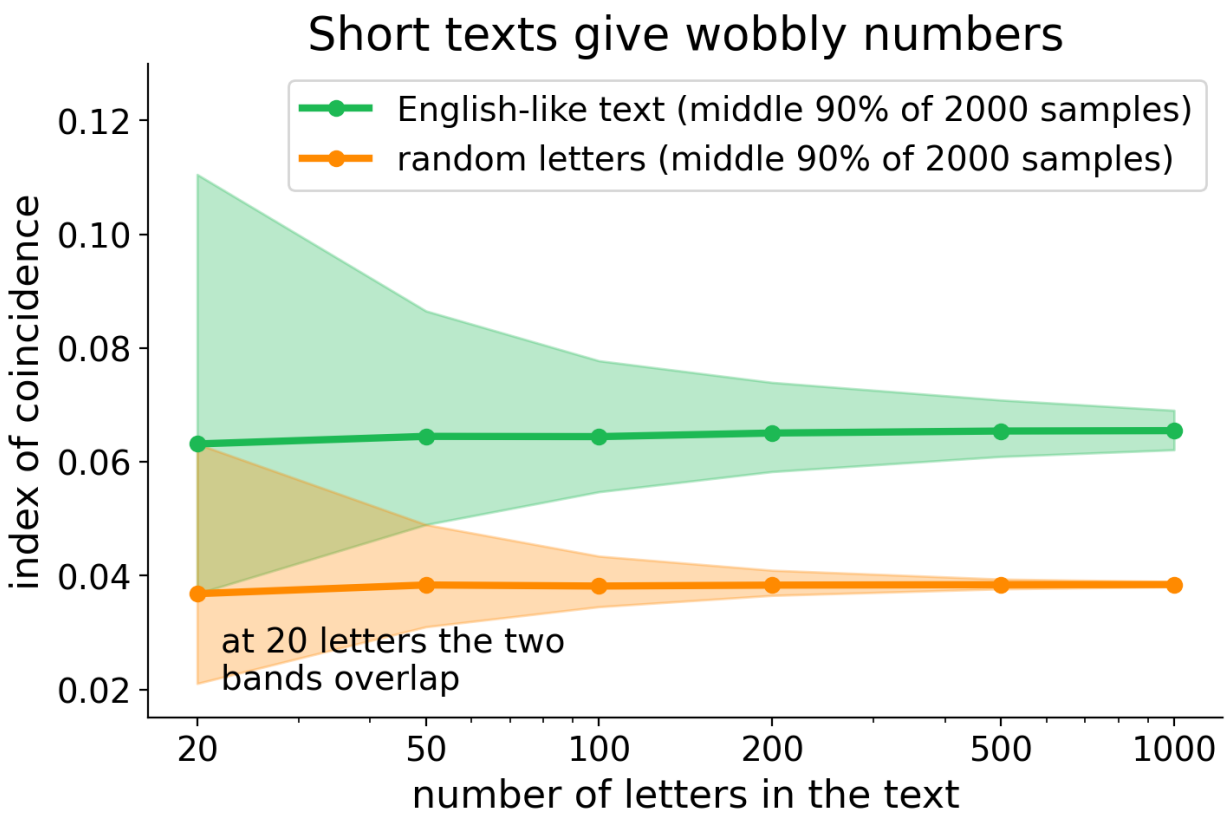


Figure 4.7: Two shaded bands showing the IoC of many simulated texts. For English-like text the band is centered near 0.066, and for random letters near 0.038. At 20 letters both bands are wide and overlap; by 500 letters they are thin and far apart.

- **Short texts give wobbly numbers.** In the chart above, 20-letter samples of English-like text land anywhere between 0.037 and 0.11, and random ones between 0.021 and 0.063. The two ranges overlap. By 200 letters, English-like text stays between about 0.058 and 0.074, and random text between about 0.037 and 0.041. So the longer the text, the more you can trust the IoC.
- **Columns are shorter than the whole text.** A 271-letter message split into 15 columns gives columns of only 18 letters. Their IoCs are noisy, which is why a multiple of the key length can score even higher than the key length itself.
- **Unusual texts have unusual IoCs.** A list of names, a text full of numbers spelled out, or a message that avoids the letter e can sit well away from 0.066. Use 0.060 to 0.075 as “English-

like", not exactly 0.066.

- **Strip spaces and punctuation first.** Count only letters, and make capitals and small letters the same.
- **Two layers of cipher.** A substitution followed by a transposition still has an English-like IoC, because neither step flattens the counts. If the IoC says "one alphabet" but nothing reads, suspect two steps.
- **Do not round too early.** Keep at least four decimal places while you work. The differences that matter are in the third decimal place.

By hand or by computer

The IoC is arithmetic on counts: a calculator's job, and a computer's when there are many of them. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
The IoC of the whole text	15 to 20 minutes	Counts[2], letter-count sheet[3]	dCode[4]
Split into L columns and average their IoCs, for L from 1 to 10	2 to 3 hours	Vigenère[5], Vigenère sheet[6]	Guballa[7], dCode[8]
Friedman's estimate of the key length	2 minutes with a calculator	Vigenère sheet[9]	n/a

Try it yourself

1. Work out the IoC of the word LETTERS. (Hint: $N = 7$.)
2. A 400-letter ciphertext has an IoC of 0.067. When you count the letters, the most common are E, T, A and O, in about the right amounts for English. What kind of cipher is it most likely to be, and why?
3. A ciphertext has an IoC of 0.044. Use Friedman's estimate (with 0.066 and 0.038) to guess the key length. Then the column test gives averages of 0.044 for length 3, 0.068 for length 6, 0.045 for length 7 and 0.069 for length 12. What key length do you choose?

[1] Brain, pencil or computer?, the chapter before Guide 1.

[2] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English's, the IoC, the length and its factors.

[3] Letter counts and IoC, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-letter-counts, to copy into Google Sheets or download for Excel.

[4] dCode's index of coincidence: dcode.fr/index-coincidence.

[5] Vigenère, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/vigenere. Slices for any key length, with each slice's IoC.

[6] Vigenère slices and IoC by key length, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-vigenere-slices, to copy into Google Sheets or download for Excel.

[7] Guballa's Vigenère solver: guballa.de/vigenere-solver.

[8] dCode's Vigenère solver: dcode.fr/vigenere-cipher.

[9] Vigenère slices and IoC by key length, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-vigenere-slices, to copy into Google Sheets or download for Excel.

In the app: Spot the cipher asks you to tell one alphabet from several by the IoC. The Vigenère exercise takes you from the key length to the answer.

Words to know

- **Index of coincidence (IoC)**: the chance that two letters picked at random from a text are the same letter.
- **Kappa (κ)**: the IoC you expect from a language or from random letters: about 0.066 for English and $1/26 \approx 0.038$ for random letters.
- **Sigma ($\sum$)**: a sign that means "add up all of these".
- **Frequency (f)**: how many times a letter appears in a text.
- **Monoalphabetic**: using one cipher alphabet for the whole message (Caesar, substitution).
- **Polyalphabetic**: switching between several cipher alphabets (Vigenère, Beaufort).
- **Period or key length**: how many letters the key has before it repeats.
- **Column test**: splitting a ciphertext into L columns and averaging their IoCs to test whether the key length is L.
- **Friedman's estimate**: a formula that guesses the key length from the IoC of the whole message.

Going further

- Index of coincidence (Wikipedia)
- Index of coincidence (Practical Cryptography)
- William F. Friedman (Wikipedia)
- Vigenère cipher (Wikipedia)
- Simon Singh, *The Code Book* (1999), chapter 2, on how the Vigenère cipher was finally broken.
- The National Cipher Challenge: hints, tips and cribs

Answers

1. LETTERS has L 1, E 2, T 2, R 1, S 1. The top is $0 + 2 + 2 + 0 + 0 = 4$ (from E: 2×1 and T: 2×1). The bottom is $7 \times 6 = 42$. IoC $= 4/42 \approx 0.095$.
2. A transposition. The IoC is English-like, so only one alphabet is in play. And the common letters are the right ones (E, T, A, O), so the letters have not been renamed, only moved.
3. Friedman: $(0.066 - 0.038)/(0.044 - 0.038) = 0.028/0.006 \approx 4.7$, so "around 5". The column test jumps at 6 and 12, and 12 is a multiple of 6. Choose **6**, the smallest length that jumps. (Friedman was close, but not exact, as usual.)

Guide 5

Repeats and key length

The Kasiski examination: how repeated fragments give away the length of a Vigenère key

By the end you can find repeated fragments in a Vigenère ciphertext, turn the gaps between them into the length of the key, and then break the cipher one column at a time.

The big idea

A Vigenère cipher hides a message by shifting each letter by a different amount, following a keyword that repeats over and over. That sounds unbreakable, and for 300 years many people thought it was. But English repeats itself. Words like “the” and “and” come up again and again. Every so often, a repeated word happens to sit under the same part of the keyword, and then it is encrypted the same way both times. The ciphertext shows a repeat. The gap between the two copies must then be a whole number of keywords long. Collect a few gaps, find the number that divides them all, and you have the length of the key. This trick was published by the Prussian army officer Friedrich Kasiski in 1863 (Charles Babbage seems to have found it earlier but never published it), and it is called the **Kasiski examination**.

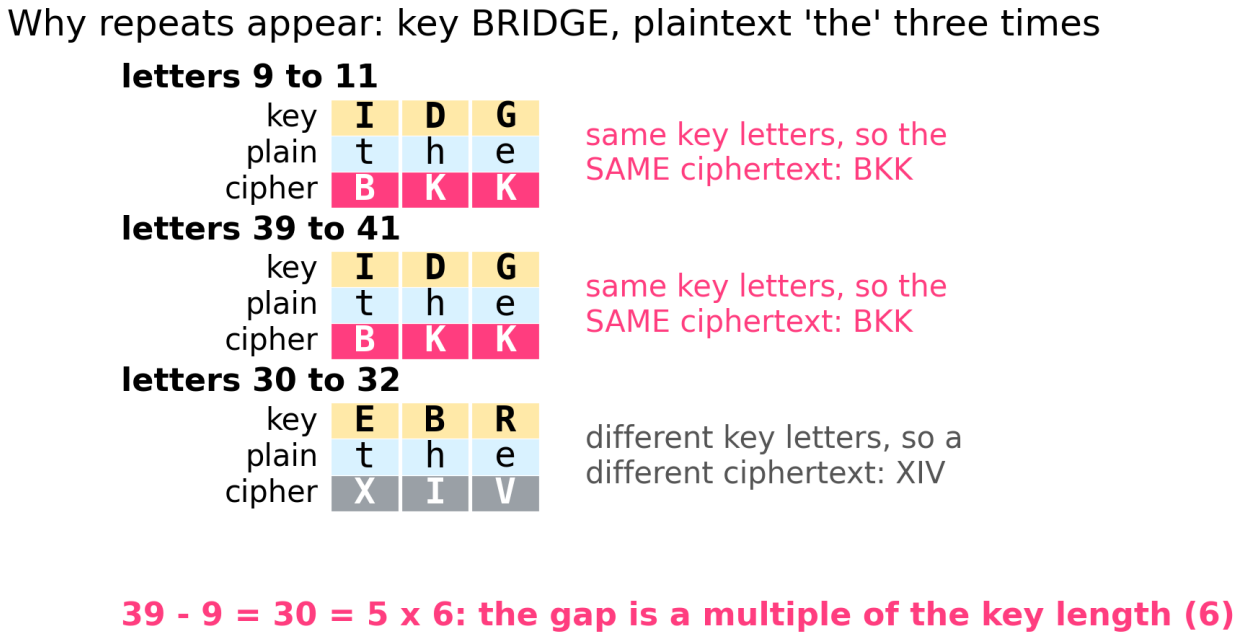


Figure 5.1: Three copies of the word “the” encrypted with the key BRIDGE. At letters 9 and 39 the word sits under the same key letters I, D, G, so both copies become BKK. At letter 30 it sits under E, B, R and becomes XIV. The gap 39 minus 9 is 30, which is 5 times 6, a multiple of the key length.

Which ciphers it breaks

Cipher family	Does Kasiski help?	How
Vigenère	Yes	Gaps between repeats are multiples of the key length. Then solve each column as a Caesar shift.
Beaufort, variant Beaufort	Yes	Same repeats, same gaps. Only the rule for each column is different.
Porta, Gronsfeld, other repeating-key ciphers	Yes	Anything whose key repeats with a fixed length leaves the same kind of repeats.
Autokey, running key	No	The key never repeats, so repeats are accidents. The gaps have no common factor.
Caesar, substitution	Not needed	Repeats appear, but every gap “works”, because the key length is 1.
Transposition	No	Repeats of plaintext words are broken up when the letters move.

Why it works

A quick reminder of the Vigenère cipher

Number the letters from A = 0 to Z = 25. To encrypt, write the keyword over and over above the message. Shift each message letter forward by the number of the key letter above it, and wrap around from Z back to A. With the key `BRIDGE`:

```
key:     B R I D G E B R I D G E B R ...
plain:   m e e t m e a t t h e o l d ...
cipher:  N V M W S I B K B K K S M U ...
```

The first letter m (12) is shifted by B (1) to give N (13). The second letter e (4) is shifted by R (17) to give V (21). And so on. Notice that the plaintext e becomes V, then M, then I, then K: the same plaintext letter turns into different ciphertext letters. That is what makes Vigenère harder than a Caesar cipher.

When repeats happen

The key `BRIDGE` has 6 letters, so the key letters line up again every 6 letters. Letters 1, 7, 13, 19, … are all shifted by B. Letters 2, 8, 14, … are all shifted by R. And so on.

Now suppose the word “the” appears at letter 9 and again at letter 39. The gap is $39 - 9 = 30$, and 30 is 5×6. So both copies sit under exactly the same key letters (I, D, G), and both become `BKK`. The ciphertext shows a repeat.

If “the” appears at letter 30, the gap from 9 is 21, which is not a multiple of 6. That copy sits under different key letters (E, B, R) and becomes `XIV`. No repeat.

So the rule is: **a repeat in the ciphertext usually means the same plaintext under the same key letters, so the gap is a multiple of the key length.** “Usually”, because two different pieces of plaintext can now and then produce the same three ciphertext letters by accident. We will deal with those accidents below.

A pencil-and-paper experiment

Write any sentence that uses “the” several times. Write the keyword `CAT` over and over above it, and encrypt only the letters of each “the”. Circle the copies of “the” that come out the same. Number the letters of the sentence (ignoring spaces) and subtract the starting positions of two circled copies: every such gap will be a multiple of 3.

> **In the app:** Repeats lists the runs of letters that repeat in your text, the gaps between them, and how often each number divides a gap. Its Comb view finds the key length another way, by sliding the text against itself.

The grown-up version

Factors and multiples

A **factor** (or **divisor**) of a number is a whole number that divides it with nothing left over. The factors of 18 are 1, 2, 3, 6, 9 and 18. A **multiple** of 6 is any number you get by multiplying 6 by a whole number: 6, 12, 18, 24, 30, …

If every gap is a multiple of the key length, then the key length is a factor of every gap. So we are looking for a number that is a factor of all the gaps at once: a **common factor**.

The greatest common divisor

The biggest number that divides all of a set of numbers is their **greatest common divisor**, written gcd. For example:

- Factors of 18: 1, 2, 3, **6**, 9, 18
- Factors of 30: 1, 2, 3, 5, **6**, 10, 15, 30
- Factors of 102: 1, 2, 3, **6**, 17, 34, 51, 102

The factors that appear in all three lists are 1, 2, 3 and 6. The greatest is 6, so $\gcd(18, 30, 102) = 6$.

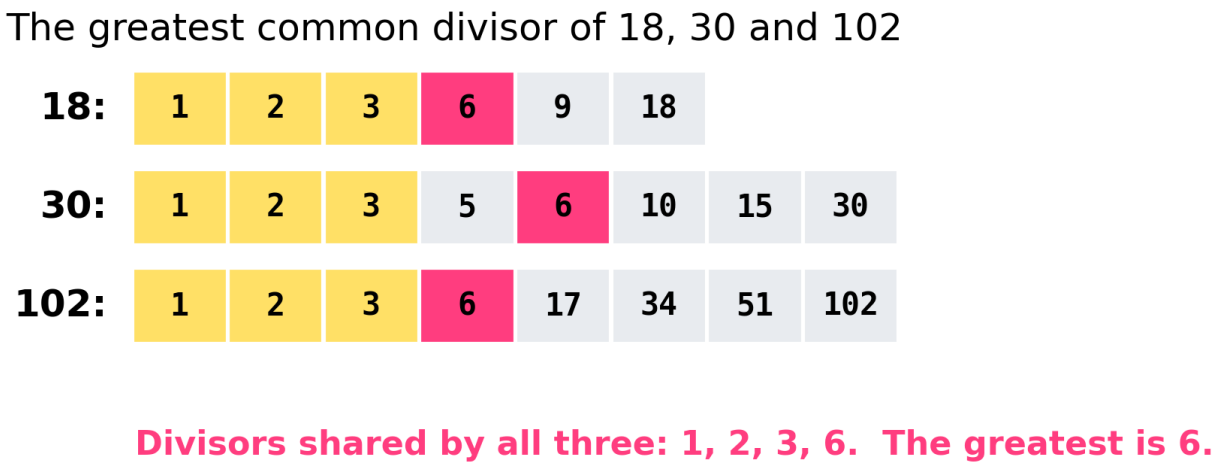


Figure 5.2: The divisors of 18, 30 and 102 laid out as rows of tiles. The divisors 1, 2 and 3 are yellow in every row, and 6 is pink in every row: the greatest number that divides all three.

Listing factors gets slow for big numbers. Euclid, the Greek mathematician, found a faster way more than 2000 years ago: **replace the bigger number by the remainder when you divide it by the smaller one, and repeat until the remainder is 0.** The last number that is not 0 is the gcd. For 102 and 30:

- $102 = 3 \times 30 + 12$, so the remainder is 12. Now find $\gcd(30, 12)$.
- $30 = 2 \times 12 + 6$, so the remainder is 6. Now find $\gcd(12, 6)$.
- $12 = 2 \times 6 + 0$. The remainder is 0, so the answer is **6**.

To find the gcd of three or more numbers, find the gcd of the first two, then the gcd of that answer with the next number, and so on.

Why the gcd might be too big or too small

If every gap you found happened to be a multiple of 12 while the key has 6 letters, the gcd says 12. That is harmless: the key `BRIDGEBRIDGE` still breaks the message. The real danger is the opposite: one accidental repeat can drag the gcd down to 1. So careful codebreakers do not just take the gcd of everything. They make a **factor table** and count.

A worked example

Here is a 218-letter ciphertext. It was made with a Vigenère cipher, and we will pretend we do not know the key.

```
NVMWS IBKBK KSMUU LRPBK VLTIC IQQMX IVUDV EOUBK KPBDX WNINR
XVNSX JBKKT BKPIX SNKPH UPEDQ ORXPK PHXMW VZDTH UYMUO ZFITH
GHTKW WNIDR DHOJU YMSGX IZAIR SPUMG CEJKI WZLFF TGSMM CCQZM
MUIZT EOUBK KRGFT OUAUY MUOZF IVRXX IUWQU XMZOK ZXIVT DSTVE
BLRCP LZHGG IKPHI EWV
```

(The spaces every five letters are only to help your eyes. Count letters, not groups.)

Step 1: find the repeats

Look for groups of three or more letters that appear more than once. Three is a good minimum: pairs repeat by accident far too often. Number the letters from 1, and write down where each repeat starts.

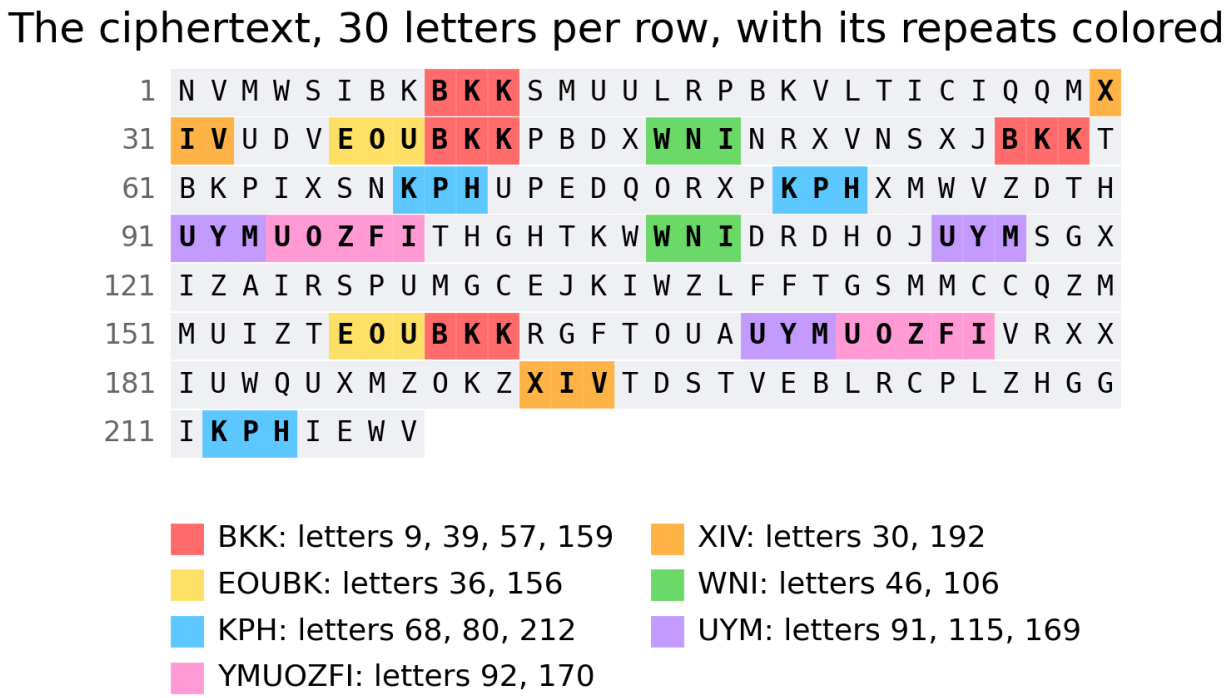


Figure 5.3: The ciphertext written 30 letters per row. Each repeated group has its own color: BKK in red appears four times, EOUBK in yellow twice, XIV in orange twice, WNI in green twice, KPH in blue three times, UYM in purple three times, and YMUOZFI in pink twice.

Repeat	Starts at letter	Gaps between neighbors
BKK	9, 39, 57, 159	30, 18, 102
XIV	30, 192	162

Repeat	Starts at letter	Gaps between neighbors
EOUBK	36, 156	120
WNI	46, 106	60
KPH	68, 80, 212	12, 132
UYM	91, 115, 169	24, 54
YMUOZFI	92, 170	78

Long repeats like YMUOZFI (7 letters) are the most trustworthy: seven letters almost never match by accident.

Step 2: build a factor table

For each gap, check which of the numbers 2 to 10 divide it. A table makes the pattern jump out.

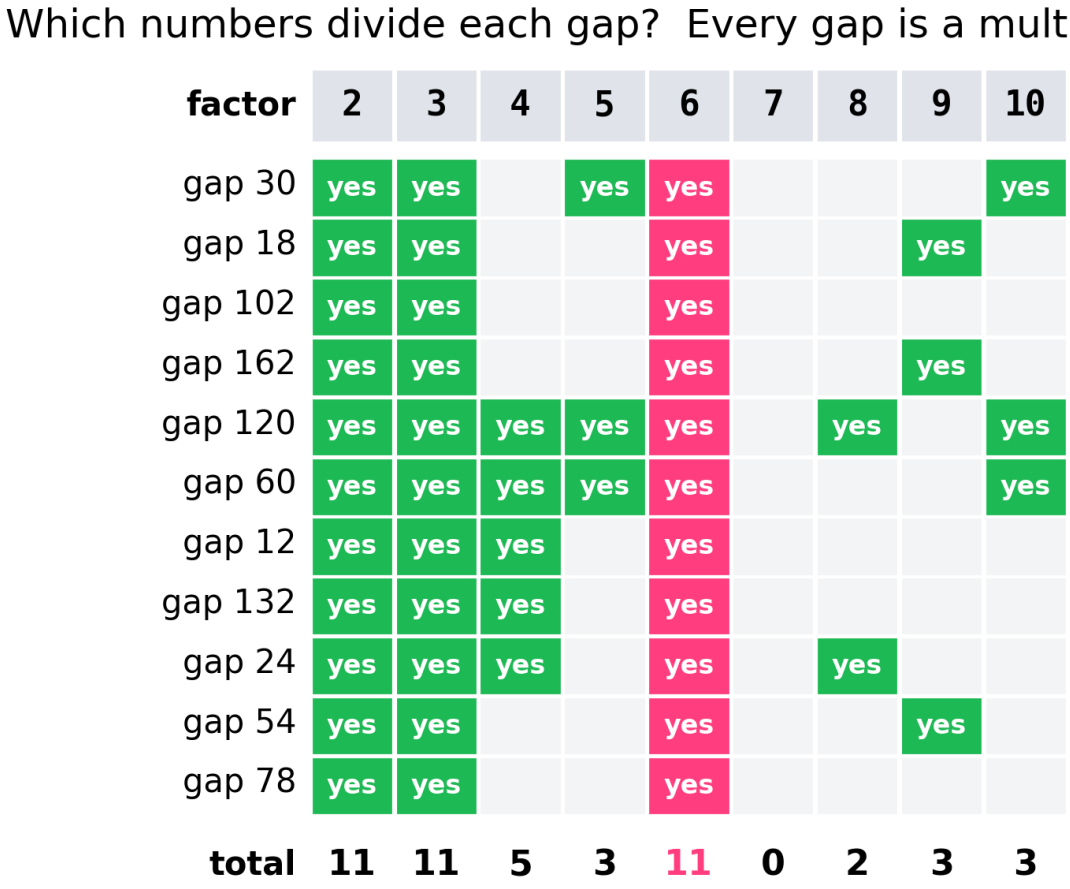


Figure 5.4: A table with one row per gap (30, 18, 102, 162, 120, 60, 12, 132, 24, 54, 78) and one column per factor from 2 to 10. A cell says yes when the factor divides the gap. The column for 6 is pink and says yes in every row. Totals: 2 divides 11 gaps, 3 divides 11, 4 divides 5, 5 divides 3, 6 divides 11, 7 divides none, 8 divides 2, 9 divides 3, 10 divides 3.

Factors 2, 3 and 6 each divide all 11 gaps. So the key length could be 2, 3 or 6. When several factors tie, the biggest of them is the best guess, because any key length that works makes all of its own factors work too. A 6-letter key makes every gap a multiple of 2 and of 3 as well. Here, $\gcd(30, 18, 102, 162, 120, 60, 12, 132, 24, 54, 78) = 6$.

Our guess: the key has 6 letters. If you know about the index of coincidence (Guide 04), you can double-check: splitting this ciphertext into 6 columns gives an average column IoC of 0.067, like English, while 3 columns give only 0.055 and 2 columns 0.048.

Step 3: split into columns

Write the ciphertext in rows of 6. Column 1 holds letters 1, 7, 13, 19, …; column 2 holds letters 2, 8, 14, 20, …; and so on. Every letter in one column was shifted by the same key letter, so **each column is a Caesar cipher**. Columns 1 and 2 have 37 letters each; columns 3 to 6 have 36.

Step 4: solve each column as a Caesar shift

Now we use frequency analysis (Guide 02, Frequency analysis, explains it fully). In plain English, e is the most common letter, then t and a. In a Caesar column, the same pattern of tall and short bars appears, just slid along the alphabet. So slide the English pattern along until its peaks line up with the column's peaks.

Here is column 2. Its most common letters are K (8 times), U (6) and V (5).

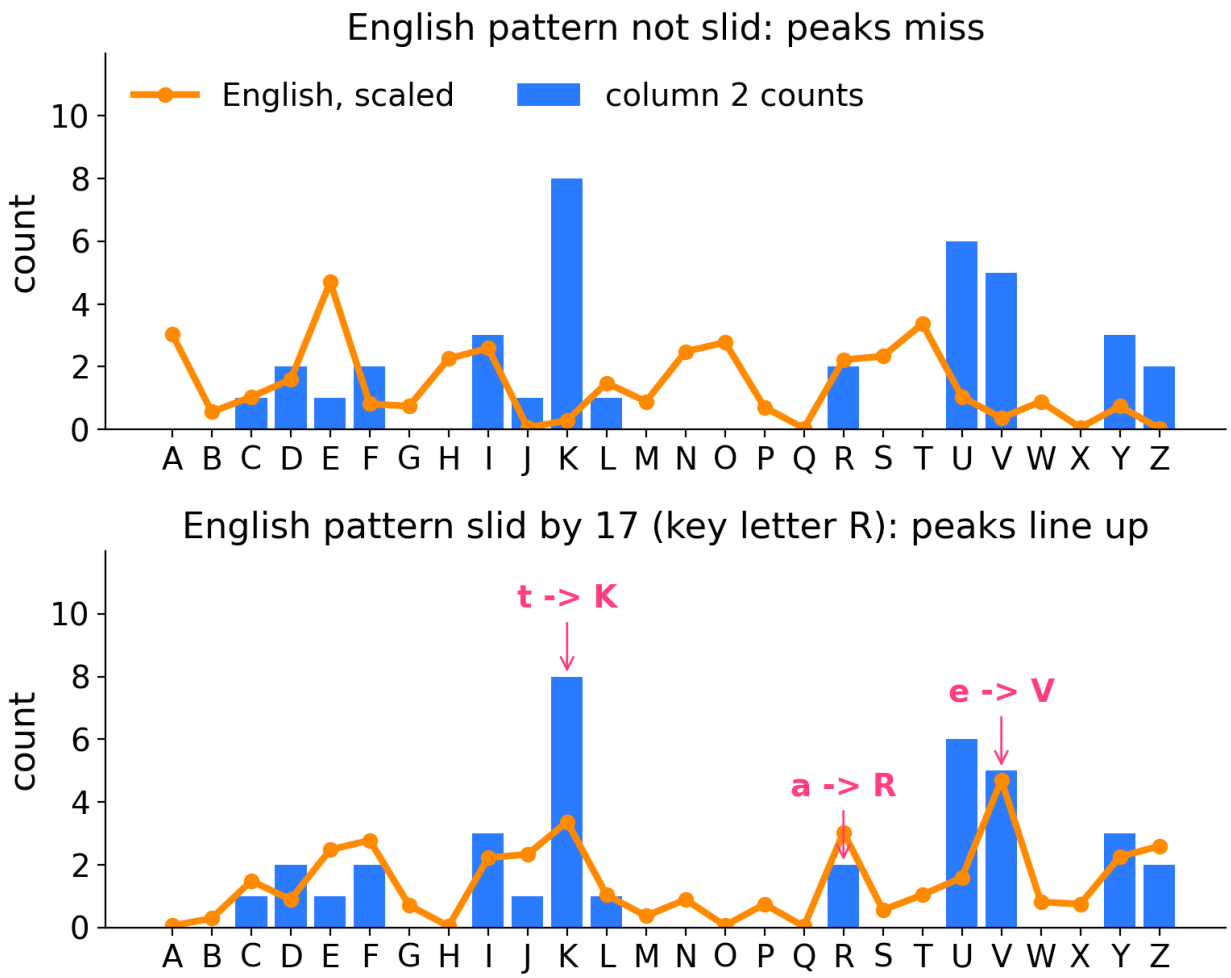


Figure 5.5: Two bar charts of column 2's letter counts in blue, with the English letter pattern drawn as an orange line. Top: the English pattern is not slid, and its peaks miss the blue bars. Bottom: the English pattern is slid by 17 places, for key letter R, and its peaks land on the blue bars: t becomes K, e becomes V, and a becomes R.

A tempting shortcut is "the most common letter is e". That would make K = e, a shift of 6 (key letter G). But then the next most common letters U and V would stand for o and p, and p is rare in English. Try instead K = t: that is a shift of 17 (key letter R), and then U = d and V = e. All three are common. Comparing the whole pattern, not just one letter, picks R clearly. (A score called **chi-squared**, explained in Guide 02, does this comparison with numbers: for column 2 it gives 29 for R and 95 for the next best shift.)

Doing this for all six columns:

Column	Most common letters	Best shift	Key letter
1	I (5), B (4), M (4)	1	**B**
2	K (8), U (6), V (5)	17	**R**
3	M (5), B (5), P (4), T (4)	8	**I**
4	H (6), K (5), W (4)	3	**D**

Column	Most common letters	Best shift	Key letter
5	K (4), R (4), S (3), T (3)	6	**G**
6	X (6), I (4), S (4), E (4)	4	**E**

The key letters spell BRIDGE. "Most common letter = e" would have got only four of the six columns right (columns 2 and 6 fail), which is why you compare whole patterns. Shift every letter back by its key letter and the message appears:

```
meet me at the old mill at nine. bring the map and the lamp. the map shows the
path from the old mill to the river...
```

When the key letters spell a real word, you almost certainly have it right. If one column gives a strange letter (BRIDXE), re-check that column.

> **In the app:** Once you have a key length, Vigenère splits the text into slices and lets you solve each slice as a Caesar, one at a time or all side by side.

A variant: the Beaufort cipher

The Beaufort cipher, named after Admiral Francis Beaufort (the same man as the Beaufort wind scale), uses a repeating keyword just like Vigenère, but with a different rule. Instead of "plain plus key", it is "key minus plain":

$$\text{cipher} = \text{key} - \text{plain}$$

If the answer is below 0, add 26. For example, with key letter B (1) and plain letter t (19): $1 - 19 = -18$, and $-18 + 26 = 8$, which is I. To decrypt you do exactly the same thing, plain $=$ key $-$ cipher: $1 - 8 = -7$, and $-7 + 26 = 19$, which is t. The same operation locks and unlocks. That makes Beaufort a **reciprocal** cipher.

Here is the same message encrypted with Beaufort and the key BRIDGE:

```
PNEKU ABYPW CQQOW VVTBY VVTAA AAQAL UNWDR EOOPW CTBFT KZAPR
TLZQF ZPWCP BYBYP QPYBZ STYFA SVLNY BZPWG NRDTB IKEMY JXAXZ
GBJYU KZAZR NZYZI KEOGL UJQYV QNOEA KETYI KNXXD XAUWQ GOQNW
QOIHT EOOPW CRWDX SSIIK EMYJX AVPPL UOUQS LQJCW NLUNX DUPHE
PVVGN XRZGC UYBZE EGN
```

Kasiski still works. The key still repeats every 6 letters, so the same plaintext under the same key letters still gives the same ciphertext. The repeats are at the same places as before (PWC at letters 9, 39, 57 and 159, and so on), with the same gaps.

This time there are accidents. Four extra repeats turn up that are not the same plaintext at all: BYP (gap 56), VVT (gap 5), AAA (gap 1) and ZPW (gap 26). If you took the gcd of all 15 gaps you would get 1, which is useless. The factor table saves you: 6 still divides 11 of the 15 gaps, and the loners (5 and 1) stand out as accidents. **Throw out the gaps that disagree with everything else.**

Solving a column is slightly different. In a Beaufort column, key $=$ cipher $+$ plain. Column 2's most common letter is Y (24). If Y stands for t (19), the key letter is $24 + 19 = 43$, and $43 - 26 = 17$, which is R. Once again the key comes out as BRIDGE.

How do you know it is Beaufort and not Vigenère? Solve the columns with both rules: the rule whose key letters spell a word is the right one. There is also a third cousin, **variant Beaufort** (cipher = plain − key), which is the same as Vigenère decryption, so a Vigenère solver breaks it with a "backwards" key.

Why autokey resists Kasiski

The **autokey** cipher starts with a short keyword (the **primer**) and then uses *the message itself* as the rest of the key. With primer `BRIDGE`, the first six letters are shifted by B, R, I, D, G, E, and from then on each letter is shifted by the plaintext letter six places earlier. The key never repeats with a fixed length, so there is no length to find.

Here is our message in autokey:

```
NVMWS IMXXA QSLWF PPZLW ZQYPB KVVTX IVUNV TUHFH TLNPI ALPMM
ELOSI SIZLD WLAMV DMMOJ FZPFP PZERF PPCBJ XYEEL OLVRV YXYPV
IYWKZ XHHUT JXPJV HZTIY APWUL HVLWI HOWWE WPHMH LWFPP ZFQFQ
WOUJG IYGTD RNSRE SSJYV PCWRX YRFZO LUBBF MSLUU HMSMR HFIBR
EIXNI HKMLA VNYIC CCX
```

It has far fewer repeats: `LWFP` at 13 and 141, `FPP` at 15, 74, 80 and 143, `PPZ` at 16, 75 and 144, and `EL0` at 51 and 89. The gaps are 128, 59, 6, 63, 59, 69 and 38. Their gcd is 1, and 6 divides only one of them.

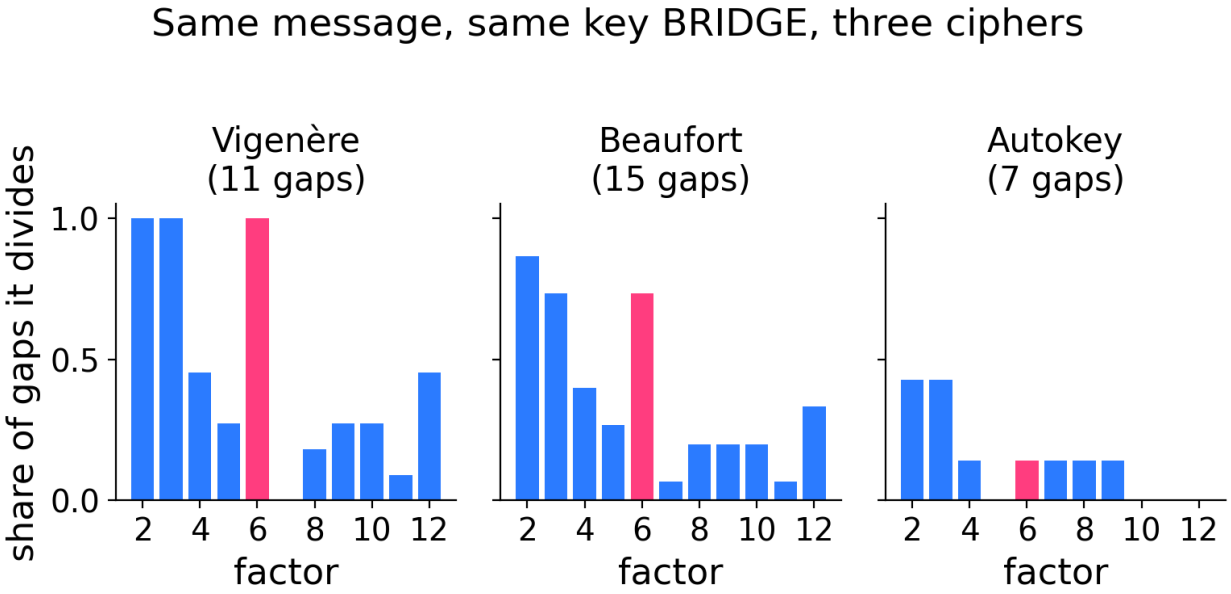


Figure 5.6: Three bar charts, one per cipher, showing what share of the gaps each factor from 2 to 12 divides. Vigenère: 2, 3 and 6 divide every gap. Beaufort: 6 still divides about three quarters of the gaps. Autokey: no factor divides more than about 4 in 10, and 6 divides only one gap in seven.

Why do any repeats appear at all? In autokey, a ciphertext repeat needs both the plaintext *and* the key under it to repeat. The key under letter 13 is the plaintext at letter 7. So `LWFP` repeats because the whole phrase "at the old mi" appears twice in the message ("meet me **at the old mi**ll" and "wait **at**

the old mill"). The gap of 128 is just the distance between two copies of a phrase. It has nothing to do with the primer's length.

So when Kasiski gives gaps with no common factor, and splitting into columns never makes the letter counts look like English, suspect autokey or a running key. Those need a different attack: guessing a probable word and working out the key it implies (see Guide 06, Cribs and patterns).

When to try it

- The index of coincidence of the whole message is low (between about 0.038 and 0.055): several alphabets are in play.
- You can see repeated groups of three or more letters by eye.
- The message is reasonably long (150 letters or more) and the key is short compared to the message. Kasiski needs each part of the key to be reused many times.
- You want an answer you can check by hand with a pencil, without a computer.

When not to

- The IoC is near 0.066. One alphabet is in play: use frequency analysis instead.
- The message is short and the key is long. A 60-letter message with a 12-letter key may show no true repeats at all.
- The ciphertext has an even length, no doubled letters inside pairs, and no J: that smells like Playfair (Guide 09), which also makes repeats, but for different reasons.

Traps and tips

- **Use trigrams or longer.** Two-letter repeats happen by chance all the time. Count three-letter and longer repeats, and trust long ones most.
- **Count, do not just take the gcd.** One accidental repeat can drag the gcd down to 1. A factor table shows which factor nearly all the gaps share.
- **Prefer the largest factor that almost every gap shares**, then check it with the index of coincidence (Guide 04). If the key length is 6, the gaps are also all multiples of 2 and 3, so those small factors always score at least as well.
- **Key letters that spell a word are your best check.** `BRIDGEBRIDGE` just means your length was double the true one.
- **A column that will not fit** usually has too few letters. Use the neighboring columns and the half-decrypted text to guess it: if the message reads `meet me at the olX mill`, the X column is off.

By hand or by computer

Finding repeats by eye is slow and easy to get wrong; a program finds every one. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

[1] Brain, pencil or computer?, the chapter before Guide 1.

Job	By hand	Learn with	Just do it
Find repeated runs of three or more letters	20 to 40 minutes	Repeats[2]	dCode[3]
Factor the gaps and tally the factors	5 to 10 minutes	Repeats[4], factors sheet[5]	dCode[6]
Solve each slice as a Caesar	about 10 minutes a slice	Vigenère[7], Vigenère sheet[8]	Guballa[9], dCode[10]

Try it yourself

1. Find $\gcd(42, 70)$ using Euclid's method. Then find $\gcd(42, 70, 98)$.
2. A Vigenère ciphertext has repeats with gaps 24, 36, 60, 84 and 17. What key length do you guess, and what do you do with the 17?
3. In a Vigenère column, the most common letters are W, J and S. If W stands for e, what is the key letter for that column? Do J and S then make sense?

> **In the app:** The Vigenère exercise walks the whole path: key length, slices, then each slice as a Caesar. Its place in the family tree: Vigenère.

Words to know

- **Kasiski examination**: finding repeated groups in a ciphertext and using the gaps between them to find the key length.
- **Trigram**: a group of three letters in a row.
- **Gap (or distance)**: how many letters separate the start of one copy of a repeat from the start of the next.
- **Factor (or divisor)**: a whole number that divides another with nothing left over.
- **Multiple**: a number you get by multiplying a number by a whole number (12 is a multiple of 6).
- **Greatest common divisor (gcd)**: the largest number that divides every number in a list.
- **Euclid's algorithm**: a fast way to find the gcd by taking remainders over and over.
- **Factor table**: a grid that shows which small numbers divide which gaps.
- **Beaufort cipher**: a repeating-key cipher with the rule cipher = key minus plain; it is its own inverse.

[2] Repeats, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/repeats. Repeated runs, their gaps, and the factor tally.

[3] dCode's Kasiski test: dcode.fr/kasiski-test.

[4] Repeats, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/repeats. Repeated runs, their gaps, and the factor tally.

[5] Repeats and factors, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-repeats-and-factors, to copy into Google Sheets or download for Excel.

[6] dCode's Kasiski test: dcode.fr/kasiski-test.

[7] Vigenère, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/vigenere. Slices for any key length, with each slice's IoC.

[8] Vigenère slices and IoC by key length, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-vigenere-slices, to copy into Google Sheets or download for Excel.

[9] Guballa's Vigenère solver: guballa.de/vigenere-solver.

[10] dCode's Vigenère solver: dcode.fr/vigenere-cipher.

- **Autokey**: a cipher whose key is a short primer followed by the message itself, so the key never repeats.
- **Primer**: the short keyword that starts an autokey key.

Going further

- Kasiski examination (Wikipedia)
- Vigenère cipher (Wikipedia)
- Cryptanalysis of the Vigenère cipher (Practical Cryptography)
- Beaufort cipher (Wikipedia)
- Autokey cipher (Wikipedia)
- Euclidean algorithm (Wikipedia)
- Simon Singh, *The Code Book* (1999), chapter 2, “Le Chiffre Indéchiffrable”, on Babbage and Kasiski.
- The National Cipher Challenge: hints, tips and cribs

Answers

1. $70 = 1 \times 42 + 28$, then $42 = 1 \times 28 + 14$, then $28 = 2 \times 14 + 0$, so gcd$(42, 70) = 14$. Then $98 = 7 \times 14 + 0$, so gcd$(14, 98) = 14$, and gcd$(42, 70, 98) = 14$.
2. 24, 36, 60 and 84 are all multiples of 12 (and so of 2, 3, 4 and 6 too); their gcd is 12. The 17 shares no factor with them, so it is almost certainly an accidental repeat: throw it out. Guess 12, then check whether the key turns out to be a 6-letter word written twice (or a 4-letter word written three times). The index of coincidence of the columns will tell you which length is smallest.
3. W is 22 and e is 4, so the shift is $22 - 4 = 18$: key letter **S**. Then J (9) stands for $9 - 18 = -9$, plus 26 is 17, which is r; and S (18) stands for $18 - 18 = 0$, which is a. Both r and a are common in English, so S is a good guess.

Guide 6

Cribs and patterns

Guess a word, pin it to the ciphertext, and let the cipher give itself away

By the end you can use one guessed word to unlock a substitution, rebuild a keyword alphabet, find a Vigenère key, and measure the width of a transposition.

The big idea

Codebreakers rarely start from nothing. A letter probably begins with a greeting. A report probably mentions the weather. A message in a story probably names someone you met in the last message. A word you expect to find in the plaintext is called a **crib** (or a **probable word**). If you can guess where the crib sits in the ciphertext, you can line the two up and read off part of the key. And part of the key is usually enough to get the rest.

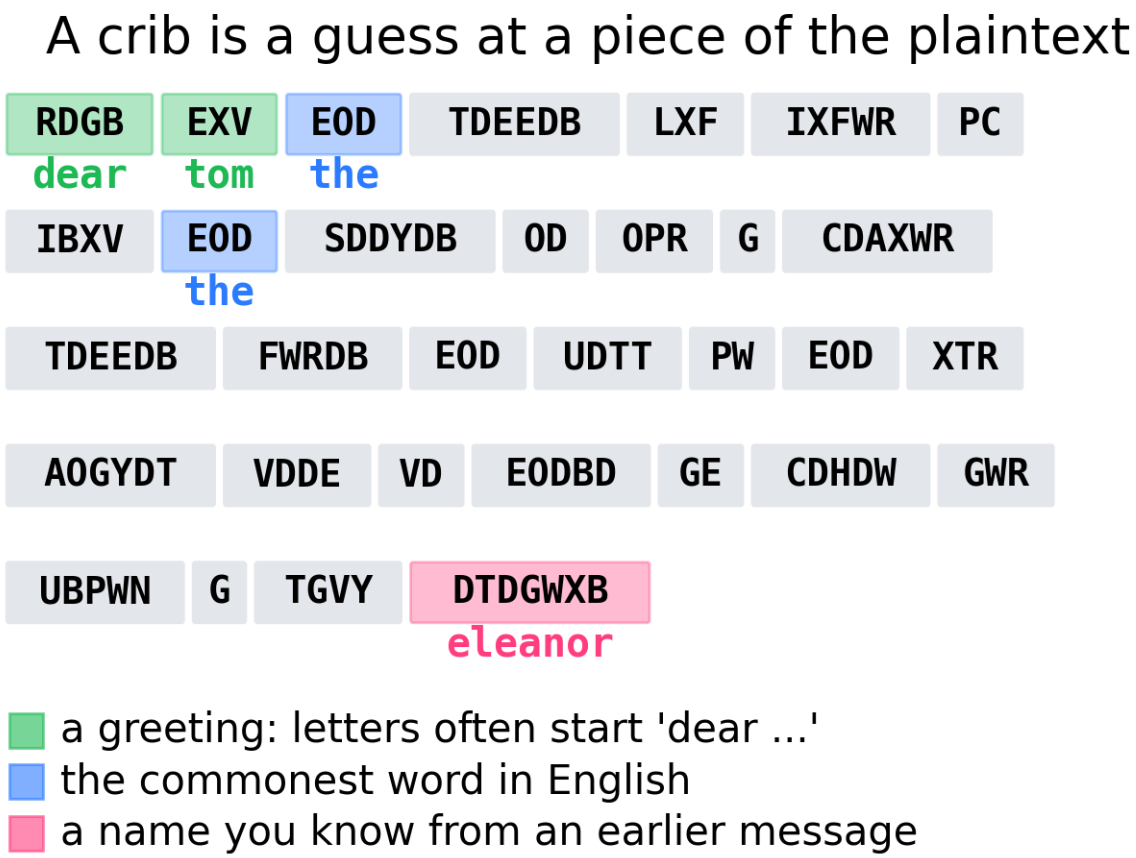


Figure 6.1: A substitution ciphertext drawn as word tiles. Three cribs are written under their tiles: “dear tom” at the start (a greeting), “the” under the commonest three-letter word, and “eleanor” under the last word (a name from an earlier message).

Which ciphers it breaks

Cribs are the most general tool in this whole series. What changes from cipher to cipher is how you “line up” the crib.

Cipher family	Do cribs help?	How
Caesar, affine	Yes, instantly	One crib letter gives a Caesar shift. Two crib letters usually give both numbers of an affine key.
Simple or keyword substitution, with word breaks	Yes, strongly	Word **patterns** find where a crib fits. Each crib fills in several letters at once.
Keyword substitution	Yes, and more	A part of the cipher alphabet lets you guess the keyword and fill in the rest.

Cipher family	Do cribs help?	How
Substitution without word breaks	Yes, more slowly	Slide the crib's pattern along the text, looking for letters that repeat in the right places.
Vigenère, Beaufort	Yes	**Crib dragging**: slide the crib along, subtract, and look for a readable piece of key.
Autokey	Yes, with a twist	Dragging gives a piece of the key, and in autokey the key *is* earlier plaintext.
Columnar transposition, rail fence	Yes	Where the crib's letters land tells you the number of columns.
Playfair, Hill	Yes	A crib is the classic way into a Hill cipher (see Guide 09, Two letters at a time).
Enigma	Yes, historically	Bletchley Park's machines were driven by cribs (see the history section).

Where good cribs come from

A crib is only useful if it is likely to be right. Good sources, from strongest to weakest:

- **Names and words from earlier messages.** In a puzzle series, characters, places and objects come back. If the last message was signed by Eleanor, this one may be too.
- **Openings and closings.** Letters start with "dear …" and end with a name. Reports start with a date or a place.
- **Words the story makes likely.** A message about a stolen painting probably contains `painting`, `museum` or `gallery`.
- **The commonest words.** `the`, `and`, `that`, `of`. Weak on their own, because they fit almost anywhere, but strong once you have a few other letters.
- **Habits of the sender.** Spelled-out numbers, a favorite sign-off, the same phrase at the start of every message.

A long crib beats a short one. A crib with repeated letters (like `letter` or `keeper`) beats one without, because repeats show up in the ciphertext as a pattern you can spot.

Why it works: word patterns

Patterns survive substitution

A substitution cipher replaces each letter with another letter, always the same one. So if a word has two letters the same, its ciphertext also has two letters the same, in the same places. The letters change; the **shape** does not.

To write that shape down, label the letters of a word in the order they first appear: the first new letter is A, the next new letter is B, and so on. A letter that comes back gets its old label again.

- `that`: t is A, h is B, a is C, and t comes back as A. Pattern **ABCA**.
- `letter`: l is A, e is B, t is C, t again is C, e again is B, r is D. Pattern **ABCCBD**.
- `people`: p A, e B, o C, p A, l D, e B. Pattern **ABCADB**.

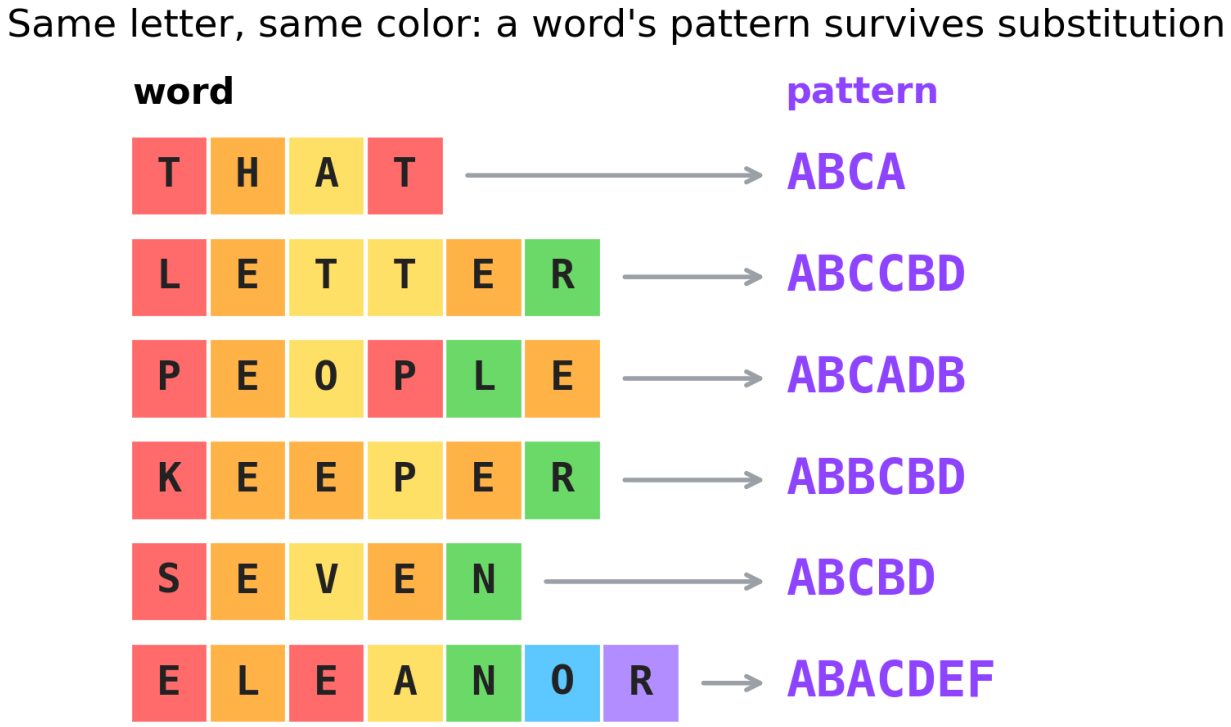


Figure 6.2: Six words drawn as colored tiles, where equal letters share a color: THAT (ABCA), LETTER (ABCCBD), PEOPLE (ABCADB), KEEPER (ABBCBD), SEVEN (ABCBD) and ELEANOR (ABACDEF).

The ciphertext word `TDEEDB` has pattern ABCCBD, the same as `letter`. The ciphertext word `EOD` has pattern ABC, which tells you very little: 198 of the 5,000 commonest English words have three different letters.

Pattern lists

Cryptographers keep **pattern lists**: every common word, filed under its pattern. You look up a ciphertext word's pattern and get a short list of candidates. Some patterns are rare, and those are gold. In a list of the 5,000 commonest English words (counted from a large collection of books):

Pattern	How many common words	Some of them
ABCD	528	with, have, from, what, your, time
ABCA	15	that, says, dead, else, high, edge
ABCCBD	13	better, letter, common, follow, bottom, cotton
ABCBD	14	never, state, visit, seven, fever, motor
ABBCBD	5	seemed, assist, deeper, keeper, peered
ABACDEF	7	against, moments, suspect, alarmed, elegant

So a ciphertext word with pattern ABBCBD has only five common candidates. Once you know even one or two of its letters, usually only one candidate survives.

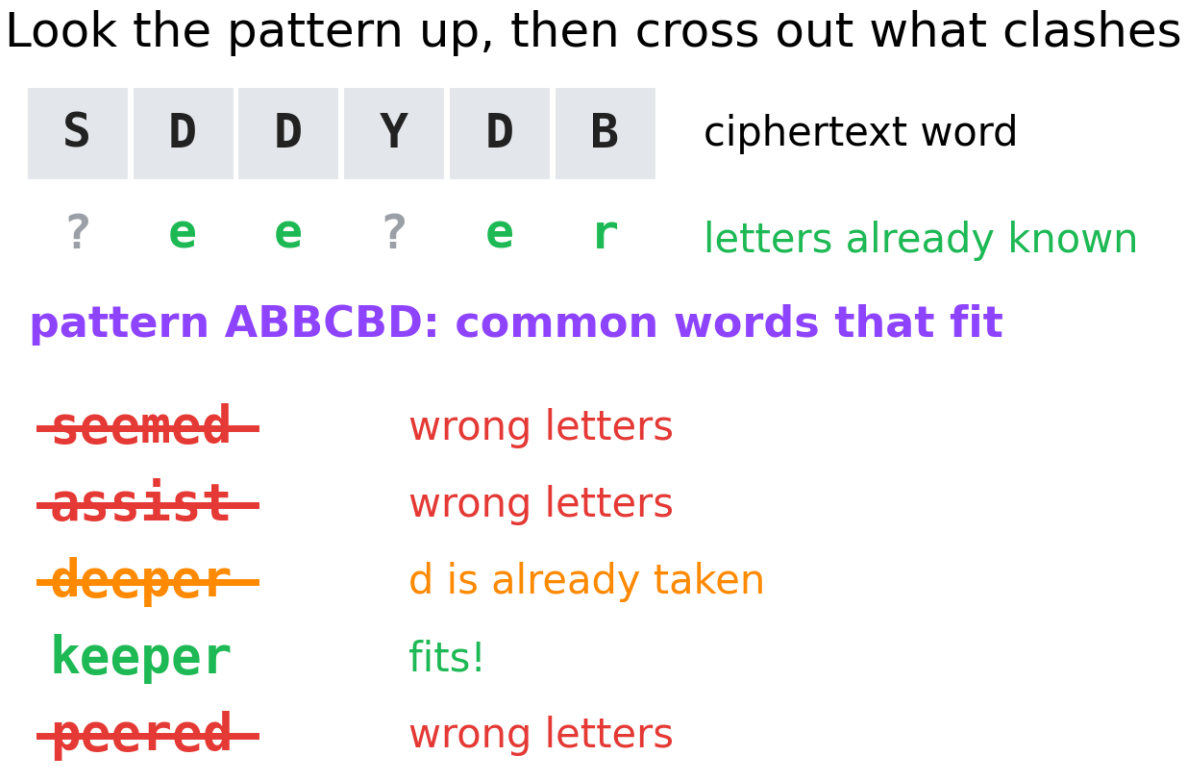


Figure 6.3: The ciphertext word SDDYDB, with e, e, e and r already known. Its pattern ABBCBD has five common words. Seemed, assist and peered have the wrong letters, deeper needs a d that is already used, and keeper fits.

> **In the app:** Pattern words trains you to recognize a word from the shape of its repeated letters.

A worked example: a keyword substitution

Here is a message, with its word breaks kept. In the previous puzzle, a girl called Eleanor wrote to her friend Tom.

```
RDGB EXV, EOD TDEEDB LXF IXFWR PC IBXV EOD SDDYDB. OD OPR G
CDAXWR TDEEDB FWRDB EOD UDTT PW EOD XTR AOGYDT. VDDE VD EODBD
GE CDHDW GWR UBPWN G TGVY. DTDGWXB
```

Step 1: the name

The last word, `DTDGWXB`, stands alone after the final full stop, just where a signature goes. Its pattern is ABACDEF, and so is the pattern of `eleanor`. Try it. That gives six letters at once: cipher D = e, T

= I, G = a, W = n, X = o, B = r.

Step 2: the greeting

The first word `RDGB` now reads `?ear`. A letter that starts "?ear Tom" can only be `dear`, so R = d. The second word `EXV` reads `?o?`, and `tom` fits: E = t, V = m.

Step 3: the commonest word

`EOD` appears four times. With E = t and D = e it reads `t?e`, which must be `the`. So O = h.

Step 4: patterns check the guesses

`TDEEDB` now reads `letter`. It has pattern ABCCBD, as `letter` should, so the guesses agree with each other. `SDDYDB` reads `?ee?er`. Its pattern is ABBCBD, and of the five common words with that pattern only `deeper` and `keeper` match `?ee?er`. But plain d is already taken (cipher R), so it is `keeper`: S = k, Y = p.

Step 5: the rest falls out

With 12 letters known, every word is nearly readable: `LXF` is `you`, `IBXV` is `from`, `CDAXWR` is `second`, `CDHDW` is `seven`, and so on. The message is:

```
dear tom, the letter you found is from the keeper. he hid a second letter under
the bell in the old chapel. meet me there at seven and bring a lamp. eleanor
```

You have now found 21 of the 26 letters of the cipher alphabet. The message never used j, q, w, x or z, so those five are still missing. For this message that does not matter, but if the next message uses the same key, it will. That is where the keyword comes in.

Rebuilding a keyword alphabet

How a keyword alphabet is made

A random cipher alphabet is hard to remember, so people build one from a **keyword**:

1. Write the keyword, crossing out any letter you have already written. `GUARDIAN` becomes `GUARDIN`.
2. Carry on with the rest of the alphabet in order, starting from the letter after the keyword's last letter, skipping letters already used, and wrapping round from Z to A.

After `GUARDIN` the alphabet carries on from O: O, P, Q, (R is used), S, T, (U is used), V, W, X, Y, Z, then round to (A is used) B, C, (D is used) E, F, (G is used) H, (I is used) J, K, L, M.

```
plain:   a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher:  G U A R D I N O P Q S T V W X Y Z B C E F H J K L M
```

Some puzzle makers start the second part from A instead of from the letter after the keyword. That gives `GUARDINBCEFHJ...`. Either way, the idea below works.

Seeing part of it is enough

Write the 21 letters you found under the plain alphabet, with a ? for the five you are missing:

```
plain:   a b c d e f g h i j k l m n o p q r s t u v w x y z
found:   G U A R D I N O P ? S T V W X Y ? B C E F H ? ? L ?
```

Two things jump out:

- **The start spells a word.** G U A R D I N looks like `guardian` with its repeated A crossed out.
- **The rest is in alphabetical order.** From O onward the found letters climb through the alphabet: O P ? S T V W X Y ? B C E F H ? ? L ?. The only letters that ever get skipped are letters of the keyword.

So the gaps fill themselves. Between P and S the only unused letter is Q, so j = Q. After Y comes Z, so q = Z. Between H and L the unused letters are J and K, so w = J and x = K. After L the only letter left is M, so z = M.

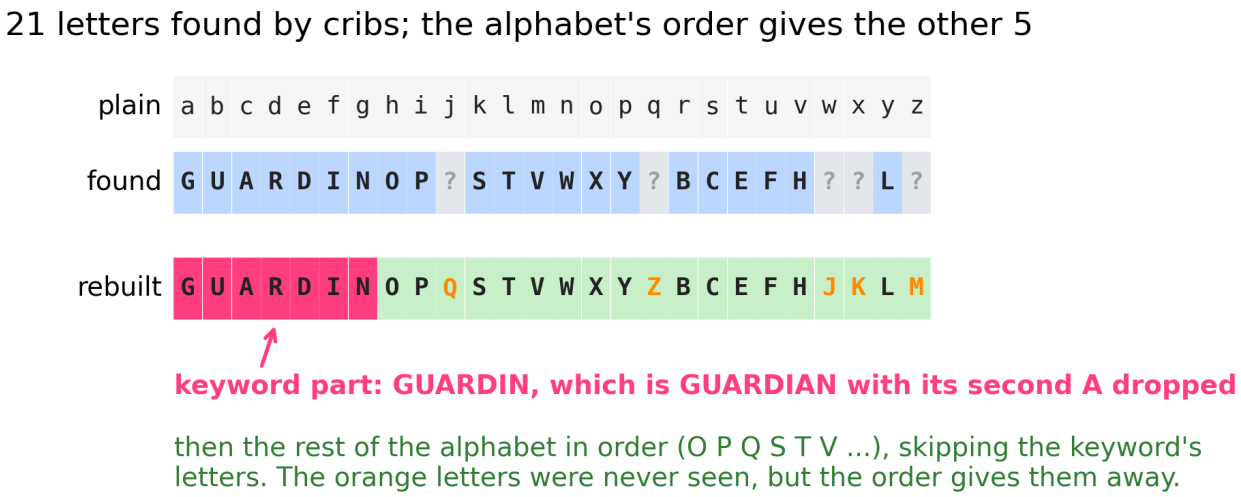


Figure 6.4: The plain alphabet, the 21 cipher letters found from cribs (with five question marks), and the rebuilt alphabet: the keyword part GUARDIN in pink, then the rest of the alphabet in order, with the five never-seen letters Q, Z, J, K and M filled in orange.

This works even if you only see the tail. The letters missing from the alphabetical run (here R, U, D, G, I, N and A) must be the keyword's letters. Unscramble them and you often have the keyword.

> **In the app:** Substitution shows your key as you fill it in, so a keyword alphabet's giveaway run, such as STUVWXYZ, stands out. More on the cipher: keyword substitution.

The grown-up version

Letters as numbers

To work with the Vigenère cipher you need letters as numbers: A = 0, B = 1, C = 2, and so on up to Z = 25. Encryption **adds** the key letter to the plain letter. If the sum goes past 25, you take away 26 so that it wraps back round to the start of the alphabet.

Mathematicians write "wrap round after 26" as $\pmod{26}$, read "modulo 26" or "mod 26". The Vigenère rule is

$$c \equiv p + k \pmod{26}$$

where p is the plain letter, k the key letter and c the cipher letter. The sign $\equiv$ means "is the same, once you have wrapped round". For example, plain s (18) with key letter L (11) gives $18 + 11 = 29$, and $29 - 26 = 3$, which is D.

Turning the rule round

If you know the cipher letter and you have *guessed* the plain letter, you can work out the key letter by subtracting:

$$k \equiv c - p \pmod{26}$$

If the answer is negative, add 26. That one line is the whole of crib dragging.

For the **Beaufort** cipher the rule is $c \equiv k - p$, so the key is $k \equiv c + p \pmod{26}$. Same method, different sum.

A worked example: crib dragging a Vigenère

Here is a 115-letter Vigenère ciphertext. The story tells us it is about a professor, so `professor` is a good crib.

```
FTYKB RYHPR FBKED UCEQE QVHUJ MFUSH RBJVV RJADV RBTRE JSZFP
TUVVI DPPWA XIOGH UJCWQ QXNNS KGFYU OATVS DTVPR KOCGH UJGFC
FQXRP VIESA EFOJS
```

(The groups of five are only for reading. They are not word breaks.)

Step 1: slide and subtract

Put the crib under the first nine ciphertext letters and subtract, letter by letter. Then move the crib one place to the right and do it again. At each position you get a nine-letter "key fragment". At a wrong position the fragment is gibberish. At the right position it is a piece of the real key, so it should look like part of a word, or a word repeating.

Crib dragging: slide the probable word along the ciphertext

position	ciphertext under the crib	key fragment (cipher minus crib)	
1	FTYKBRYHP	QCKFXZGTY	
2	TYKBRYHPR	EHWWNGPBA	
3	YKBRYHPRF	JTNMUPXDO	
4	KBRYHPRFB	VKDTDXZRK	
5	BRYHPRFBK	MAKCLZNNT	
6	RYHPRFBKE	CHTKNNJWN	
7	YHPRFBKED	JQBMBJSQM	
8	HPRFBKEDU	SYDAXSMPD	
9	PRFBKEDUC	AARWGMLGL	
10	RFBKEDUCE	**CONFALCON**	**readable!**
11	FBKEDUCEQ	QKWZZCKQZ	
12	BKEDUCEQE	MTQYQKMCN	
13	KEDUCEQEQ	VNPPYMYQZ	
14	EDUCEQEQV	PMGXAYMCE	

crib = professor, slid one letter at a time

Figure 6.5: A table sliding the crib professor along the ciphertext, positions 1 to 14. The key fragments are gibberish except at position 10, highlighted, where the fragment reads CONFALCON.

At position 10 the ciphertext is `RFBKEDUCE`. Subtract `professor`:

	R	F	B	K	E	D	U	C	E
cipher c	17	5	1	10	4	3	20	2	4
crib p	15	17	14	5	4	18	18	14	17
$c - p$	2	-12	-13	5	0	-15	2	-12	-13
add 26 if negative	2	14	13	5	0	11	2	14	13
key letter	C	O	N	F	A	L	C	O	N

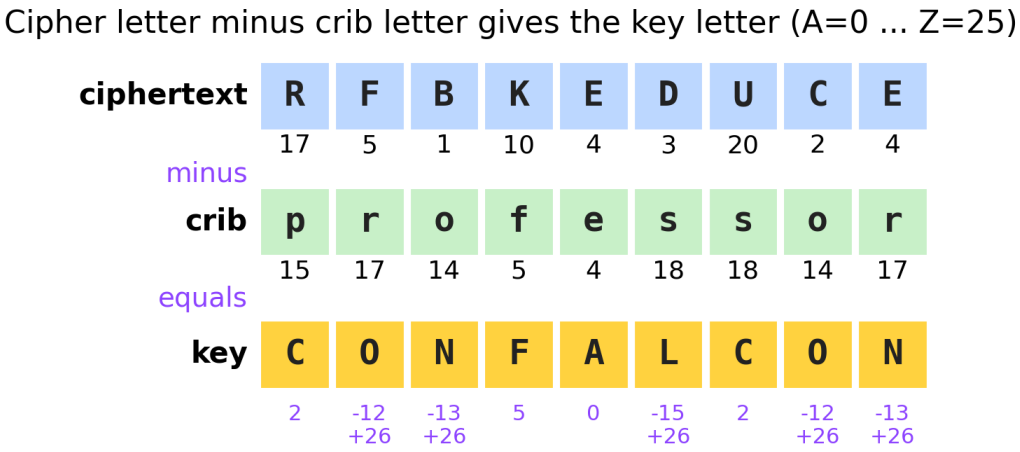


Figure 6.6: The subtraction at position 10, drawn as three rows of tiles: the ciphertext RFBKEDUCE, the crib professor, and the key fragment CONFALCON, with each letter's number and the add-26 steps.

Step 2: read the key

`CONFALCON` is the word `falcon`, starting partway through and going round again. The fragment repeats after six letters (C O N F A L, then C O N), so the key has **6** letters. Position 10 used key letter C. Counting back, position 9 used L, 8 used A, 7 used F, and position 1 used the same key letter as position 7. So the key, read from the start, is `FALCON`.

Step 3: check

Decrypt the whole message with `FALCON`: `at nine the professor left the museum by the east door. the map is hidden inside the clock in his study. find it before the guards return at dawn`. Done, and you never needed to find the key length first.

Why a short crib is risky

Try the same thing with the crib `the`. There are 113 positions, and only one gives a common three-letter word as its fragment: position 4 gives `RUN`. But that is a false alarm! The first real `the` is at position 7, and its fragment is `FAL`, which is not a word at all, just the start of `falcon`. Short cribs give too many accidents and too few clues. Use the longest crib you trust.

Cribs for transposition

A transposition keeps every letter and only moves them. So you cannot subtract anything. Instead, a crib tells you **where letters must end up next to each other**.

In a columnar transposition with w columns, you write the message in rows of w letters and read the ciphertext down the columns. Two plaintext letters that are w places apart sit one above the other in the same column. When that column is read out, they come out **side by side**. So if the width is w,

then for every letter of the crib, the letter w places later must appear right after it somewhere in the ciphertext.

Here is a 90-letter columnar transposition. The story says a submarine is involved.

```
SRIRN HTRAD RHNMW YLNTB NLAAN POTRN SLAIH CUILF EEHSF AIEAP
ATDEH MESCR OIOEK GILNT EREAW UETRE NRBTG ADBOA
```

(Again, the groups of five are only for reading. A pair can be split across two groups, so look across the gaps.)

Write `submarine` in rows of 5: `s u b m a` above `r i n e`. The stacked pairs are `SR`, `UI`, `BN` and `ME`. All four appear in the ciphertext (it even starts with `SR`). In rows of 4 the stacked pairs are `SA`, `UR`, `BI`, `MN`, `AE`, and none of them appears. Notice that it does not matter where in the message the crib starts: the pairs are the same.

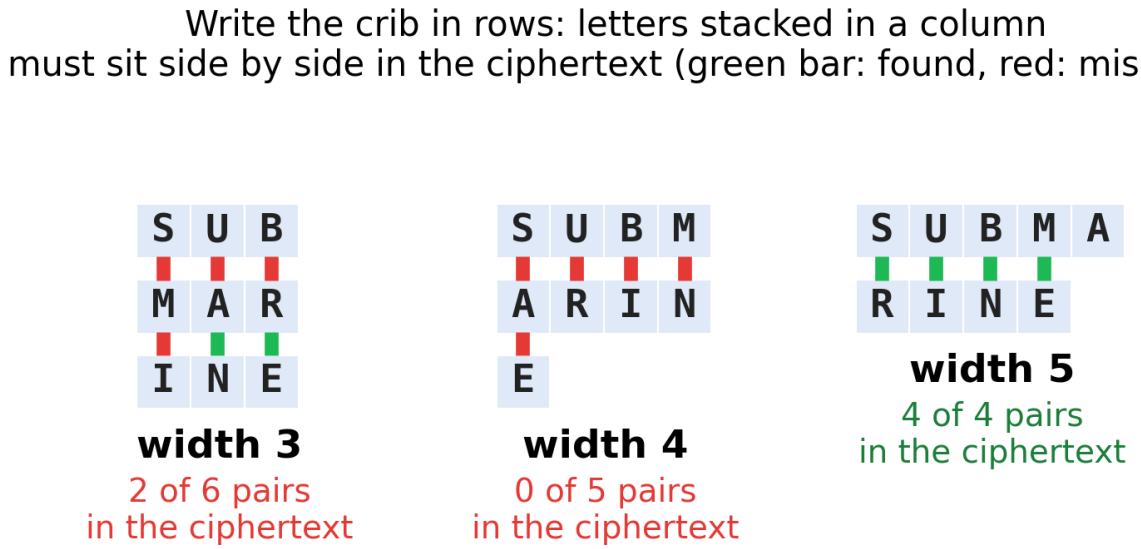


Figure 6.7: The crib SUBMARINE written in rows of 3, 4 and 5. Bars join the letters stacked in each column: green if that pair appears in the ciphertext, red if not. Width 3 has 2 of 6 pairs, width 4 has 0 of 5, width 5 has 4 of 4.

Do this for every width you think possible and compare the share of pairs found:

Width	2	3	4	**5**	6	7
Pairs found	3 of 7	2 of 6	0 of 5	**4 of 4**	0 of 3	1 of 2

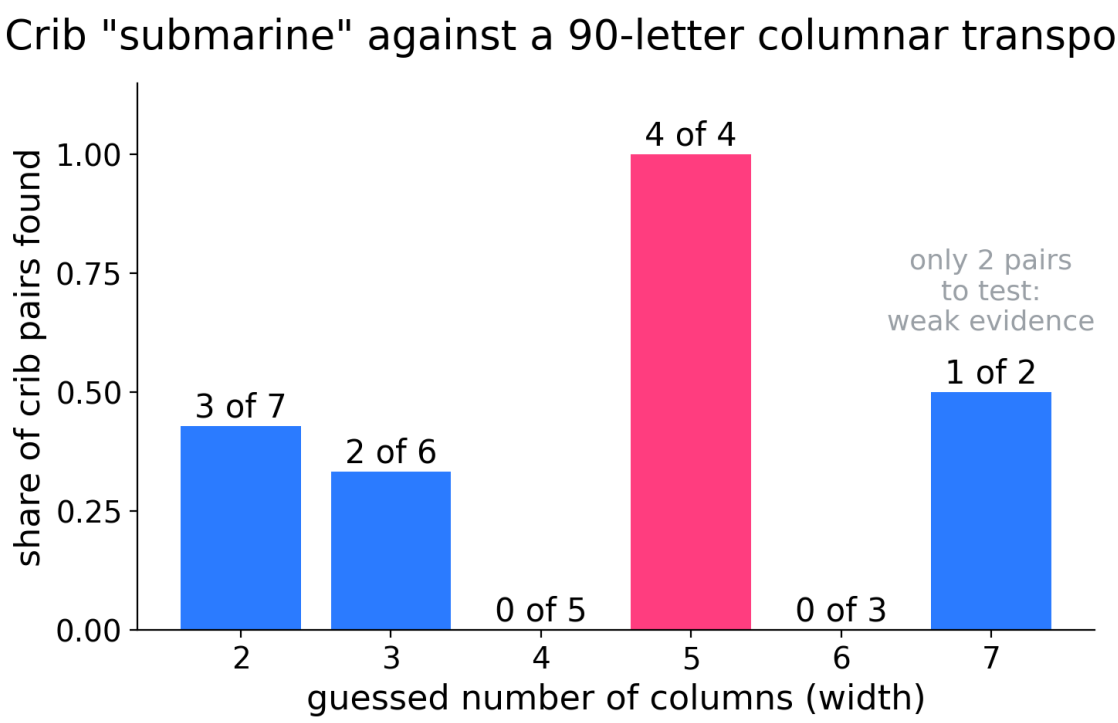


Figure 6.8: Bar chart of the share of crib pairs found for widths 2 to 7. Width 5 is the only full bar, 4 of 4. Width 7 reaches 1 of 2, which is weak evidence because it only has 2 pairs to test.

Only width 5 finds every pair. Some wrong widths find a few pairs by luck, because common pairs like AN and RE turn up anywhere. Then the crib helps again: the pairs tell you which ciphertext columns hold which plaintext letters, and that helps you put the columns in order. Guide 07, Transposition detective work, finishes the job.

History: cribs at Bletchley Park

During the Second World War the German armed forces encrypted their radio messages with **Enigma**, an electric machine with rotating wheels. Polish mathematicians Marian Rejewski, Jerzy Różycki and Henryk Zygalski first broke it in the 1930s and, in July 1939, shared their methods with Britain and France. At Bletchley Park, the British codebreaking center, Alan Turing designed an electro-mechanical machine called the **bombe** to search for the day's Enigma settings, and Gordon Welchman improved it. Bombes were working from 1940.

A bombe could not start from nothing. It needed a crib: a guess at a stretch of plaintext and where it sat in the message. Military messages were full of routine. Weather reports contained words like WETTERVORHERSAGE ("weather forecast"). Some stations sent "nothing special to report" day after day. When the routine was not enough, the RAF sometimes laid mines in a chosen spot, so that German warnings would name that spot. This was called "gardening".

One quirk of Enigma made placing a crib much easier: **it never turned a letter into itself**. So if you slide a crib along a ciphertext and any crib letter lands on the same ciphertext letter, that position is impossible.

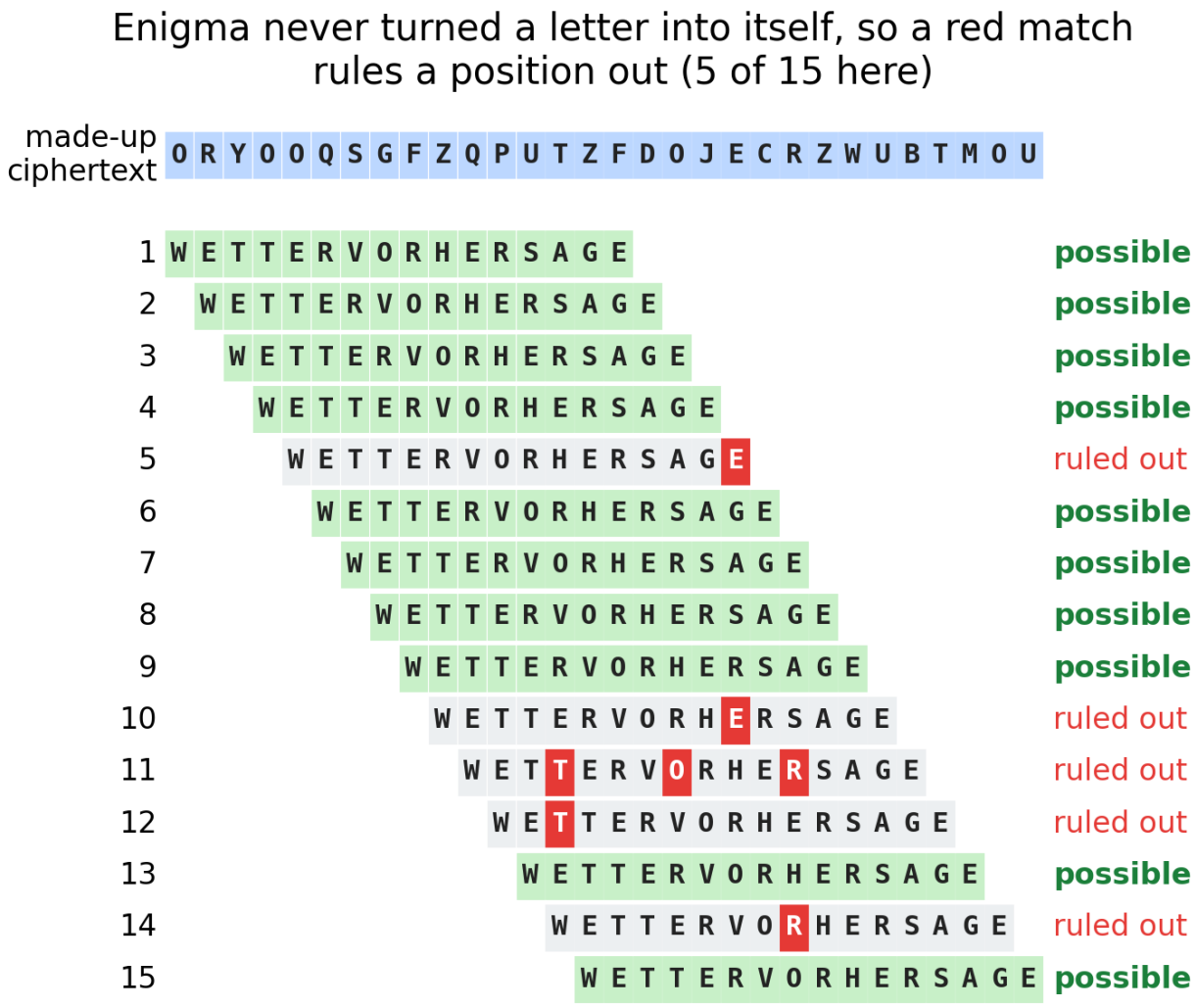


Figure 6.9: A made-up 30-letter ciphertext with the crib WETTERVORHERSAGE slid under it in all 15 positions. Positions where a crib letter sits on the same ciphertext letter are marked red and ruled out (5 of 15); the other 10 stay possible.

In this made-up example, 5 of the 15 positions are ruled out at once. On real messages, codebreakers did this by hand before giving a crib to the bombes.

When to try it

- **You know something about the message.** A story, a sender, a subject, an earlier message. Make a list of likely words before you start counting letters.
- **The ciphertext keeps its word breaks.** Look up the patterns of the longest words and the words with repeated letters first.
- **A substitution is half solved.** Write out the cipher alphabet in plain-alphabet order and look for

a keyword and an alphabetical run.
- **You know it is a Vigenère but the key length is unclear.** A good crib finds the key without Kasiski or the index of coincidence (Guides 04 and 05).
- **You know it is a transposition** (the letter counts look like English, but nothing reads). A crib finds the width.

When not to

- **No context at all.** With no story, only weak cribs like `the` are left. Start with frequency analysis (Guide 02) or a fitness score (Guide 08) instead.
- **Very short messages.** With 30 letters, almost any guess can be made to fit somewhere.
- When a crib leads to letters that clash. **Drop it at once.** A wrong crib, defended stubbornly, wastes more time than anything else.

Traps and tips

- **Test, do not trust.** A crib is a guess. Every guess must agree with every other: if two cribs give two different cipher letters for the same plain letter, one of them is wrong.
- **Short cribs give false alarms.** Dragging `the` along 115 letters produced `RUN` by accident, while the real position gave `FAL`. Prefer cribs of six letters or more.
- **Expect the fragment to be part of a word, not a whole word.** A key fragment usually starts partway through the keyword (`CONFALCON`, not `FALCON`).
- **No word breaks?** Slide the crib's *pattern* along the text. For `letter`, look for any six letters where the 3rd and 4th match and the 2nd and 5th match. Rare patterns are found quickly this way.
- **Names and odd words.** A name like `eleanor` is not in a word list, so a pattern list will not suggest it. That is exactly why names from earlier messages make such good cribs.
- **Two keyword conventions.** If the alphabetical run seems to break oddly, the second part of the alphabet may start from A instead of after the keyword. Try both.
- **Which way round?** Some puzzles write the keyword alphabet as the *plain* row instead of the cipher row. If no keyword appears in the cipher row, look in the other direction.
- **Padding in transpositions.** Extra letters like `X` at the end can create stray pairs. Do not let a single pair decide the width.

By hand or by computer

A good crib comes from thinking about the message; checking it at every position is mechanical. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
Look up a word's letter pattern	5 to 15 minutes a word, with a word list	Pattern words[2]	quipqiup[3]

[1] Brain, pencil or computer?, the chapter before Guide 1.
[2] Pattern words, an exercise in the cryptology app: apps.thorwhalen.com/cryptology/#/ex/patterns/1.
[3] quipqiup, a substitution solver: quipqiup.com.

Job	By hand	Learn with	Just do it
Drag a crib along a Vigenère text	about a minute a position, for hundreds of positions	Vigenère[4]	dCode[5]
Rebuild a keyword alphabet from the key	10 minutes	Substitution[6], substitution sheet[7]	dCode[8]

Try it yourself

1. Write the pattern of `bookkeeper`. Then find the odd one out: `moon`, `seen`, `took`, `noon`, `look`.
2. This Vigenère ciphertext hides the crib `secret` at letters 4 to 9: `HDPGANFAERKZFEDPASWJOHDPQHZQG`. Find the key fragment, then the key.
3. This columnar transposition contains the word `garden`: `MTIHAEENHLOENNEMNERNHDEDUAOXEETGDBITOHSTOX` Is the width 3 or 4?
4. Here is part of a keyword cipher alphabet. Fill in the missing letters and name the keyword.

```
plain:   a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher:  P U M ? I N O ? R S T V W X Y ? A B C D E F ? ? J L
```

> **In the app:** Substitution with word breaks is a keyword cipher to crack by hand: start from a crib and the common words.

Words to know

- **Crib** (or **probable word**): a word or phrase you expect to find in the plaintext.
- **Word pattern**: the shape of a word's repeated letters, written with A, B, C… in order of first appearance (`that` is ABCA).
- **Pattern list**: a list of common words grouped by their patterns.
- **Keyword alphabet**: a cipher alphabet made from a keyword (repeats removed) followed by the rest of the alphabet in order.
- **Crib dragging**: sliding a crib along a ciphertext, subtracting at each position, and looking for a readable piece of key.
- **Key fragment**: the piece of key you get by subtracting a crib from the ciphertext.
- **Modulo 26** ($\bmod 26$): wrapping round the alphabet, so that 26 counts as 0 again.
- **Bombe**: the machine designed by Alan Turing at Bletchley Park that used cribs to find Enigma settings.

[4] Vigenère, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/vigenere. Slices for any key length, with each slice's IoC.

[5] dCode's Vigenère solver: dcode.fr/vigenere-cipher.

[6] Substitution, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/substitution. You choose the letters; it fills in the text.

[7] Substitution key and partial decrypt, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-substitution, to copy into Google Sheets or download for Excel.

[8] dCode's substitution solver: dcode.fr/monoalphabetic-substitution.

Going further

- Video: Flaw in the Enigma Code, Numberphile, 11 minutes: why a crib can only sit where no letter matches.
- Known-plaintext attack (Wikipedia)
- Cryptanalysis of the Enigma: cribs (Wikipedia)
- Bombe (Wikipedia)
- Keyword cipher (Wikipedia)
- Cryptanalysis of the Vigenère cipher (Practical Cryptography)
- Simon Singh, *The Code Book* (1999), chapter 4, on Enigma, cribs and Bletchley Park.
- The National Cipher Challenge: hints, tips and cribs

Answers

1. `bookkeeper`: b A, o B, o B, k C, k C, e D, e D, p E, e D, r F, so **ABBCCDDEDF**. The odd one out is `noon` (ABBA). The other four are all ABBC.
2. Letters 4 to 9 of the ciphertext are `GANFAE`. Subtract `secret`: G (6) minus s (18) is -12, plus 26 is 14, which is O. A (0) minus e (4) is $-4 + 26 = 22$, W. N (13) minus c (2) is 11, L. Carrying on gives the fragment `OWLOWL`. It repeats after three letters, and position 4 used key letter O, so position 1 did too: the key is **OWL**. The message reads `the secret door is behind the clock`.
3. In rows of 3, `garden` stacks as `g a r` above `d e n`, giving the pairs `GD`, `AE` and `RN`. All three are in the ciphertext. In rows of 4 the pairs are `GE` and `AN`, and neither is there. The width is **3**. (The sender added two Xs to fill the last row, which is why the ciphertext has 42 letters.)
4. After N the run goes O, then Q: so P is in the keyword, and h = **Q**. After Y comes Z, so p = **Z**. After F come G and H, so w = **G** and x = **H**. The run ends J, L, skipping K, so K is a keyword letter, and d = **K**. The keyword part is `PUMKIN`, which is **PUMPKIN** with its repeated P crossed out.

Guide 7

Transposition detective work

All the right letters, in all the wrong places

By the end you can tell, from counting letters, that a message has been scrambled rather than disguised, and you can unscramble the common kinds by hand: reversal, rail fence, columnar and block transposition.

The big idea

There are two basic ways to hide a message. A **substitution** cipher swaps each letter for a different one, but leaves it in its place. A **transposition** cipher keeps every letter exactly as it is, but moves it somewhere else. It is an anagram of the whole message, made by following a rule. So a transposition leaves a giveaway fingerprint: the letter counts are exactly the counts of ordinary English. Lots of E and T, a few Z and Q. The ciphertext “looks English” letter by letter, but no words appear anywhere.

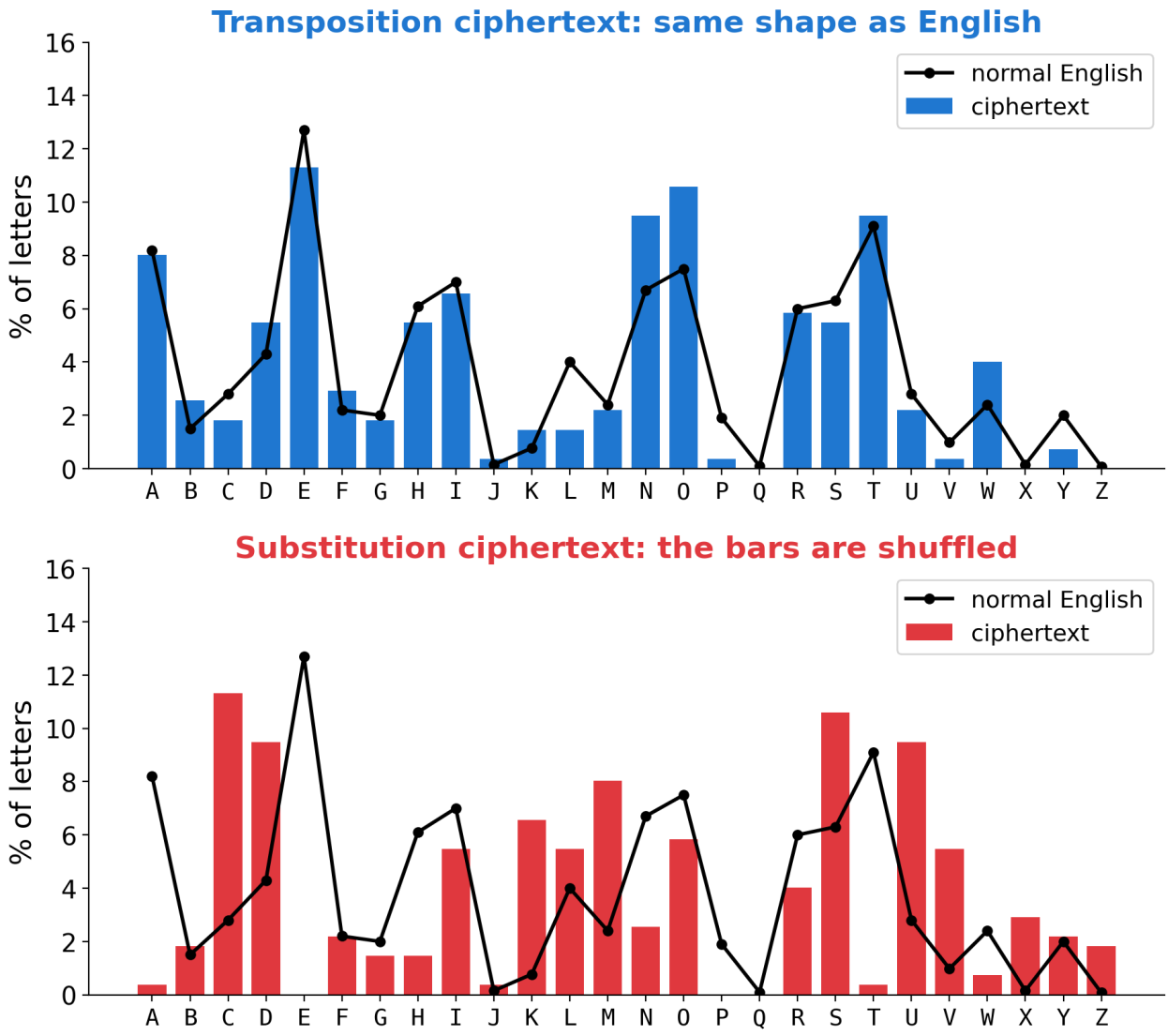


Figure 7.1: Letter counts of a 274-letter ciphertext made by transposition follow the normal English line closely (E and T tall, J, Q, X and Z tiny), while the same message made by substitution has its bars shuffled: C, S and U tall, E and T almost empty.

Once you know it is a transposition, your job is detective work: work out the **rule** that moved the letters, and run it backward.

Which ciphers it breaks

Cipher	Do these methods help?	How
Reversal	Yes	Read it backward.
Rail fence	Yes	Try 2, 3, 4… rails; work out how many letters sit on each rail.

Cipher	Do these methods help?	How
Columnar transposition	Yes	Try widths that divide the length; cut into columns; slide them until rows read.
Block (permutation) transposition	Yes	Same grid, but cut into rows instead of columns.
Double transposition	Partly	Much harder: a computer search is usually needed.
Substitution, Caesar, affine	No	The letter counts give them away as *not* transpositions (Guide 02, Frequency analysis).
Vigenère, Playfair, Hill	No	Their letter counts are flatter than English (Guide 04 and Guide 09).
A substitution and a transposition together	Partly	Counts look like a substitution; solve that part with a computer search (Guide 08).

Why it works

Spotting a transposition

Since a transposition only moves letters, anything you can measure by **counting** single letters stays exactly the same as in English. Here are the four tells.

1. **The letter counts match English.** E is the most common, then T, A, O, I, N, S, H, R. J, Q, X and Z are rare or missing.
2. **About 40 percent of the letters are vowels** (A, E, I, O, U), as in English.
3. **The index of coincidence** (the chance that two letters picked at random from the text are the same) is about 0.067, the value for English. (Guide 04 explains it. A substitution also keeps this number, but a Vigenère cipher pushes it down toward 0.038.)
4. **But nothing reads.** No THE, no AND, no words at all. Or sometimes it reads backward.

A neat extra clue: in English, Q is almost always followed by U. A transposition moves them apart, but it cannot remove the U. If the ciphertext has three Qs, it will have at least three Us.

The family of scrambles

Reversal. The simplest of all: write the message backward. `THGINOTSELDNACEROMDNES` is “send more candles tonight” backward. The tell: you see THE backward, as EHT, and common word endings such as ING appear as GNI.

Rail fence. Write the message in a zigzag across a few horizontal lines (the **rails**), then read each rail from left to right.

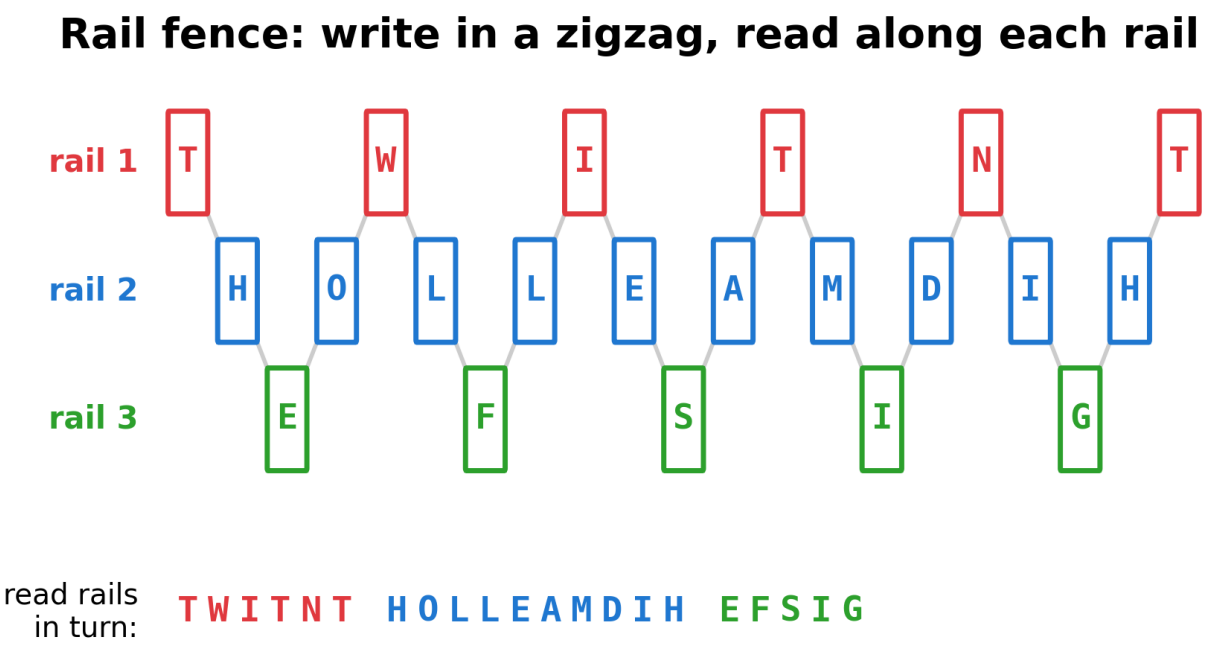


Figure 7.2: The plaintext THEOWLFLIESATMIDNIGHT written in a zigzag over three rails, colored by rail, and read off rail by rail to give TWITNT, HOLLEAMDIH and EFSIG.

The key is just the number of rails, so there are only a handful to try. To undo it, you need to know how many letters landed on each rail. With 3 rails the zigzag repeats every 4 letters (down, down, up, up). Of our 21 letters, positions 1, 5, 9, 13, 17 and 21 are on rail 1 (6 letters), positions 3, 7, 11, 15 and 19 are on rail 3 (5 letters), and the other 10 are on rail 2. So split the ciphertext `TWITNTHOLLEAMDIHEFSIG` into 6, 10 and 5 letters: `TWITNT`, `HOLLEAMDIH`, `EFSIG`. Write them back on their rails and read the zigzag.

Columnar transposition. Pick a keyword, such as TIGER. Write the message in rows under it, as many letters per row as the keyword has. Number the keyword's letters in alphabetical order (E is 1, G is 2, I is 3, R is 4, T is 5). Then read down each column, in that number order.

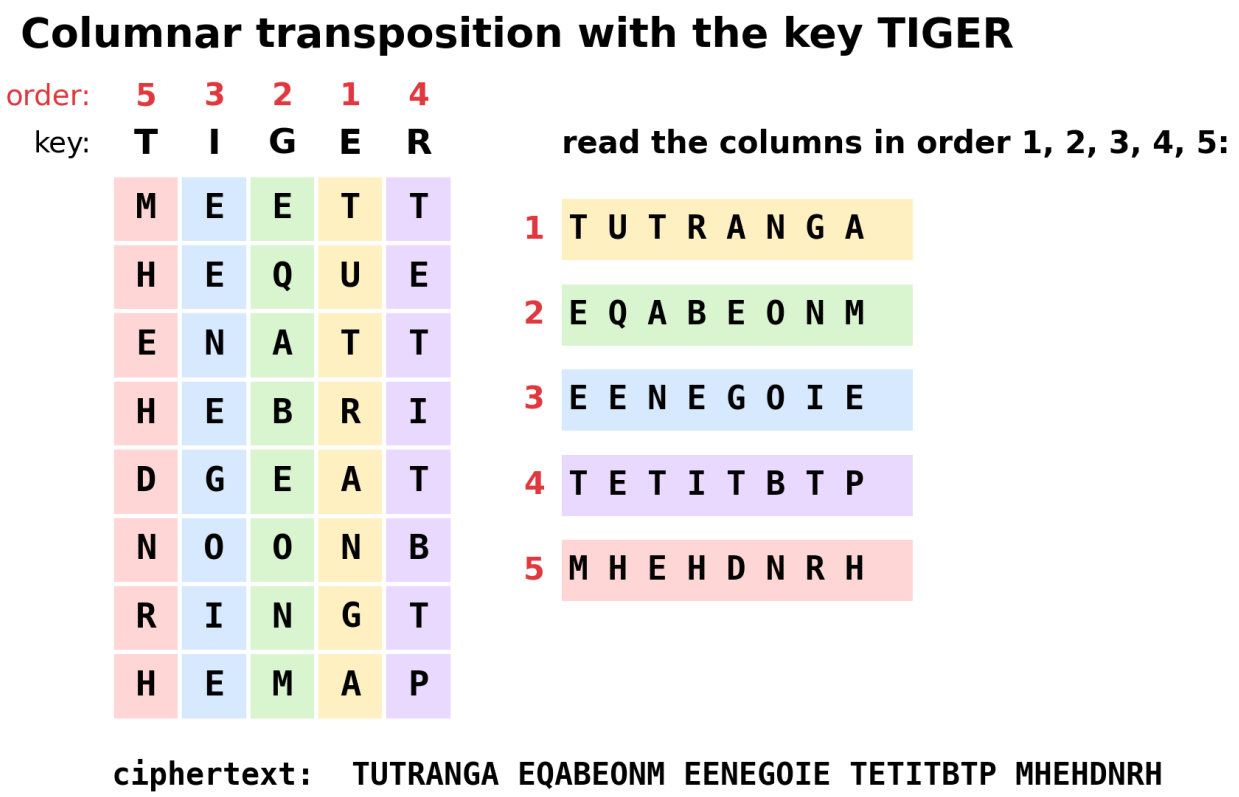


Figure 7.3: The message MEETTHEQUEEN… written in rows of five under the keyword TIGER, with column order 5 3 2 1 4; the columns are read in the order 1 to 5 to make the ciphertext TUTRANGA EQABEONM EENEGOIE TETITBTP MHEHDNRH.

Block transposition (also called a **permutation cipher**). Cut the message into blocks, say of 4 letters, and shuffle the letters inside every block the same way. With the shuffle "2nd, 4th, 1st, 3rd", `LOOK UNDE RTHE BIGS TONE` becomes `OKLO NEUD TERH ISBG OETN`. Notice that this is really the same grid as columnar: write the message in rows of 4, and the shuffle moves whole columns. The only difference is that you read the result row by row instead of column by column.

> **In the app:** The family tree shows the transposition family: rail fence, columnar, block and reversal.

The grown-up version

Width from factors

For a columnar or block transposition, the number of columns is called the **width** (or **period**). If the sender filled the grid completely (often adding filler letters such as X at the end), then the width times the number of rows equals the length of the message. So the width must be a **factor** (or **divisor**) of the length: a number that divides it with nothing left over.

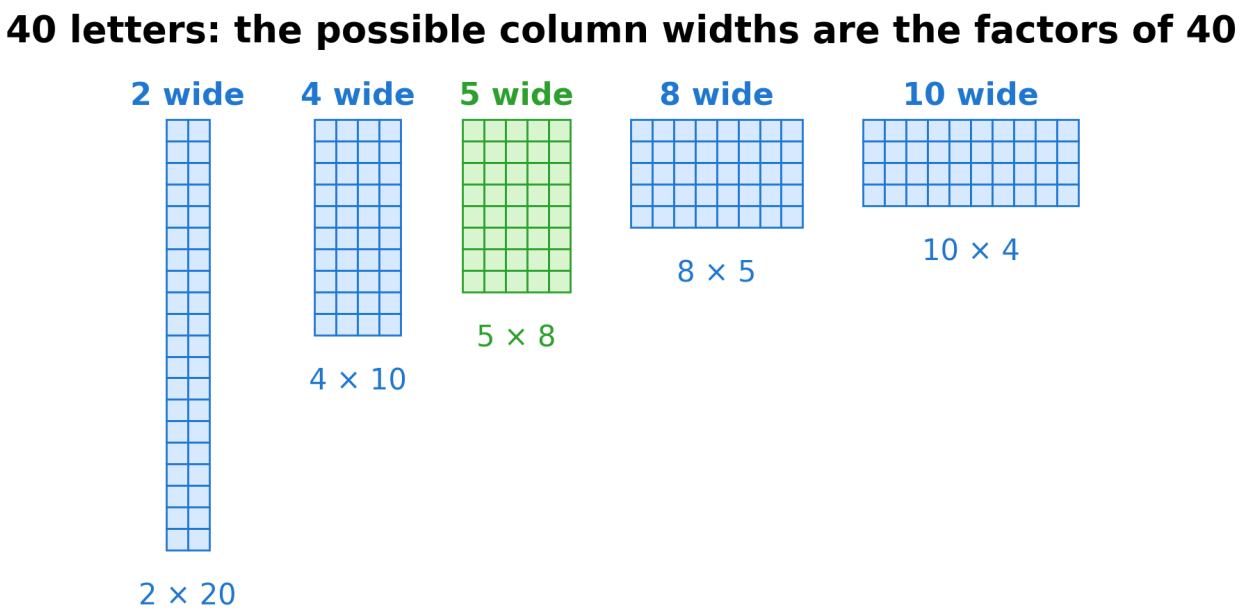


Figure 7.4: A 40-letter message can fill grids that are 2, 4, 5, 8 or 10 letters wide (2 by 20, 4 by 10, 5 by 8, 8 by 5, 10 by 4), because those are the factors of 40; the 5-wide grid is highlighted.

Write the length n as a product of primes: $40 = 2 \times 2 \times 2 \times 5$. Every factor is built from some of those primes: 2, 4, 5, 8, 10, 20. Widths of 1 or 40 would not scramble anything, and 20 would leave just two rows, so in practice you try 4, 5, 8 and 10 first. A prime length (such as 37 or 41) is a strong hint that the grid was **not** filled completely, and then the columns have two different heights (see Traps and tips).

Letter contacts

Once the letters are in a grid of the right width, the columns are in the wrong order. Anagramming them uses **contacts**: pairs of letters that sit next to each other in English.

- **Q is followed by U.** This is the strongest contact there is.
- Common pairs (**digraphs**): TH, HE, IN, ER, AN, RE, ON, AT, EN, ND, ST, ES.
- Impossible or very rare pairs: QX, JZ, HQ, any consonant tripled.
- Rare letters (J, K, Q, V, X, Z) are excellent anchors because they have very few partners.

A worked example

You receive this ciphertext (40 letters):

```
TUTRANGA EQABEONM EENEGOIE TETITBTP MHEHDNRH
```

Step 1. Is it a transposition? Count the letters: E 8, T 6, N 4, A 3, H 3, and B, G, I, M, O, R twice each, then D, P, Q and U once. E and T are the most common, as in English. There is no J, X or Z. The vowels are A (3), E (8), I (2), O (2), U (1): 16 of 40, exactly 40 percent. The index of coincidence

is 0.078, close to English's 0.067 (short texts wobble). And there is one Q and one U. Everything says English letters, but nothing reads. It is a transposition.

Step 2. Which widths? The length is 40, so try widths 4, 5, 8 and 10. Start with 5.

Step 3. Cut into strips. For a columnar transposition the ciphertext is the columns one after another. With width 5, there are 8 rows, so cut the ciphertext into 5 strips of 8 letters. Call them A to E and stand them side by side.

A	B	C	D	E
T	E	E	T	M
U	Q	E	E	H
T	A	N	T	E
R	B	E	I	H
A	E	G	T	D
N	O	O	B	N
G	N	I	T	R
A	M	E	P	H

Step 4. Use the Q. Strip B has Q in row 2. The only strip with U in row 2 is A. So strip A goes immediately to the right of strip B. Reading B then A down the rows gives the pairs ET, QU, AT, BR, EA, ON, NG, MA. Several of those are common English pairs. Good sign.

Step 5. Grow the chain to the right. What comes after A? Try each remaining strip:

B A + C	B A + D	B A + E
ETE	ETT	ETM
QUE	QUI	QUH
ATN	ATT	ATH
BRE	BRI	BRH
EAG	EAT	EAD
ONO	ONB	ONN
NGI	NGT	NGR
MAE	MAP	MAH

QUH is impossible, which rules out E. Between C and D, D gives QUI, BRI, EAT and MAP, which look like pieces of words, so try D.

Step 6. Place the last two. C and E are left, and they go on the left. Try E then C, then C then E:

- E C B A D: MEETT, HEQUE, ENATT, HEBRI, DGEAT, NOONB, RINGT, HEMAP
- C E B A D: EMETT, EHQUE, … which puts H before Q, impossible.

So the order is E C B A D. Read the rows: `meett hequee natth ebrid geatn oonbr ingth emap`, which is "meet the queen at the bridge at noon bring the map".

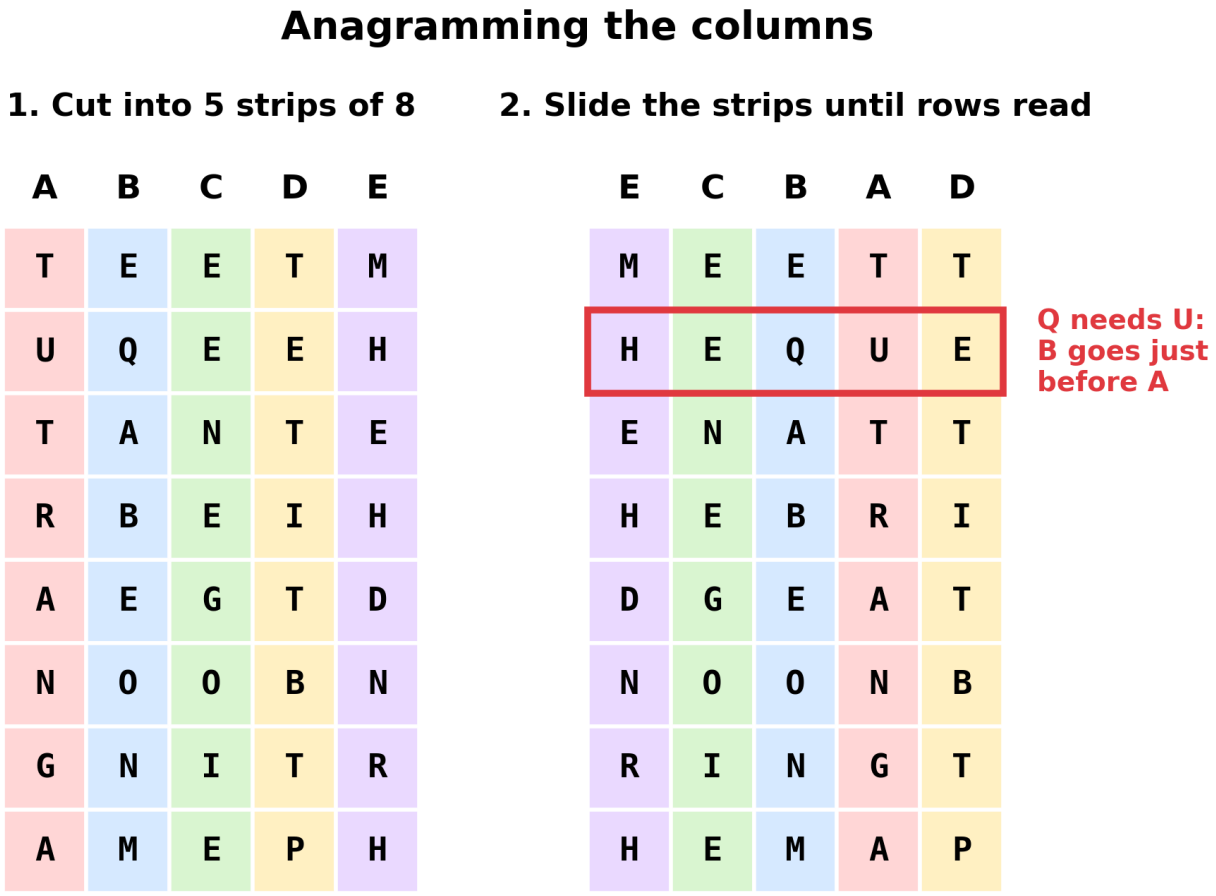


Figure 7.5: On the left, the ciphertext cut into five strips of eight letters labeled A to E; on the right, the strips slid into the order E C B A D so the rows read MEETT HEQUE ENATT HEBRI…, with the row containing Q and U outlined.

Step 7 (bonus). Recover the key. The strips were read in the order A, B, C, D, E, which were columns 4, 3, 2, 5, 1 of the plaintext grid. So the column numbers across the top were 5, 3, 2, 1, 4. A keyword with that alphabetical order is TIGER. You do not need the keyword to read the message, but finding it can help with the next message, which might use the same one.

> **In the app:** Columns writes your text into a grid of the width you choose; tap one column, then another, to swap them. Blocks does the same for short blocks, and Rail fence reads the text along a zigzag of the rails you choose.

Double transposition: a harder note

A **double transposition** does a columnar transposition and then another one on the result, often with a second keyword. The letter counts still match English, so the tells are the same. But after two scrambles, the strips no longer line up into simple rows, and anagramming by hand becomes very hard. Armies used it for exactly that reason, into the Second World War. To break it you usually need

a long message and a computer search over both keys at once, scored the way Guide 08 (Scoring and hill climbing) explains.

When to try it

- The letter counts look like English, but nothing reads.
- The ciphertext has a Q with no U next to it, or THE appears backward as EHT.
- The length has handy factors (40, 48, 60, 72…), especially if it ends in a few Xs.
- Reach for the rail fence first (only a few keys), then reversal, then columnar widths in order.

When not to: if E and T are not near the top of your counts, or Z or Q are suspiciously common, it is not a pure transposition. Try substitution first.

Traps and tips

- **Short texts wobble.** In 40 letters, E might come third instead of first. Look at the overall shape (common letters common, rare letters rare), not the exact order.
- **Incomplete grids.** If the last row was not filled, the leftmost columns of the plaintext grid are one letter longer than the rest. Then the strips are not all the same length. With 42 letters in width 5, two columns have 9 letters and three have 8: you must guess which strips are the long ones.
- **Filler letters.** Senders often pad the last row with X or other nulls. A few Xs in a row near the end of a decryption are padding, not a mistake.
- **Columnar or block?** Try both: cut the ciphertext into columns (vertical strips) and also into rows of the same width. Only one will anagram into English.
- **Use squared paper and scissors.** Write the strips on squared paper, cut them out and slide them around. It is much faster than rewriting.
- **Start from rare letters.** Q, J, X, Z and V have few partners, so they pin down neighbors quickly.
- **Word spaces are gone.** Real messages lose their spaces and punctuation before being scrambled, so read through the gaps: `MEETTHEQUEEN` is three words.

By hand or by computer

With transposition, the counts say what you face in seconds; the rearranging is where the hours go. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
List the possible widths from the length	2 minutes	Counts[2]	n/a

[1] Brain, pencil or computer?, the chapter before Guide 1.

[2] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English's, the IoC, the length and its factors.

Job	By hand	Learn with	Just do it
Write the text in a grid and move the columns	10 to 20 minutes a width	Columns[3], grid sheet[4]	dCode[5]
Try rail fences of 2 to 6 rails	about 5 minutes a rail count	Rail fence[6]	dCode[7]
Try every order of a block of 5	120 orders: hours	Blocks[8]	dCode[9]

Try it yourself

1. This message was written on 3 rails: `MMMIETETINGTEADH` (16 letters). How many letters are on each rail? What does it say?
2. A transposition ciphertext has 36 letters. Which widths should you try?
3. Is this a transposition? `TLLAA OLXBL LUHAA OLIYP KNLHA UVVUI YPUNA OLTHW`. (Hint: count E and L.)
4. This columnar transposition has width 3: `OUEHAONRETLKDTM`. Cut it into 3 strips of 5 and find the order that reads.

In the app: Practice with Columnar transposition, Block transposition and Rail fence.

Answers

1. The zigzag repeats every 4 letters. Rail 1 gets positions 1, 5, 9, 13 (4 letters), rail 3 gets positions 3, 7, 11, 15 (4 letters), and rail 2 the other 8. Split: `MMMI`, `ETETINGT`, `EADH`. Read the zigzag: “meet me at midnight”.
2. The factors of 36 are 2, 3, 4, 6, 9, 12 and 18. Try 4, 6 and 9 first, then 3 and 12.
3. No. There is no E at all, and L (rare-ish in English) appears 8 times in 40 letters. The letters themselves have been changed, so it is a substitution. (It is a Caesar shift of 7: L stands for E. It is the worked example’s message again.)
4. The strips are `OUEHA`, `ONRET` and `LKDTM`. Order 3, 1, 2 gives rows LOO, KUN, DER, THE, MAT: “look under the mat”.

Words to know

- **Transposition**: a cipher that keeps the letters but changes their order.
- **Substitution**: a cipher that changes the letters but keeps their order.

[3] Columns, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/columns. Any width; tap two columns to swap them.

[4] Columnar grid, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-columnar, to copy into Google Sheets or download for Excel.

[5] dCode’s columnar transposition solver: dcode.fr/columnar-transposition-cipher.

[6] Rail fence, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/rail.

[7] dCode’s rail fence solver: dcode.fr/rail-fence-cipher.

[8] Blocks, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/blocks. Rearranges every block the same way.

[9] dCode’s transposition solver: dcode.fr/transposition-cipher.

- **Reversal**: writing the message backward.
- **Rail fence**: writing the message in a zigzag across rails and reading each rail.
- **Columnar transposition**: writing in rows under a keyword and reading the columns in keyword order.
- **Block (permutation) transposition**: shuffling the letters inside each block of fixed size the same way.
- **Width (period)**: the number of columns in the grid, or the size of each block.
- **Factor (divisor)**: a number that divides another with nothing left over.
- **Contact**: two letters that sit next to each other in the text.
- **Digraph**: a pair of letters, such as TH.
- **Anagramming**: rearranging letters (or columns) until words appear.
- **Null (padding)**: a meaningless letter added to fill a grid.
- **Double transposition**: two transpositions, one after the other.

Going further

- Practical Cryptography: Rail fence cipher
- Practical Cryptography: Columnar transposition cipher
- Wikipedia: Transposition cipher (includes double transposition)
- Wikipedia: Rail fence cipher
- Simon Singh, *The Code Book* (1999), chapter 1, for transposition in history.
- The National Cipher Challenge for practice ciphertexts.

Guide 8

Scoring and hill climbing

Teaching a computer to ask: how English is this?

By the end you can explain exactly how a computer cracks a substitution cipher with no human help: how it scores a guess, how it improves the guess one small change at a time, and why it sometimes gets stuck.

The big idea

A simple substitution cipher has more possible keys than there are grains of sand on Earth, so nobody can try them all. But you do not need to try them all. You need two things. First, a **score**: a number that says how much a piece of text looks like English. Second, a way to **improve** a key: change it a tiny bit, and keep the change only if the score goes up. Repeat that a few thousand times and gibberish slowly turns into English. This is called **hill climbing**, because you are always walking uphill on the score. The picture below shows the score along a real run: the decryption starts as nonsense and ends as the real message.

Gibberish turns into English as the score climbs

	score	first 44 letters of the decryption
step 0	**-2111**	boiegzgkjoboeywidiihitovzdihbvzkvtscetcetbss
step 250	**-1595**	theikmkuchthirseyeeoenhamyeotamuangbinbintgg
step 500	**-1558**	theipmpychthirseueedenhamuedtamyankgingintkk
step 1000	**-1307**	theofdfichthouseleeperhadleptadiarkgorgortkk
step 1500	**-1238**	theobdbighthouseleeperhadleptadiaryforfortyy
step 2000	**-1205**	theoldlighthousekeeperhadkeptadiaryforfortyy
step 4000	**-1187**	theoldlighthousekeeperhadkeptadiaryforfortyy

Figure 8.1: As a computer swaps letters in its key and keeps only the swaps that raise the score, the decryption of a real substitution cipher turns from nonsense into “the old lighthouse keeper had kept a diary for forty years”.

Knowing how this works is the whole point of this guide. Codebreaking websites have a “solve” button that does exactly this. If you know what is inside the button, you know when to trust it, when it will fail, and how to fix it when it does. The method is the skill, not the button.

Which ciphers it breaks

Cipher family	Does scoring and hill climbing help?	How
Simple substitution, keyword substitution	Yes, this is its best use	The key is a scrambled alphabet. A move swaps two letters.
Columnar and block transposition	Yes	The key is a column order. A move swaps two columns or slides a run of them.
Playfair, four-square	Yes, with care	The key is a 5x5 square. Moves swap letters, rows or columns. Needs simulated annealing and 100+ letters.

Cipher family	Does scoring and hill climbing help?	How
Vigenère, Beaufort	It works, but there is a faster way	Find the key length first (Guide 04 and Guide 05), then each column is just a Caesar shift.
Caesar, affine	Not needed	Only 26 or 312 keys: just try them all and score each one.
Hill	Rarely	A crib solves it directly (Guide 09, Two letters at a time).
One-time pad	No	Every possible message is equally likely, so the score cannot pick the right one.

The score part is useful almost everywhere, even when you do not climb: any time you have a handful of candidate decryptions, scoring them tells you which one to read first.

Why it works

Step 1: what makes text “English”?

Single letter counts are a start (E is common, Z is rare), but they cannot tell `theoldlighthouse` from `ehtolhgidtlouseh`: both have exactly the same letters. What really marks English is which letters sit **next to each other**. TH, HE and ING are everywhere. QX and JZW never happen.

So instead of counting single letters, count groups of four letters in a row, called **quadgrams**. Take a few big books, slide a four-letter window along every sentence, and count how often each window appears. The table used in this guide was made from about 4.8 million quadgrams in public-domain novels. THAT, THER and WITH are near the top, along with NTHE and DTHE (the end of one word running into “the”). Most of the 456,976 possible quadgrams never appear at all.

To score a piece of text, slide the same window along it and look each quadgram up.



Figure 8.2: Every four-letter window of THEWINDBLEW gets a score between -3 and -5, adding up to -32.45. The same eleven letters shuffled, WTNHBIEDLWE, are mostly windows English never uses, each scoring -7.68, for a total of -56.37.

The English phrase scores about -32. The shuffled phrase, made of exactly the same letters, scores about -56. Higher (closer to zero) means more English. Now we need to explain why the numbers are negative, and why we **add** them.

Step 2: why add instead of multiply?

The quadgram THEW makes up about 0.00089 of all quadgrams in the books, roughly 9 in every 10,000. That number is its **probability**. To get the chance of a whole text, you would multiply the probabilities of all its quadgrams together. For our 297-letter message that means multiplying 294 small numbers. The answer has more than 1,100 zeros after the decimal point. Ordinary computer numbers cannot go below about 10^{-308} (a decimal point followed by 307 zeros and then a 1). Anything smaller silently becomes exactly 0, and then every key scores 0 and you can tell nothing apart.

The fix is an idea mathematicians use all the time: the **logarithm**, or **log** for short.

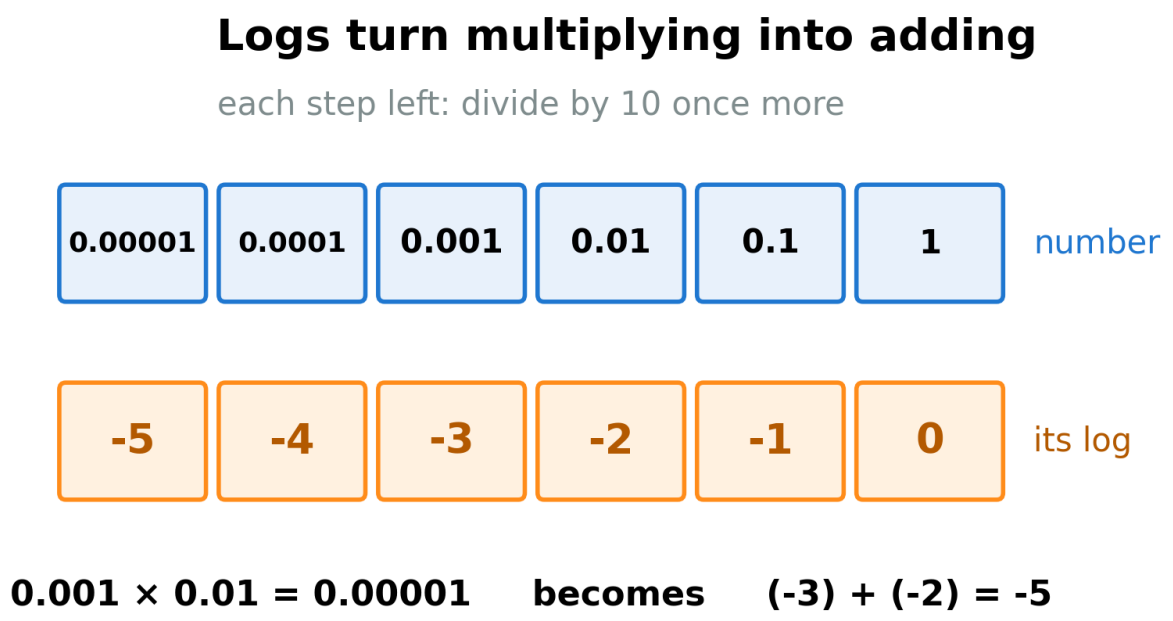


Figure 8.3: The numbers 1, 0.1, 0.01, 0.001 and so on, with their logs 0, -1, -2, -3 below them, and the rule that multiplying 0.001 by 0.01 is the same as adding -3 and -2.

Here is the whole idea. The log of a number counts **how many times you multiply by 10** to get it.

- $\log_{10} 1000 = 3$, because $10 \times 10 \times 10 = 1000$.
- $\log_{10} 1 = 0$, because you multiply by 10 zero times.
- For numbers smaller than 1, you divide instead, and the count goes negative: $\log_{10} 0.001 = -3$, because $1 \div 10 \div 10 \div 10 = 0.001$.
- Numbers in between have logs in between. THEW's probability, 0.00089, is a bit less than 0.001, so its log is a bit less than -3: it is -3.05.

The little 10 in $\log_{10}$ just says "we are counting tens". Now the magic: multiplying two numbers is the same as **adding** their logs. Try it: $0.001 \times 0.01 = 0.00001$. Count the tens: $(-3) + (-2) = -5$, and indeed $\log_{10} 0.00001 = -5$. In symbols:

$$\log_{10}(a \times b) = \log_{10} a + \log_{10} b$$

So instead of multiplying 294 tiny probabilities (and getting a number too small to store), we add 294 logs and get an ordinary number like -1187. Since every probability is less than 1, every log is negative, and the total is negative. A bigger probability means a log closer to zero, so **closer to zero is more English**.

One snag: a quadgram that never appeared in the books has probability 0, and $\log_{10} 0$ does not exist (you would have to divide by 10 forever). So we pretend each unseen quadgram appeared a tenth of a time. In our table that gives a floor of -7.68: about 1 chance in 48 million.

The grown-up version

The fitness score

Write the text as letters, and call its quadgrams $q_1, q_2, q_3, \ldots$ (the first four letters, then letters 2 to 5, and so on). A text of n letters has $n - 3$ of them. Write $p(q)$ for the probability of quadgram q in the big books. The **fitness** of the text is

$$F = \sum_{i=1}^{n-3} \log_{10} p(q_i)$$

The symbol $\sum$ is a capital Greek S, for "sum". It means: let i run from 1 up to $n - 3$, work out $\log_{10} p(q_i)$ each time, and add them all up. It is just a short way to write $\log_{10} p(q_1) + \log_{10} p(q_2) + \cdots + \log_{10} p(q_{n-3})$.

Longer texts have more terms, so their totals are bigger negative numbers. To compare texts of different lengths, divide by the number of quadgrams. English text averages about -4 per quadgram. Random letters average about -7.

Hill climbing

A **key** is whatever the cipher needs: for substitution, a scrambled alphabet saying which plain letter each cipher letter stands for. The **search space** is the set of all keys: for substitution there are $26! = 26 \times 25 \times 24 \times \cdots \times 2 \times 1$, about 4×10^{26} of them (the "!" is read "factorial"). A **move** is a small change to a key: here, swapping two letters.

1. Start with a random key. Decrypt with it and score the result.
2. Make a move: swap two letters of the key at random.
3. Decrypt and score again. If the score went **up**, keep the new key. If not, undo the swap.
4. Go back to step 2. Stop when thousands of moves in a row have not helped.

The final key is a **local maximum**: no single swap can improve it. If you are lucky it is the real key.

Local maxima and random restarts

Picture all possible keys spread out as a landscape, where the height of the ground is the score. Hill climbing is a walker in thick fog who can only feel the slope under their feet and only ever steps uphill.

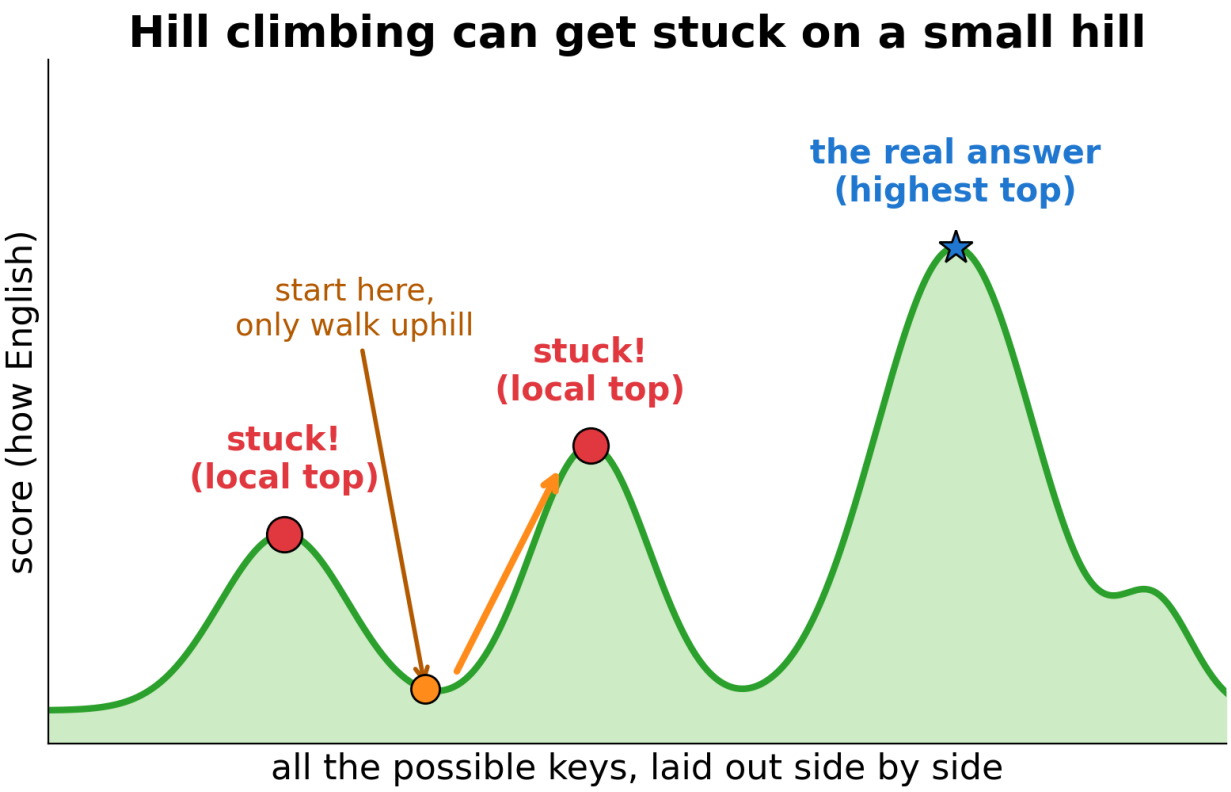


Figure 8.4: A landscape of scores over all possible keys, with two small hills marked “stuck” and one tall mountain marked “the real answer”. A climber who starts between the small hills and only walks uphill ends on a small hilltop.

Every hilltop feels like the top, because every step from it goes down. A small hilltop is a **local maximum**: a key that looks right “nearby” but decrypts to nonsense. The tallest peak, the real key, is the **global maximum**. The simplest cure is to **restart**: run the climb again from a different random key, several times, and keep the best result.

Simulated annealing

A smarter cure is to let the walker take a few downhill steps, especially early on, so they can walk off a small hill and find a bigger one. When blacksmiths **anneal** metal they heat it and let it cool slowly, so the atoms can shuffle into a strong arrangement before they freeze in place. **Simulated annealing** copies that.

It keeps a number called the **temperature**, T, which starts high and is lowered a little after every move. A better key is always kept. A worse key is sometimes kept, with a chance that depends on how much worse it is and on how hot it is. Write Δ (the Greek capital “delta”, meaning “change”) for the new score minus the old score. For a worse key Δ is negative, and the chance of keeping it is

$$P = 10^{\Delta/T}$$

For example, at $T = 30$ a move that is 30 points worse has $\Delta/T = -1$, so $P = 10^{-1} = 0.1$: it

is kept 1 time in 10. Later, at $T = 3$, the same move has $\Delta/T = -10$, so $P = 10^{-10}$: practically never. Hot means adventurous; cold means careful. By the end it behaves like plain hill climbing.

A worked example

Here is the start of a ciphertext made with a simple substitution (297 letters in all, spaces and punctuation removed).

```
ZITGS RSOUI ZIGXL TATTH TKIQR ATHZQ ROQKN YGKYG KZNNT QKLTC
TKNFO UIZIT VKGZT RGVFZ ITVTQ ZITKZ ITLIO HLZIQ ZHQLL TRQFR ...
```

Step 1. Score the ciphertext as it is. Read as if it were plain English, it scores -2160.2. That is -7.35 per quadgram: gibberish, as expected. The real message (which the computer does not know) scores -1186.9, or -4.04 per quadgram.

Step 2. Climb. The program starts from a random key (score -2110.5) and tries one random swap per iteration. Here is one real moment of the run, at iteration 1000, when the score is -1307.0 and the decryption begins like this:

```
theofdfichthouseleeperhadleptadiarkgorgort...
```

Some moves it might try:

Swap these plain letters	New score	Change	Kept?
k and y	-1288.3	+18.7	yes
g and f	-1286.7	+20.3	yes
l and k	-1302.9	+4.1	yes
e and t	-1764.0	-456.9	no

Swapping e and t would wreck every THE, so the score falls off a cliff. Swapping g and f turns `gorgort` into `forfort`, part of “for forty”, and the score jumps.

Step 3. Watch the score. The program ran 6 climbs (restarts), 4,000 iterations each. Two of them reached exactly -1186.9 and read `theoldlighthousekeeperhadkeptadiaryforfortyyears...`. The other four stopped on lower hills, between -1597 and -1734.

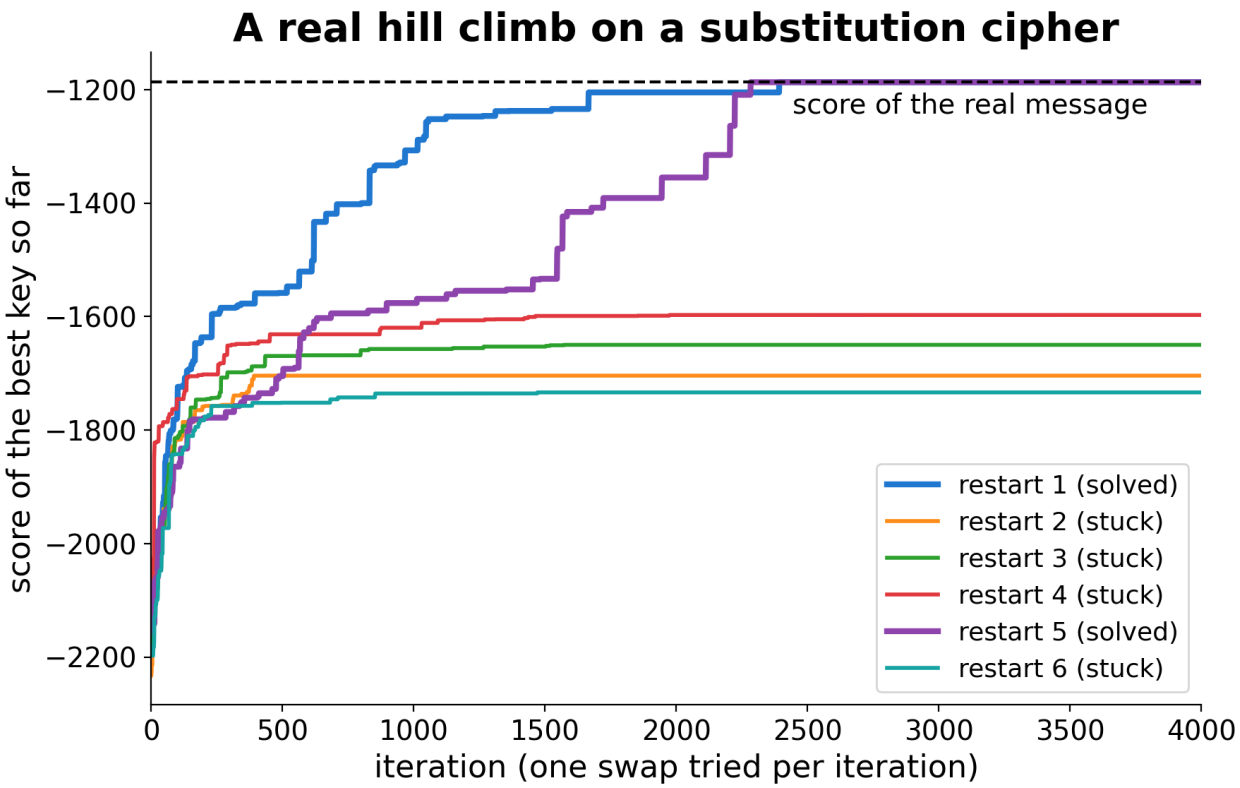


Figure 8.5: Score against iteration for six restarts of hill climbing on the same substitution ciphertext: two climb all the way to the score of the real message, about -1187, while four flatten out between about -1600 and -1730 and never improve again.

One of the stuck climbs ended on `ewaobtbhdwewoulacaapanwitcapeithingfonfon...`, scoring -1597.1. Look closely and you can see pieces of English (`ing`, `with`), but in the wrong places. No single swap fixes it: it would need several swaps at once, and each one alone makes the score worse. That is a local maximum.

Step 4. Try annealing. The same ciphertext, with simulated annealing (temperature lowered smoothly from 30 to 0.3 over 8,000 iterations), solved it in one go. In the picture you can see the score wobble down as well as up while it is hot.

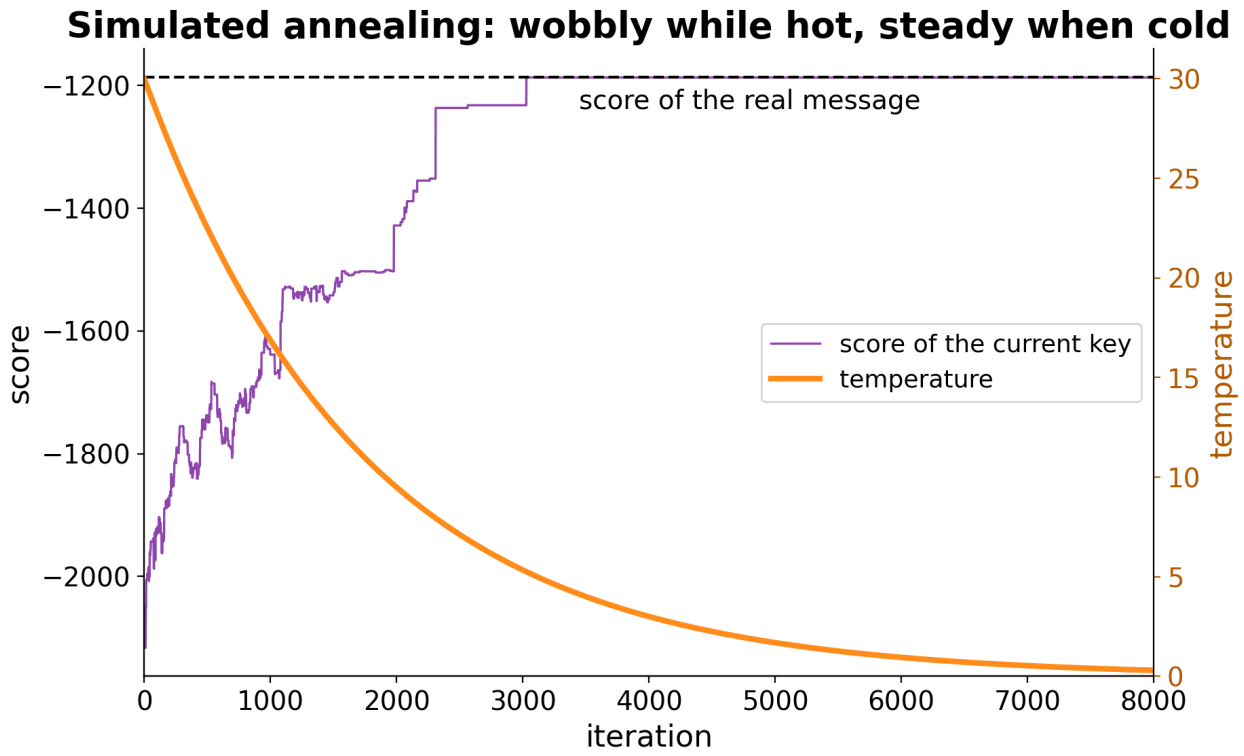


Figure 8.6: Simulated annealing on the same ciphertext: the score zigzags up and down early while the temperature (orange curve) is high, then settles and climbs to the score of the real message once the temperature drops.

To compare fairly, the program then gave each method 30 fresh random starts of 8,000 iterations. Plain hill climbing found the message 8 times out of 30. Simulated annealing found it 22 times out of 30. Neither is magic; both are just walkers in fog with different rules.

How it extends to other ciphers

The beautiful part is that only two things change from cipher to cipher: **what a key looks like** and **what a move is**. The score stays the same.

- **Columnar or block transposition.** The key is an order of columns, such as 3-1-4-0-2. Moves: swap two columns, or lift a run of neighboring columns and put it somewhere else (a run that is already in the right order should move as one piece). Try each possible width in turn (Guide 07, Transposition detective work).
- **Playfair.** The key is a 5x5 square of letters (Guide 09). Moves: swap two letters, swap two rows or two columns, or flip the square. The landscape is very bumpy, so annealing works much better than plain climbing, and it needs about 100 letters or more.
- **Anything new.** If someone invents a cipher you have never seen, ask: what is the key, and what is a small change to it? If you can answer both, you can climb.

In the app: The app has no hill climber: in Substitution you do by hand what it does. Try a change to the key, and keep it if more of the text reads as English.

When to try it

- You have identified a cipher with a key too big to try in full (substitution, transposition, Playfair).
- You have at least about 100 letters. Around 200 or more is comfortable.
- You have a few candidate decryptions and want the computer to rank them: score them all.

When not to: when the key space is small (just try every key), when a smarter method exists (IoC for Vigenère, a crib for Hill), and when the text is very short.

Traps and tips

- **Short texts fool the score.** With 40 letters, a wrong key can score better than the right one. The real message is the highest score only when there is enough text for the patterns to show.
- **Unusual text scores badly.** A message full of names, foreign words, or a deliberate trick (like never using the letter E) is less "English" than the books. The climber may then prefer a wrong key. If a solve is nearly readable, finish it by hand.
- **Nearly right is a great place to be.** A decryption like `theoldlichthouse` tells you exactly which swap to make next. Humans are very good at the last few letters.
- **Restart, restart, restart.** If the result is not readable, the climb probably stopped on a small hill. Run it again from a new random key before you suspect the cipher type.
- **Compare like with like.** Only compare totals for texts of the same length, or use the average per quadgram.
- **Strip everything but letters** before scoring, or make sure spaces and punctuation are handled the same way everywhere.
- **A score that never gets near English** (stuck around -6 per quadgram whatever you do) usually means the cipher type is wrong, not that you need more restarts.

By hand or by computer

This guide is about the methods computers use, so the honest answer is: do one or two steps by hand to see the idea, then let the program run thousands. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
Score one text with quadgrams	20 minutes or more	n/a	CrypTool 2[2]

[1] Brain, pencil or computer?, the chapter before Guide 1.
[2] CrypTool 2, a free learning environment with real attacks (Windows): cryptool.org/en/ct2.

Job	By hand	Learn with	Just do it
One hill-climbing step: swap two letters and rescore	5 to 10 minutes	Substitution[3], substitution sheet[4]	n/a
A full hill climb or simulated annealing	thousands of steps: impossible by hand	n/a	quipqiup[5], Guballa[6], CrypTool 2[7]

Try it yourself

1. Using the idea "count the tens": what is $\log_{10} 100$? What is $\log_{10} 0.0001$? Use logs to find $0.01 \times 0.001 \times 0.1$.
2. The quadgrams of THEWIND are THEW (-3.05), HEWI (-3.67), EWIN (-3.86) and WIND (-3.72). What is the fitness of THEWIND? What is its average per quadgram?
3. In simulated annealing with $P = 10^{\Delta/T}$: at temperature $T = 10$, what is the chance of keeping a move that is 20 points worse? What about at $T = 20$? At $T = 40$?
4. A friend runs a hill climber on a 30-letter substitution cipher and gets a perfectly readable but wrong answer. How is that possible?

In the app: Substitution without word breaks is the kind of text a hill climber is built for. Solve one by hand, and notice which steps you would hand to a computer.

Answers

1. $\log_{10} 100 = 2$ (two tens). $\log_{10} 0.0001 = -4$ (divide by 10 four times). For the product, add the logs: $(-2) + (-3) + (-1) = -6$, so the answer is 0.000001.
2. $-3.05 - 3.67 - 3.86 - 3.72 = -14.30$. There are 4 quadgrams, so the average is $-14.30 \div 4 = -3.575$, about -3.58: very English.
3. At $T = 10$: $\Delta/T = -20/10 = -2$, so $P = 10^{-2} = 0.01$, or 1 in 100. At $T = 20$: -1, so 1 in 10. At $T = 40$: -0.5, so $P = 10^{-0.5} \approx 0.32$, about 1 in 3.
4. A short text has few quadgrams, so many different keys can turn it into something English-looking. With only 30 letters there may be a wrong key whose decryption scores higher than the real message, and the climber has no way to know which one the sender meant. More text, or a crib, settles it.

Words to know

- **Quadgram**: a group of four letters in a row, such as TION.

[3] Substitution, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/substitution. You choose the letters; it fills in the text.

[4] Substitution key and partial decrypt, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-substitution, to copy into Google Sheets or download for Excel.

[5] quipqiup, a substitution solver: quipqiup.com.

[6] Guballa's substitution solver: guballa.de/substitution-solver.

[7] CrypTool 2, a free learning environment with real attacks (Windows): cryptool.org/en/ct2.

- **Fitness (score)**: a number saying how English a text looks; here, the sum of the logs of its quadgram probabilities. Closer to zero is better.
- **Probability**: how often something happens, as a fraction of all the times it could.
- **Logarithm (**$\log_{10}$**)**: how many times you multiply (or, if negative, divide) by 10 to get a number.
- **Key space (search space)**: the set of all possible keys.
- **Move**: a small change to a key, such as swapping two letters.
- **Hill climbing**: keep making moves, keeping only the ones that raise the score.
- **Local maximum**: a key that no single move can improve, but that is not the best key.
- **Random restart**: starting the climb again from a new random key.
- **Simulated annealing**: hill climbing that sometimes keeps worse keys, more often while the "temperature" is high.
- **Temperature**: the number in simulated annealing that controls how often worse moves are accepted. It starts high and falls.

Going further

- Video: Hill Climbing Algorithm & Artificial Intelligence, Computerphile, 7 minutes (the idea in general).
- Video: Basics of Cryptology, Part 8: Hill Climbing, Cryptography for Everybody (CrypTool), 22 minutes (for strong readers and adults).
- Practical Cryptography: quadgram statistics as a fitness measure (with a downloadable quadgram table)
- Practical Cryptography: breaking a substitution cipher by hill climbing
- Wikipedia: Hill climbing
- Wikipedia: Simulated annealing
- Wikipedia: Logarithm
- Simon Singh, *The Code Book* (1999), for the history of the ciphers these methods break.
- The National Cipher Challenge for practice ciphertexts of every kind.

Guide 9

Two letters at a time

Polybius, Playfair and Hill: ciphers that work on pairs

By the end you can recognize a Playfair or a Hill cipher just by counting, and you can break a Hill cipher from four letters you can guess.

The big idea

Most simple ciphers change one letter at a time, so E always turns into the same thing, and counting letters gives the game away. **Polygraphic** ciphers (“poly” means many, “graph” means letter) change letters in groups, usually pairs. Now E turns into different letters depending on its partner: the E in HE and the E in ER are encrypted differently. Counting single letters stops working. But pairs of letters have their own fingerprint, because some pairs are far more common than others.

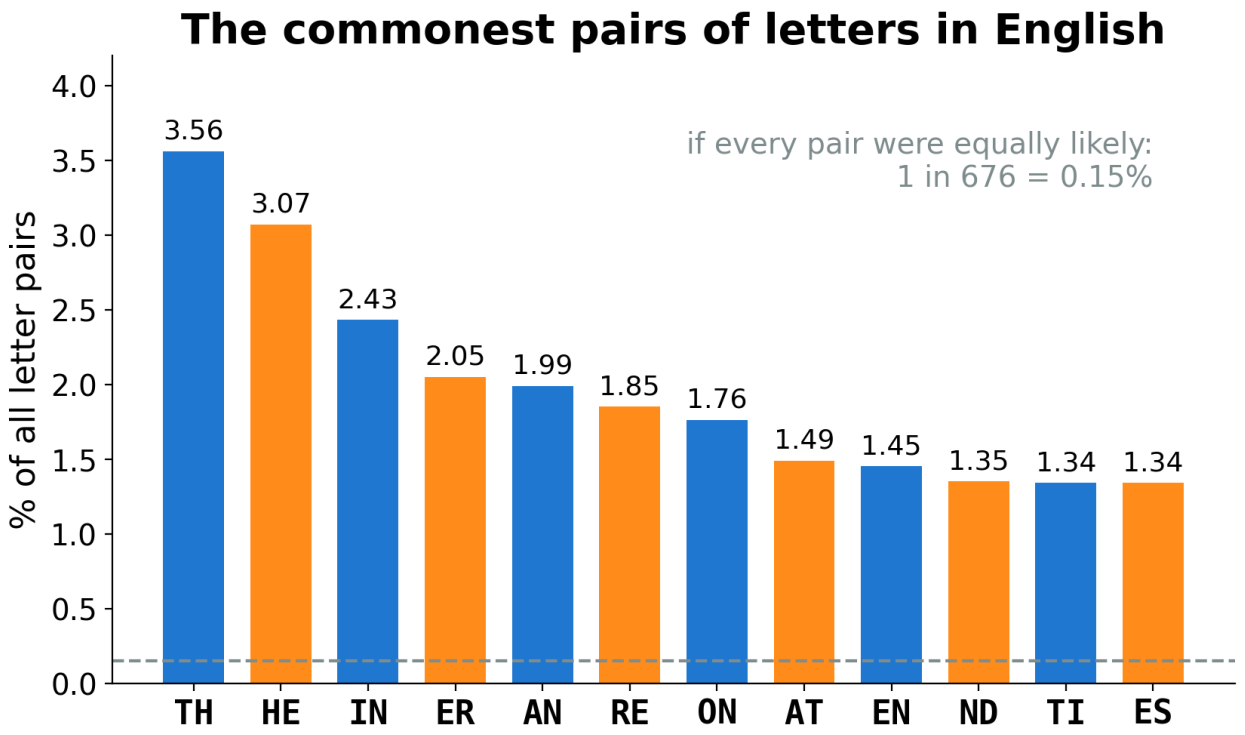


Figure 9.1: The twelve commonest pairs of letters in English, from TH (3.56 percent of all pairs) and HE (3.07 percent) down to TI and ES (1.34 percent each), compared with 0.15 percent, the share each pair would have if all 676 pairs were equally likely.

TH is more than twenty times as common as an average pair. A cipher that works on pairs is really a substitution on pairs instead of letters, and that is where you attack it. Each of these ciphers also leaves its own set of **tells**: small rules it can never break, which you can check by counting.

Which ciphers it breaks

Cipher	Do these methods help?	How
Polybius square	Yes	Each letter becomes two digits. Turn each pair back into one symbol, then use frequency analysis (Guide 02).
ADFGVX	Partly	A Polybius square followed by a columnar transposition: undo the transposition first (Guide 07), then the square.
Playfair	Yes	Check its tells. Then use a crib, reversed pairs, and pair frequencies, or a computer search (Guide 08).

Cipher	Do these methods help?	How
Four-square, two-square	Partly	Similar tells to Playfair, similar attacks.
Hill (2x2)	Yes	Check its tells. A four-letter crib gives four equations: solve them for the key.
Caesar, Vigenère, substitution	No	They work one letter at a time: see Guides 02 to 05.
Transposition	No	Letter counts look like plain English (Guide 07).

Why it works

The Polybius square: letters as coordinates

Write the alphabet in a 5x5 square. There are 26 letters and only 25 spaces, so I and J share a square (J is the rarest letter, and you can usually guess it back from the context). Number the rows and columns 1 to 5. Now every letter has an address: row first, then column.

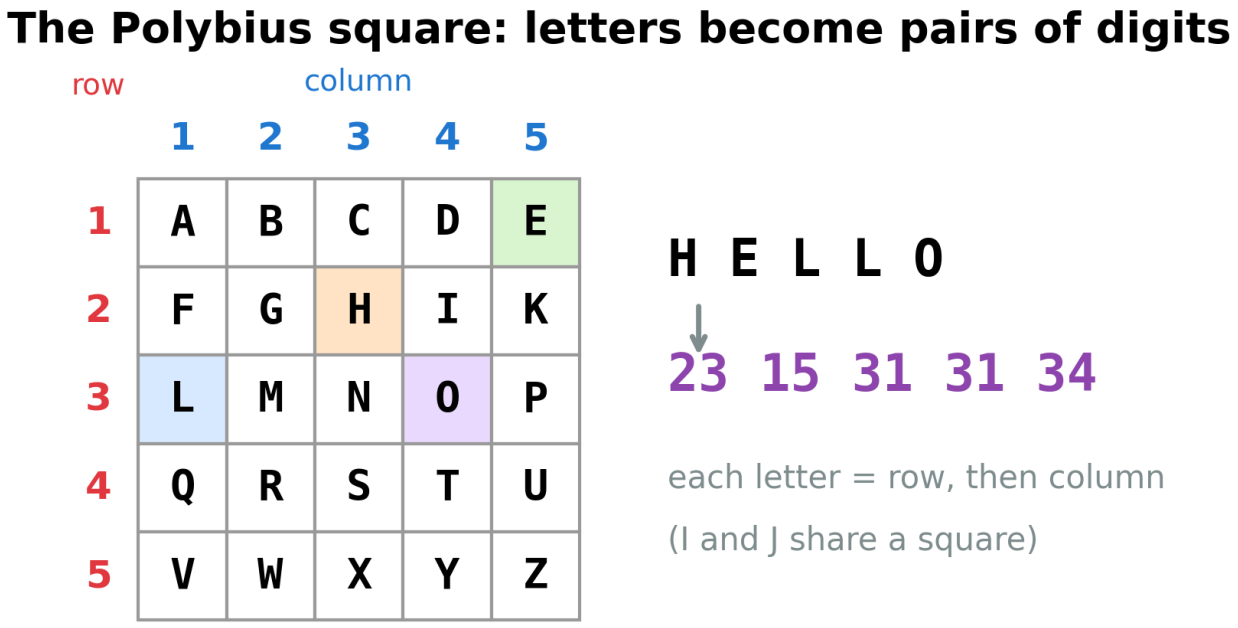


Figure 9.2: A 5x5 square of letters A to Z (I and J together) with rows numbered 1 to 5 in red and columns 1 to 5 in blue; HELLO becomes 23 15 31 31 34.

On its own, Polybius is just a substitution wearing a disguise: each letter always becomes the same pair of digits. Group the digits in pairs, give each pair a symbol, and count. The tells: only the digits 1 to 5, and an even number of them.

The square becomes important because it **hides inside other ciphers**. The **ADFGVX** cipher, used by

the German army in 1918, used a 6x6 square (26 letters plus 10 digits), with rows and columns labeled A, D, F, G, V and X instead of numbers. Those six letters were chosen because they sound very different in Morse code. Each letter became two of them, and then the whole string was scrambled by a columnar transposition. Tell: a ciphertext made of only the letters A, D, F, G, V and X. Other ciphers built on the square include bifid and the Nihilist cipher.

Playfair: a square that encrypts pairs

Playfair was invented in 1854 by Charles Wheatstone and named after his friend Lord Playfair, who promoted it. It was used by the British army in the Boer War and the First World War. It uses a 5x5 square too, but filled from a **keyword**. For the keyword LANTERN: write its letters without repeats (L, A, N, T, E, R), then the rest of the alphabet in order, skipping letters already used and merging J into I.

```
L A N T E
R B C D F
G H I K M
O P Q S U
V W X Y Z
```

Preparing the message. Take the letters only (J becomes I) and split them into pairs. A pair may never be two of the same letter, so if a pair would be doubled, put an X between them. If one letter is left over at the end, add an X. “The balloon will rise at noon” becomes:

```
TH EB AL LO ON WI LX LR IS EA TN OX ON
```

Look at WI LX LR: “will” has a double L, so an X goes between the Ls. In “noon”, the double O gets an X.

Encrypting each pair. There are three rules, depending on where the two letters sit in the square.

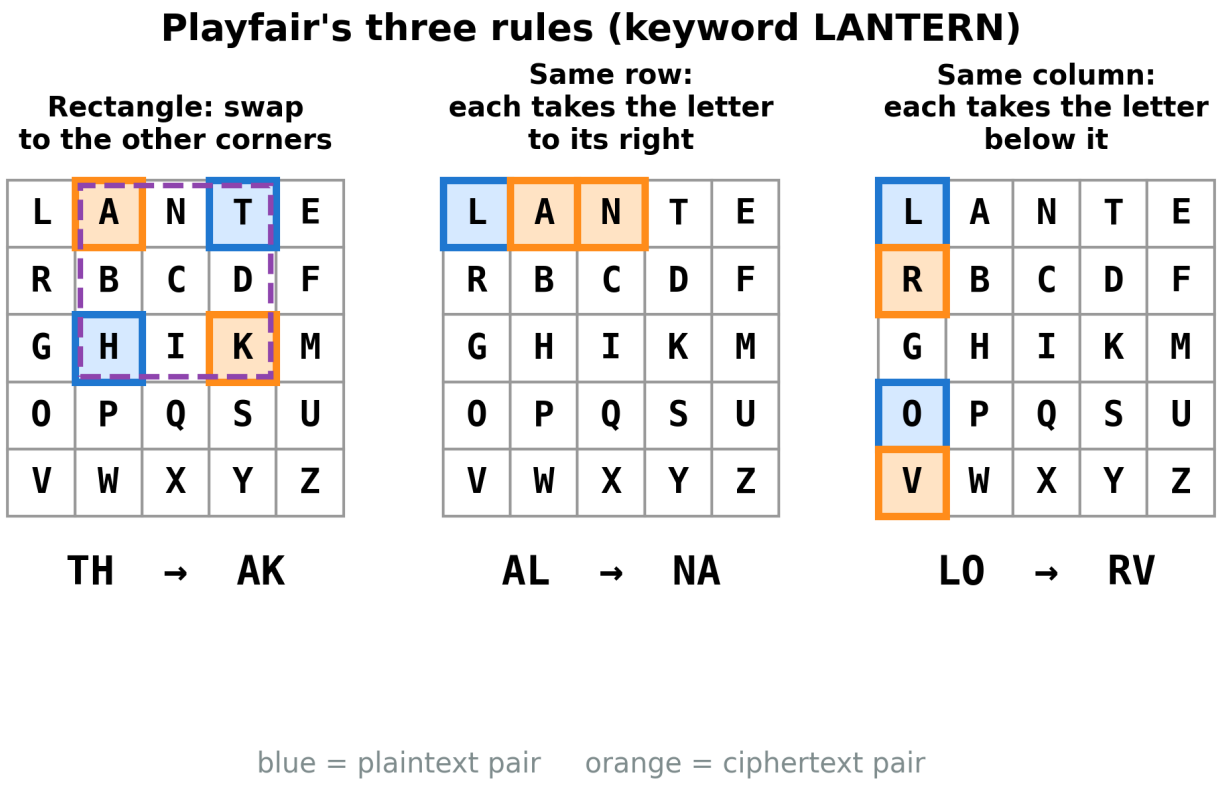


Figure 9.3: Three copies of the LANTERN square. Rectangle rule: T and H are corners of a rectangle, and each is replaced by the corner in its own row, giving AK. Same row rule: A and L each move one step right, giving NA. Same column rule: L and O each move one step down, giving RV.

1. **Rectangle.** If the letters are in different rows and columns, they mark two corners of a rectangle. Replace each letter with the other corner **in its own row**. TH becomes AK.
2. **Same row.** Replace each letter with the one to its right (wrapping from the end of the row back to the start). AL becomes NA.
3. **Same column.** Replace each letter with the one below it (wrapping from the bottom back to the top). LO becomes RV.

The whole message encrypts to `AK AF NA RV QL XH NV RG KQ LN ET QV QL`. To decrypt, do the opposite: left instead of right, up instead of down, and rectangles stay the same. The receiver gets `THEBALLOONWILXLRISEATNOXON` and simply ignores the extra Xs.

Spotting Playfair

The rules force some patterns, and they are the tells.

- **Even length.** The message always ends up in whole pairs.
- **No doubled pair.** Split the ciphertext into pairs from the start: no pair is ever two of the same letter (like `EE`), because the plaintext pairs never are, and the rules never turn two different letters into two equal ones.
- **At most 25 letters, usually no J.** J was merged into I.
- **No letter encrypts to itself.** In a pair, the first ciphertext letter is never the first plaintext letter, and the same for the second.

- **Reversed pairs.** If AB becomes XY, then BA becomes YX. In our message ON became QL, so NO would become LQ. English has common reversed pairs (ER and RE, ES and SE, ON and NO), so look for a ciphertext pair and its reverse that are both common.
- **Index of coincidence in between.** Pair encryption smooths out the counts a little: the index of coincidence (the chance that two letters picked at random match, Guide 04) is lower than English's 0.067, but higher than a Hill cipher's.

The same paragraph three ways: what you can count

	letters	even?	doubled pairs	J count	different letters	IoC
English	342	yes	9	1	24	0.067
Playfair	346	yes	**0**	**0**	**25**	0.057
Hill	342	yes	6	11	26	**0.041**

Figure 9.4: The same 342-letter paragraph as English, as Playfair and as Hill. All three have an even length. Playfair has 0 doubled pairs, 0 Js and only 25 different letters. Hill has 6 doubled pairs, 11 Js, all 26 letters, and an index of coincidence of 0.041, against 0.057 for Playfair and 0.067 for English.

Breaking Playfair. With a crib (a word you expect in the message), you can work out which letters share rows and columns, and rebuild the square bit by bit. Without one, the strongest method is a computer search: the key is the square, a move swaps two letters (or two rows, or two columns), and the score is the English score of the decryption. That is exactly the method in Guide 08 (Scoring and hill climbing), and Playfair needs its simulated annealing version and about 100 letters or more.

> **In the app:** The family tree has pages for the Polybius square and Playfair. Counts shows a text's length and every letter's count, so an odd length or a missing J stands out.

The grown-up version: the Hill cipher

The Hill cipher, invented by the mathematician Lester Hill in 1929, turns encryption into arithmetic. You need two ideas: multiplying a matrix by a pair of numbers, and clock arithmetic.

Letters as numbers

Number the letters from zero: A = 0, B = 1, C = 2, and so on up to Z = 25. Starting at zero makes the arithmetic tidy.

Matrices: rows times columns

A **matrix** is a box of numbers. The Hill key is a 2x2 matrix, four numbers in two rows and two columns. We write it with square brackets, and give the four numbers names:

$$K = \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$$

The pair of plaintext letters is written as a column of two numbers. To multiply the matrix by the column, take each **row** of the matrix and the **column** of letters, multiply the matching numbers, and add.

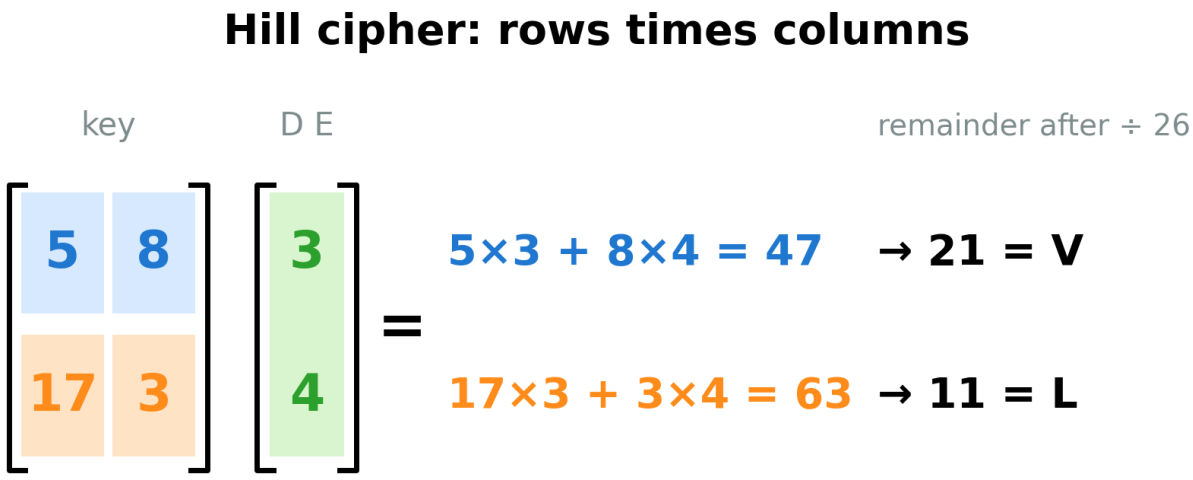


Figure 9.5: The key matrix with rows 5, 8 and 17, 3 times the column 3, 4 (the letters D and E). Top row: 5 times 3 plus 8 times 4 is 47, which leaves 21, the letter V. Bottom row: 17 times 3 plus 3 times 4 is 63, which leaves 11, the letter L.

$$\begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 5 \times 3 + 8 \times 4 \\ 17 \times 3 + 3 \times 4 \end{bmatrix} = \begin{bmatrix} 47 \\ 63 \end{bmatrix}$$

But 47 and 63 are not letters. We need to bring them back between 0 and 25.

Clock arithmetic: mod 26

Imagine a clock with 26 hours, one for each letter. Counting 47 hours around it, you go once all the way round (26) and then 21 more, landing on 21, which is V. Mathematicians write this as

$$47 \equiv 21 \pmod{26}$$

and read it “47 is congruent to 21, mod 26”. The symbol $\equiv$ means “lands on the same place on the clock”, and “mod 26” says the clock has 26 hours. In practice: **divide by 26 and keep the remainder**. $63 = 2 \times 26 + 11$, so $63 \equiv 11 \pmod{26}$, which is L. Negative numbers go backward round the clock: $-1 \equiv 25$, because one step back from 0 is 25.

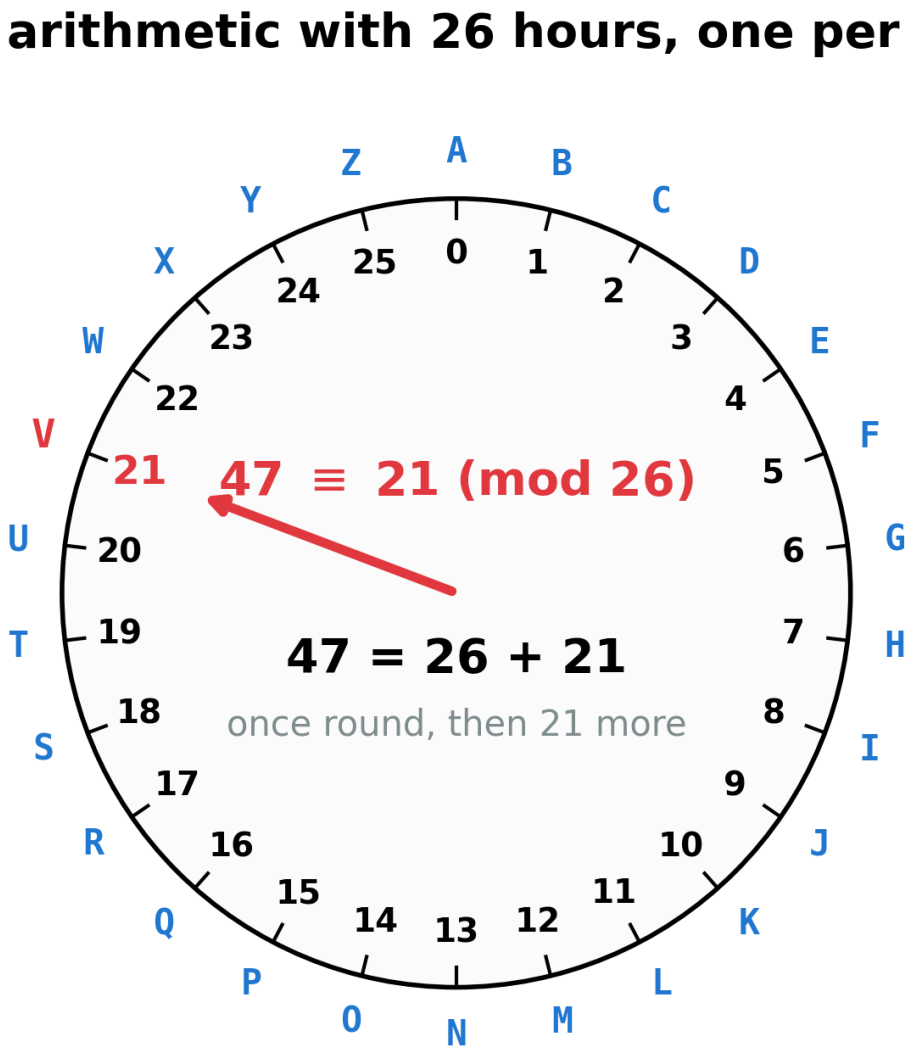


Figure 9.6: A clock face with 26 positions numbered 0 to 25 and labeled A to Z; an arrow points to 21, the letter V, showing that 47 is once round the clock and 21 more.

So the pair DE encrypts to VL. In symbols, with plaintext pair (p_1, p_2) and ciphertext pair (c_1, c_2):

$$c_1 \equiv a p_1 + b p_2 \pmod{26}, \qquad c_2 \equiv c\,p_1 + d\,p_2 \pmod{26}$$

Undoing it: inverses

To decrypt, you need a second matrix that undoes the first. On the 26-hour clock, "dividing" means multiplying by an **inverse**: the inverse of 3 is 9, because $3 \times 9 = 27 \equiv 1$. Only numbers that share no factor with 26 (so not even, and not 13) have an inverse. Here is the full list, worth copying onto a card:

number	1	3	5	7	9	11	15	17	19	21	23	25
inverse	1	9	21	15	3	19	7	23	11	5	17	25

For a matrix, the number that matters is its **determinant**, $ad - bc$. The key can be undone only if the determinant has an inverse. Then the undo matrix is

$$K^{-1} \equiv (ad - bc)^{-1} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \pmod{26}$$

(swap a and d, flip the signs of b and c, and multiply every entry by the inverse of the determinant).

Spotting Hill

- **Even length** (for a 2x2 key; a 3x3 key gives a multiple of 3).
- **Flat letter counts.** The index of coincidence is near 0.04, much flatter than English and flatter than Playfair.
- **All 26 letters can appear**, including J, and doubled pairs are allowed.
- **Repeats only in step.** The same plaintext pair, starting at an odd position (counting from 1), always gives the same ciphertext pair. So repeated pairs in the ciphertext line up on pair boundaries.

> **In the app:** The Hill cipher in the family tree, next to Playfair and the other ciphers that work on pairs.

A worked example: breaking Hill with a crib

You intercept this message, which you believe is a Hill cipher:

```
VL GZ WS UD NQ PB XV DH VF GZ YL MP VL ZL BP MP QV PB UP IK UT MS AP
```

Step 1. Check the tells. 46 letters: even. Doubled pairs are allowed in Hill, and repeats are on pair boundaries: VL appears twice, as do GZ, PB and MP. With only 46 letters the single-letter counts say little (P comes up 7 times, but that is mostly because PB and MP each appear twice), so the even length and the repeated pairs are the best clues. Everything fits Hill.

Step 2. Guess a crib. The message looks like a letter to an agent, and letters often begin with "dear". So guess that the first four letters DEAR became VLGZ. In numbers:

- D E is $(3, 4)$, and became V L, which is $(21, 11)$.
- A R is $(0, 17)$, and became G Z, which is $(6, 25)$.

Step 3. Write the equations. Use the formulas above, once for each crib pair. The top row of the key, a and b, makes the first letter of each ciphertext pair:

$$3a + 4b \equiv 21, \qquad 0a + 17b \equiv 6 \pmod{26}$$

The bottom row, c and d, makes the second letter of each pair:

$$3c + 4d \equiv 11, \qquad 0c + 17d \equiv 25 \pmod{26}$$

Step 4. Solve for b and d. Because A is 0, the second equation in each row has only one unknown. $17b \equiv 6$: multiply both sides by the inverse of 17, which is 23. $b \equiv 6 \times 23 = 138 = 5 \times 26 + 8$, so $b = 8$. Check: $17 \times 8 = 136 = 5 \times 26 + 6$. Yes. In the same way, $17d \equiv 25$ gives $d \equiv 25 \times 23 = 575 = 22 \times 26 + 3$, so $d = 3$. Check: $17 \times 3 = 51 = 26 + 25$. Yes. (If you don't have the inverse table, you can just try $b = 0, 1, 2, \ldots$ until $17b$ leaves remainder 6.)

Step 5. Solve for a and c. Put $b = 8$ into the first equation: $3a + 32 \equiv 21$, so $3a \equiv 21 - 32 = -11 \equiv 15$. Since $3 \times 5 = 15$, $a = 5$. Put $d = 3$ into the other: $3c + 12 \equiv 11$, so $3c \equiv -1 \equiv 25$. Multiply by 9 (the inverse of 3): $c \equiv 225 = 8 \times 26 + 17$, so $c = 17$. Check: $3 \times 17 = 51 \equiv 25$. Yes.

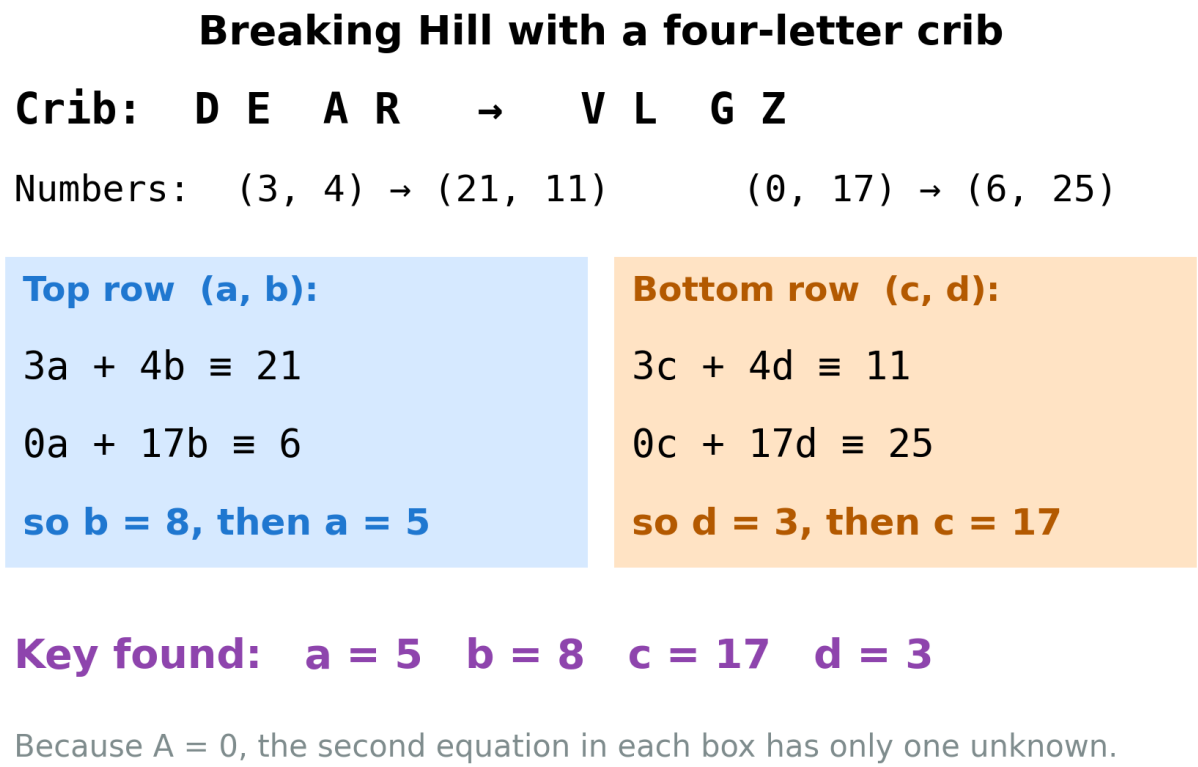


Figure 9.7: The crib DEAR, as the numbers (3, 4) and (0, 17), encrypted to VLGZ, the numbers (21, 11) and (6, 25). The top row's two equations give b = 8 and then a = 5; the bottom row's give d = 3 and then c = 17.

So the key is

$$K = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$$

Step 6. Build the undo matrix. The determinant is $ad - bc = 5 \times 3 - 8 \times 17 = 15 - 136 = -121$. On the clock, $-121 + 5 \times 26 = 9$, so the determinant is 9, and its inverse is 3. Swap a and d and flip the signs of b and c, then multiply by 3:

$$K^{-1} \equiv 3 \begin{bmatrix} 3 & -8 \\ -17 & 5 \end{bmatrix} = \begin{bmatrix} 9 & -24 \\ -51 & 15 \end{bmatrix} \equiv \begin{bmatrix} 9 & 2 \\ 1 & 15 \end{bmatrix} \pmod{26}$$

Step 7. Decrypt. The third pair is WS, which is $(22, 18)$:

$$9 \times 22 + 2 \times 18 = 234 = 9 \times 26 + 0 \equiv 0 = \text{A}, \qquad 1 \times 22 + 15 \times 18 = 292 = 11 \times 26 + 6 \equiv 6 = \text{G}$$

AG. The fourth pair UD, $(20, 3)$, gives $186 \equiv 4$ (E) and $65 \equiv 13$ (N): EN. Carrying on through the whole message gives `dear agent the papers are hidden inside the clock tower`. The crib was right.

What if the crib's place is unknown? Slide it along the ciphertext, starting only at odd positions (1, 3, 5, …) so it lines up with the pairs. Each position gives four equations. If they have no solution, or the key has no inverse, or the decryption is gibberish, move on.

The grown-up shortcut. Put the two crib pairs side by side as the columns of a matrix P, and the two ciphertext pairs as the columns of a matrix C. Then $C \equiv KP$, so $K \equiv CP^{-1}$. That is the same calculation as Steps 3 to 5, done all at once, and it works for any crib whose matrix P has an inverse.

When to try it

- **Polybius:** the ciphertext is only digits 1 to 5, or only five or six different letters (such as A, D, F, G, V, X).
- **Playfair:** even length, no doubled pair at the pair boundaries, no J, index of coincidence a bit below English.
- **Hill:** even length (or a multiple of 3), flat counts, J and doubled pairs present, and a good guess for a word in the message.

When not to: an odd length rules out Playfair and 2x2 Hill. A doubled pair at a pair boundary rules out Playfair. Letter counts that look exactly like English mean a transposition (Guide 07).

Traps and tips

- **Count pairs from the very first letter.** The tells are about pairs 1-2, 3-4, 5-6, and so on. A doubled letter across a boundary (the 2nd and 3rd letters, say) means nothing.
- **Short texts.** In 30 letters, a substitution cipher might have no doubled pair just by luck. The no-doubles tell gets convincing at around 80 letters or more.

- **The Xs.** After decrypting Playfair, remove the X between doubled letters and at the end. But some messages really contain X (as in “box”), so read carefully.
- **I or J?** A Playfair or Polybius decryption shows I where the sender wrote J: “IUST” is “just”.
- **Check every step on the clock.** One slip in a remainder ruins the whole Hill key. After solving, re-encrypt the crib with your key to check it gives the ciphertext.
- **Even determinants are impossible.** If your Hill key comes out with an even determinant (or 13), it cannot be a real key: the crib or its position is wrong.
- **Choose cribs with an A.** Since A = 0, a crib containing A makes some equations very simple, as in the worked example.

By hand or by computer

The tells of a pair cipher take minutes to check; breaking one without a crib is a job for a program. Do each job by hand once, on a short text, so you know what it involves; then hand it over (the chapter *Brain, pencil or computer?* explains why[1]). Times are rough, for a practiced reader and a ciphertext of about 300 letters.

Job	By hand	Learn with	Just do it
Check the tells: even length, no doubled pair, no J	5 minutes	Counts[2], pairs sheet[3]	Boxentriq[4]
Count the letter pairs	30 minutes or more	pairs sheet[5]	dCode[6]
Solve a 2 x 2 Hill cipher from a crib	15 to 20 minutes, with care	Hill sheet[7]	dCode[8]
Break a Playfair with no crib	hours, if at all	n/a	dCode[9], CrypTool 2[10]

Try it yourself

1. Decode this Polybius message (standard square, as in the picture): `22 34 34 14 31 45 13 25`.
2. Which of these could be Playfair? (a) `QWERTYUIOPLKJ` (b) `MNBVCCXZLKAS` (c) `HGFDSAPOIUYT`.
3. Encrypt the pair HI with the key $K = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$.

[1] Brain, pencil or computer?, the chapter before Guide 1.

[2] Counts, in the cryptology app: apps.thorwhalen.com/cryptology/#/workbench/counts. Letter counts beside English’s, the IoC, the length and its factors.

[3] Letter pairs and the Playfair tells, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-digraphs, to copy into Google Sheets or download for Excel.

[4] Boxentriq’s cipher identifier: boxentriq.com/code-breaking/cipher-identifier.

[5] Letter pairs and the Playfair tells, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-digraphs, to copy into Google Sheets or download for Excel.

[6] dCode’s frequency analysis: dcode.fr/frequency-analysis.

[7] Hill 2 x 2: determinant, inverse and decryption mod 26, a spreadsheet: apps.thorwhalen.com/cryptology/#/tools/sheet-hill, to copy into Google Sheets or download for Excel.

[8] dCode’s Hill solver: dcode.fr/hill-cipher.

[9] dCode’s Playfair solver: dcode.fr/playfair-cipher.

[10] CrypTool 2, a free learning environment with real attacks (Windows): cryptool.org/en/ct2.

4. With the keyword LANTERN, ON encrypts to QL. Without using the square, what does NO encrypt to?
5. Can these be Hill keys? (a) $\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$ (b) $\begin{bmatrix} 3 & 2 \\ 5 & 7 \end{bmatrix}$

Answers

1. 22 = G, 34 = O, 34 = O, 14 = D, 31 = L, 45 = U, 13 = C, 25 = K: “good luck”.
2. (a) No: it has 13 letters (odd), and it contains a J. (b) No: split into pairs, MN BV CC XZ LK AS, and CC is a doubled pair. (c) Yes, it could be: 12 letters, pairs HG FD SA PO IU YT, no doubled pair, no J.
3. H = 7, I = 8. Top: $5 \times 7 + 8 \times 8 = 99 = 3 \times 26 + 21$, so 21 = V. Bottom: $17 \times 7 + 3 \times 8 = 143 = 5 \times 26 + 13$, so 13 = N. HI becomes VN.
4. LQ: reversing a plaintext pair reverses its ciphertext pair.
5. (a) No: the determinant is $2 \times 3 - 4 \times 1 = 2$, which is even, so it has no inverse. (b) Yes: $3 \times 7 - 2 \times 5 = 11$, and 11 has an inverse (19).

Words to know

- **Polygraphic cipher**: a cipher that encrypts groups of letters (usually pairs) together.
- **Digraph**: a pair of letters, such as TH.
- **Polybius square**: a 5x5 grid that turns each letter into a row number and a column number.
- **ADFGVX**: a 6x6 Polybius square labeled A, D, F, G, V, X, followed by a columnar transposition.
- **Playfair**: a cipher that encrypts pairs with a keyword square, using the rectangle, row and column rules.
- **Hill cipher**: a cipher that multiplies pairs of letter numbers by a matrix, mod 26.
- **Matrix**: a rectangular box of numbers.
- **Mod 26 (modular arithmetic)**: arithmetic on a 26-hour clock: keep only the remainder after dividing by 26.
- **Congruent ($\equiv$)**: landing on the same place on the clock.
- **Inverse (mod 26)**: the number you multiply by to get 1 on the clock.
- **Determinant**: for a 2x2 matrix, $ad - bc$; the key can be undone only if this has an inverse mod 26.
- **Crib**: a word you guess is in the plaintext.
- **Tell**: a pattern a cipher always or never produces, which helps you recognize it.

Going further

- Video: Break a Playfair Cipher, Cryptography for Everybody (CrypTool), 14 minutes.
- Practical Cryptography: Playfair cipher
- Practical Cryptography: Hill cipher
- Practical Cryptography: ADFGVX cipher
- Wikipedia: Polybius square
- Wikipedia: Playfair cipher
- Wikipedia: Hill cipher
- Wikipedia: Modular arithmetic
- Simon Singh, *The Code Book* (1999), for Playfair and ADFGVX in war.

- The National Cipher Challenge for practice ciphertexts.

Words to know, A to Z

Every word the guides teach, in one list. The numbers after each one say which guides explain it.

- **ADFGVX**: a 6x6 Polybius square labeled A, D, F, G, V, X, followed by a columnar transposition. *(Guide 9)*
- **Affine cipher**: $E(x) = ax + b \bmod 26$, with a coprime to 26. *(Guide 3)*
- **Anagramming**: rearranging letters (or columns) until words appear. *(Guide 7)*
- **Autokey**: a cipher whose key is a short primer followed by the message itself, so the key never repeats. *(Guide 5)*
- **Beaufort cipher**: a repeating-key cipher with the rule cipher = key minus plain; it is its own inverse. *(Guide 5)*
- **Block (permutation) transposition**: shuffling the letters inside each block of fixed size the same way. *(Guide 7)*
- **Bombe**: the machine designed by Alan Turing at Bletchley Park that used cribs to find Enigma settings. *(Guide 6)*
- **Brute force**: trying every possible key. *(Guide 3)*
- **Chi-squared (χ^2)**: a score of how far a set of counts is from English; lower means more English-like. *(Guide 2)*
- **Ciphertext**: the scrambled message. **Plaintext**: the original message. *(Guide 1)*
- **Columnar transposition**: writing in rows under a keyword and reading the columns in keyword order. *(Guide 7)*
- **Column test**: splitting a ciphertext into L columns and averaging their IoCs to test whether the key length is L. *(Guide 4)*
- **Congruent ($\equiv$)**: landing on the same spot on the clock; $28 \equiv 2 \pmod{26}$. *(Guides 3, 9)*
- **Contact**: two letters that sit next to each other in the text. *(Guide 7)*
- **Coprime**: sharing no factor except 1; $\gcd(a, 26) = 1$. *(Guide 3)*
- **Crib dragging**: sliding a crib along a ciphertext, subtracting at each position, and looking for a readable piece of key. *(Guide 6)*
- **Crib (or** probable word**)**: a word or phrase you expect to find in the plaintext. *(Guides 6, 9)*
- **Determinant**: for a 2x2 matrix, $ad - bc$; the key can be undone only if this has an inverse mod 26. *(Guide 9)*
- **Digraph**: a pair of letters, such as TH. *(Guides 7, 9)*
- **Double transposition**: two transpositions, one after the other. *(Guide 7)*
- **Encoding**: a fixed way of rewriting letters (Morse, binary) with no secret key. *(Guide 1)*
- **ETAOIN SHRDLU**: the approximate order of the twelve commonest letters in English. *(Guide 2)*
- **Euclid's algorithm**: a fast way to find the gcd by taking remainders over and over. *(Guide 5)*
- **Expected count**: how many times a letter would appear in this many letters of average English. *(Guide 2)*
- **Factor**: a whole number that divides another exactly; 5 is a factor of 165. *(Guides 1, 5, 7)*
- **Factorial ($n!$)**: $n \times (n-1) \times \cdots \times 1$; $26!$ counts the scrambled alphabets. *(Guide 3)*

- **Factor table**: a grid that shows which small numbers divide which gaps. *(Guide 5)*
- **Fitness (score)**: a number saying how English a text looks; here, the sum of the logs of its quadgram probabilities. Closer to zero is better. *(Guide 8)*
- **Frequency**: how often a letter appears, as a count or a percentage. *(Guides 1, 2, 4)*
- **Frequency analysis**: breaking a cipher by comparing its letter counts with those of the language. *(Guide 2)*
- **Friedman's estimate**: a formula that guesses the key length from the IoC of the whole message. *(Guide 4)*
- **Gap (or distance)**: how many letters separate the start of one copy of a repeat from the start of the next. *(Guide 5)*
- **Greatest common divisor (gcd)**: the largest number that divides every number in a list. *(Guide 5)*
- **Hill cipher**: a cipher that multiplies pairs of letter numbers by a matrix, mod 26. *(Guide 9)*
- **Hill climbing**: keep making moves, keeping only the ones that raise the score. *(Guide 8)*
- **Index of coincidence (IoC)**: the chance that two letters picked at random from a text are the same; about 0.067 for English and 0.038 for random letters. *(Guides 1, 4)*
- **Inverse (mod 26)**: the number you multiply by to get 1 on the clock. *(Guide 9)*
- **Kappa (κ)**: the IoC you expect from a language or from random letters: about 0.066 for English and $1/26 \approx 0.038$ for random letters. *(Guide 4)*
- **Kasiski examination**: finding repeated groups in a ciphertext and using the gaps between them to find the key length. *(Guide 5)*
- **Key**: the secret setting of a cipher that the sender and receiver share. *(Guide 3)*
- **Key fragment**: the piece of key you get by subtracting a crib from the ciphertext. *(Guide 6)*
- **Key space**: the set of all possible keys; its size is the number of keys. *(Guides 3, 8)*
- **Keyword alphabet**: a cipher alphabet made from a keyword (repeats removed) followed by the rest of the alphabet in order. *(Guide 6)*
- **Keyword substitution**: a substitution whose cipher alphabet starts with a keyword, followed by the unused letters in order. *(Guide 2)*
- **Local maximum**: a key that no single move can improve, but that is not the best key. *(Guide 8)*
- **Logarithm ($\log_{10}$)**: how many times you multiply (or, if negative, divide) by 10 to get a number. *(Guide 8)*
- **Matrix**: a rectangular box of numbers. *(Guide 9)*
- **Mod 26 (modular arithmetic)**: arithmetic on a 26-hour clock: keep only the remainder after dividing by 26. *(Guide 9)*
- **Modular arithmetic**: arithmetic on a clock; mod 26, after 25 comes 0. *(Guide 3)*
- **Modulo 26 ($\bmod 26$)**: wrapping round the alphabet, so that 26 counts as 0 again. *(Guide 6)*
- **Monoalphabetic**: using one cipher alphabet throughout. **Polyalphabetic**: switching between several. *(Guides 1, 4)*
- **Monoalphabetic substitution**: a cipher that replaces each letter with another, the same way every time. *(Guide 2)*
- **Move**: a small change to a key, such as swapping two letters. *(Guide 8)*
- **Multiple**: a number you get by multiplying a number by a whole number (12 is a multiple of 6). *(Guide 5)*
- **Multiplicative inverse (a^{-1})**: the number that multiplies with a to give 1 mod 26. *(Guide 3)*
- **Null (padding)**: a meaningless letter added to fill a grid. *(Guide 7)*
- **Padding**: extra letters added to make the length fit a grid. *(Guide 1)*
- **Pattern list**: a list of common words grouped by their patterns. *(Guide 6)*
- **Period or key length**: how many letters the key has before it repeats. *(Guide 4)*
- **Playfair**: a cipher that encrypts pairs with a keyword square, using the rectangle, row and column

rules. *(Guide 9)*

- **Polyalphabetic**: switching between several cipher alphabets (Vigenère, Beaufort). *(Guide 4)*
- **Polybius square**: a 5x5 grid that turns each letter into a row number and a column number. *(Guide 9)*
- **Polygraphic cipher**: a cipher that encrypts groups of letters (usually pairs) together. *(Guide 9)*
- **Primer**: the short keyword that starts an autokey key. *(Guide 5)*
- **Probability**: how often something happens, as a fraction of all the times it could. *(Guide 8)*
- **Quadgram**: a group of four letters in a row, such as TION. *(Guide 8)*
- **Rail fence**: writing the message in a zigzag across rails and reading each rail. *(Guide 7)*
- **Random restart**: starting the climb again from a new random key. *(Guide 8)*
- **Reversal**: writing the message backward. *(Guide 7)*
- **Sigma ($\sum$)**: a sign that means “add up all of these”. *(Guide 4)*
- **Simulated annealing**: hill climbing that sometimes keeps worse keys, more often while the “temperature” is high. *(Guide 8)*
- **Substitution**: a cipher that changes the letters but keeps their order. *(Guide 7)*
- **Tell**: a pattern a cipher always or never produces, which helps you recognize it. *(Guide 9)*
- **Temperature**: the number in simulated annealing that controls how often worse moves are accepted. It starts high and falls. *(Guide 8)*
- **Transposition**: a cipher that moves letters without changing them. **Substitution**: a cipher that changes letters without moving them. *(Guides 1, 7)*
- **Trigram**: a group of three letters in a row. *(Guide 5)*
- **Width (period)**: the number of columns in the grid, or the size of each block. *(Guide 7)*
- **Word pattern**: the shape of a word’s repeated letters, written with A, B, C… in order of first appearance (`that` is ABCA). *(Guide 6)*

Where to go next

- **The cryptology app**, apps.thorwhalen.com/cryptology: the tools and exercises the green boxes point to, the family tree of ciphers, learning paths, and the competitions below with their calendars and rules.
- **The National Cipher Challenge** (UK), cipherchallenge.org: a story told in weekly ciphers each autumn, for school teams. Its archive of past challenges is the best practice there is.
- **Concours Alkindi** (France), concours-alkindi.fr: a team competition for 13 to 16 year olds, in French.
- **Science Olympiad Codebusters** (US), toebes.com/codebusters: timed tests on many cipher types, with practice tests.
- **Simon Singh, *The Code Book***: the history of codes and codebreaking, from Mary Queen of Scots to Enigma and beyond, told as a series of stories.
- **Practical Cryptography**, practicalcryptography.com: short, clear pages on each cipher and how to break it, with the statistics of English.

About this book

The nine guides were written to be read on their own, as PDFs, and are collected here as one book. Every worked example was checked by running code, and every figure was drawn by a short Python program that comes with the source.

The text and figures are licensed under Creative Commons Attribution 4.0: you may copy, share and adapt them, for any purpose, as long as you say where they came from. The source, the figure programs and every guide as its own PDF are at github.com/thorwhalen/cryptology. The cover is an illustration made with an AI image tool.

Typeset with pandoc and XeLaTeX. Figures drawn with matplotlib.

First edition, November 2026.